



UNIVERSITY OF PATRAS

**DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING**

**DIVISION OF ELECTRONICS AND COMPUTER
LABORATORY OF THE SUPERVISOR**

**IMPLEMENTATION OF THE POST QUANTUM
CRYPTOGRAPHY ALGORITHM CRYSTALS-DILITHIUM ON
FPGA USING HLS**

DIPLOMA THESIS

SEMERTZAKIS MARIOS

SUPERVISOR: THEODORIDIS GEORGE

PATRAS – SEPTEMBER, 2023

University of Patras, Department of Electrical and Computer Engineering.

MARIOS SEMERTZAKIS

© 2023 – All rights reserved

The whole work is an original work, produced by Marios Semertzakis, and does not violate the rights of third parties in any way. If the work contains material which has not been produced by him/her, this is clearly visible and is explicitly mentioned in the text of the work as a product of a third party, noting in a similarly clear way his/her identification data, while at the same time confirming that in case of using original graphics representations, images, graphs, etc., has obtained the unrestricted permission of the copyright holder for the inclusion and subsequent publication of this material.

CERTIFICATION

It is certified that the Diploma Thesis titled

IMPLEMENTATION OF THE POST QUANTUM CRYPTOGRAPHY ALGORITHM CRYSTALS-DILITHIUM ON FPGA USING HLS

of the Department of Electrical and Computer Engineering student

SEMERTZAKIS MARIOS

Registration Number: 1053540

was presented publicly at the Department of Electrical and Computer
Engineering at

09/10/2023

and was examined by the following examining committee:

Theodoridis George, Associate Professor, ECE (supervisor)

Paliouras Vasilis, Professor, ECE (committee member)

Serpanos Dimitrios, Professor, ECE (committee member)

The Supervisor

The Director of the Division

Theodoridis George
Associate Professor

Theodoridis George
Associate Professor



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

ΤΟΜΕΑΣ ΗΛΕΚΤΡΟΝΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΤΗΡΙΟ ΣΧΕΔΙΑΣΜΟΥ ΟΛΟΚΛΗΡΩΜΕΝΩΝ ΚΥΚΛΩΜΑΤΩΝ

**ΥΛΟΠΟΙΗΣΗ ΤΟΥ ΜΕΤΑΚΒΑΝΤΙΚΟΥ
ΚΡΥΠΤΟΓΡΑΦΙΚΟΥ ΑΛΓΟΡΙΘΜΟΥ CRYSTALS-DILITHIUM
ΣΕ FPGA ΜΕ ΧΡΗΣΗ HLS**

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΣΕΜΕΡΤΖΑΚΗΣ ΜΑΡΙΟΣ

ΕΠΙΒΛΕΠΩΝ: ΘΕΟΔΩΡΙΔΗΣ ΓΕΩΡΓΙΟΣ

ΠΑΤΡΑ – ΣΕΠΤΕΜΒΡΙΟΣ, 2023

Πανεπιστήμιο Πατρών, Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών.

ΜΑΡΙΟΣ ΣΕΜΕΡΤΖΑΚΗΣ

© 2023 – Με την επιφύλαξη παντός δικαιώματος

Το σύνολο της εργασίας αποτελεί πρωτότυπο έργο, παραχθέν από τον/την Μάριο Σεμερτζάκη, και δεν παραβιάζει δικαιώματα τρίτων καθ' οιονδήποτε τρόπο. Αν η εργασία περιέχει υλικό, το οποίο δεν έχει παραχθεί από τον/την ίδιο/α, αυτό είναι ευδιάκριτο και αναφέρεται ρητώς εντός του κειμένου της εργασίας ως προϊόν εργασίας τρίτου, σημειώνοντας με παρομοίως σαφή τρόπο τα στοιχεία ταυτοποίησής του, ενώ παράλληλα βεβαιώνει πως στην περίπτωση χρήσης αυτούσιων γραφικών αναπαραστάσεων, εικόνων, γραφημάτων κ.λπ., έχει λάβει τη χωρίς περιορισμούς άδεια του κατόχου των πνευματικών δικαιωμάτων για την συμπερίληψη και επακόλουθη δημοσίευση του υλικού αυτού.

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η Διπλωματική Εργασία με τίτλο

ΥΛΟΠΟΙΗΣΗ ΤΟΥ ΜΕΤΑΚΒΑΝΤΙΚΟΥ ΚΡΥΠΤΟΓΡΑΦΙΚΟΥ ΑΛΓΟΡΙΘΜΟΥ CRYSTALS-DILITHIUM ΣΕ FPGA ΜΕ ΧΡΗΣΗ HLS

του φοιτητή του Τμήματος Ηλεκτρολόγων Μηχανικών και
Τεχνολογίας Υπολογιστών

ΣΕΜΕΡΤΖΑΚΗΣ ΜΑΡΙΟΣ ΤΟΥ ΕΜΜΑΝΟΥΗΛ

Αριθμός Μητρώου: 1053540

Παρουσιάστηκε δημόσια στο Τμήμα Ηλεκτρολόγων Μηχανικών και
Τεχνολογίας Υπολογιστών στις

09/10/2023

και εξετάστηκε από την ακόλουθη εξεταστική επιτροπή:

Θεοδωρίδης Γεώργιος, Αναπληρωτής Καθηγητής, ΤΗΜΤΥ (επιβλέπων)

Παλιουράς Βασίλης, Καθηγητής, ΤΗΜΤΥ (μέλος επιτροπής)

Σερπάνος Δημήτριος, Καθηγητής, ΤΗΜΤΥ (μέλος επιτροπής)

Ο Επιβλέπων

Ο Διευθυντής του Τομέα

Θεοδωρίδης Γεώργιος
Αναπληρωτής Καθηγητής

Θεοδωρίδης Γεώργιος
Αναπληρωτής Καθηγητής

PREFACE

First of all, I would like to thank my supervisor, George Theodoridis, who trusted me with this thesis, and was advising me throughout the duration of this work. Furthermore, I would like to thank my family for supporting me all these years and for being patient with me. I also want to thank my friends for being there for me and pushing me to become a better person and finally take my degree. Last but not least, I would like to thank my girlfriend who was there for me, both in sorrows and in joys, in my whole student career and who never stopped believing in me.

ABSTRACT

IMPLEMENTATION OF THE POST QUANTUM CRYPTOGRAPHY CRYSTALS-DILITHIUM ON FPGA USING HLS

SEMERTZAKIS MARIOS

THEODORIDIS GEORGE

In the era of advancing quantum cryptography, the security of classical cryptographic systems faces threats. Due to that reason, the post quantum cryptography field emerged, seeking to develop encryption techniques resistant to quantum algorithms. One promising solution is the Dilithium-CRYSTALS algorithm, which is a lattice-based crypto-algorithm, which has not been able to be broken yet.

To achieve efficient security, the need of hardware accelerators has emerged, due to the Post-Quantum Cryptography algorithm's complexity and mathematical operations. This complexity is double-edged sword, as on one hand, it ensured resistance to quantum attacks, but on the other hand, it can lead to a slow performance when implemented on conventional hardware. Therefore, hardware accelerators specialized for these algorithms are essential to maintain acceptable levels of efficiency and performance. These accelerators can improve the speed and efficiency of the cryptographic operations, while ensuring that secure communication remains possible in the face of evolving threats posed by quantum computers.

The purpose of this thesis is the high-level synthesis of the post quantum cryptography Dilithium-CRYSTALS algorithm and its optimization in terms of latency and resource consumption. For this purpose, the Vivado HLS tool from Xilinx was used, and more specifically we chose the FPGA platform of *kintex-7* family.. First, the algorithm is imported into the Vivado HLS tool where its synthesis is implemented, which results in an RTL description. Then a series of optimizations are applied through the techniques supported by this tool in order to produce the best possible result. The final implementation is judged in terms of latency reduction and the use of hardware resources required. Finally, there was comparison between our implementation with that of our referenced one and we concluded that our implementation achieved 3 to 4 times less latency.

ΥΛΟΠΟΙΗΣΗ ΤΟΥ ΜΕΤΑΚΒΑΝΤΙΚΟΥ ΚΡΥΠΤΟΓΡΑΦΙΚΟΥ ΑΛΓΟΡΙΘΜΟΥ CRYSTALS-DILITHIUM ΣΕ FPGA ΜΕ ΧΡΗΣΗ HLS

ΣΕΜΕΡΤΖΑΚΗΣ ΜΑΡΙΟΣ

ΘΕΟΔΩΡΙΔΗΣ ΓΕΩΡΓΙΟΣ

A. Εισαγωγή

Τις τελευταίες δεκαετίες, οι κλασικοί κρυπτογραφικοί αλγόριθμοι, έχουν πολύ σημαντικό ρόλο για την υποδομή της παγκόσμιας ψηφιακής επικοινωνίας. Διατηρούν ασφαλή και ιδιωτική κάθε είδους επικοινωνία μεταξύ ατόμων, επιχειρήσεων και κυβερνήσεων.

Στις μέρες μας, λόγω της εκθετικής ανάπτυξης της τεχνολογίας, η ανθρωπότητα έχει επιτύχει μία θεαματική ανακάλυψη, τους κβαντικούς υπολογιστές. Οι κβαντικοί υπολογιστές είναι πολλά υποσχόμενοι, καθώς παρέχουν απaráμιλλη υπολογιστική ισχύ και δυνατότητες.

Ωστόσο, παράλληλα με την ανάπτυξη των κβαντικών υπολογιστών, αναπτύχθηκαν και κάποιοι κβαντικοί αλγόριθμοι, οι οποίοι έχουν θέσει σε κίνδυνο τους κλασικούς κρυπτογραφικούς αλγόριθμους. Λόγω της τεράστιας υπολογιστικής ικανότητας αυτών των υπολογιστών, ουσιαστικά «σπάνε» αυτούς τους αλγόριθμους, καθώς βασίζονται στην σκληρότητα ορισμένων μαθηματικών προβλημάτων.

Έτσι δημιουργήθηκε η ανάγκη να αναπτυχθούν οι μετακβαντικοί κρυπτογραφικοί αλγόριθμοι, οι οποίοι είναι ικανοί να διατηρούν ασφάλεια και ιδιωτικότητα τόσο από τους δυαδικούς υπολογιστές, όσο και από τους κβαντικούς. Πολλοί επιστήμονες έχουν ήδη αναπτύξει τέτοιου είδους αλγόριθμους, οι οποίοι πράγματι

αντιστέκονται στις επιθέσεις των κβαντικών αλγορίθμων. Ένας από αυτούς ονομάζεται CRYSTALS-Dilithium, ο οποίος αποτελεί τον αλγόριθμο που θα υλοποιηθεί σε πλατφόρμα FPGA με χρήση την Σύνθεση Υψηλού Επιπέδου.

B. CRYSTALS-Dilithium

Ο Dilithium είναι μία ψηφιακή υπογραφή, βασισμένη στο Learning with Errors (LWE) και στο Short Integer Solutions (SIS). Αποτελεί μέρος του Cryptographic Suite for Algebraic Lattices (CRYSTALS), όπου υποβλήθηκε στο National Institute of Standards and Technology (NIST). Ο Dilithium χρησιμοποιεί μία ομοιόμορφη κατανομή για να βελτιστοποιήσει το μέγεθος του δημόσιου κλειδιού αντί για Gaussian δειγματοληψία.

Ο Dilithium αποτελείται από τρία βασικά components. Το Key generation, το Signature Generation και το Signature Verification. Το πρώτο είναι υπεύθυνο για την δημιουργία ενός ζεύγους κλειδιών, το δημόσιο και το κρυφό (public and secret key). Το κρυφό κλειδί χρησιμοποιείται για την δημιουργία μίας ψηφιακής υπογραφής, όπου αυτή η υπογραφή με την σειρά της, σε συνδυασμό με το δημόσιο κλειδί, γίνεται η επιβεβαίωση αν το μήνυμα είναι από πιστοποιημένο χρήστη ή όχι.

Γ. Σύνθεση Υψηλού Επιπέδου

Η σύνθεση υψηλού επιπέδου (HLS) είναι μία μέθοδος σχεδιασμού κατά την οποία

μεταφράζεται αυτομάτως ένας κώδικας C/C++ σε RTL επίπεδο, και με την βοήθεια του εργαλείου Vivado HLS, όπου μας παρέχει πολλαπλό αριθμό ντιρεκτίβων, μας επιταχύνει την διαδικασία βελτιστοποίησης μίας υλοποίησης. Η διαδικασία της σύνθεσης του υψηλού επιπέδου (HLS) περιλαμβάνει πολλά βασικά βήματα.

Αρχικά, γλώσσες όπως C/C++ και η SystemC χρησιμοποιούνται ως είσοδοι για να περιγράψουν την επιθυμητή υλοποίηση, με την βοήθεια ενός testbench αρχείου για την επιβεβαίωση της ορθής λειτουργίας. Η επαλήθευση εκτελείται με τη μεταγλώττισή του κώδικα C με το testbench αρχείο για να διασφαλιστεί η ορθή παραγωγή δεδομένων εξόδου. Στη συνέχεια, η σύνθεση υψηλού επιπέδου περιλαμβάνει την εισαγωγή περιορισμών και ντιρεκτιβών, για την περιγραφή του κυκλώματος RTL. Το εργαλείο HLS παράγει ένα κύκλωμα IP, το οποίο ενσωματώνει την σχεδίαση υψηλού επιπέδου σε ξεχωριστό μπλοκ. Εν συνεχεία, το C/RTL co-simulation επαληθεύει την ορθότητα του σχεδιασμού συγκρίνοντας τις εξόδους του testbench με του RTL. Μετά την δημιουργία IP, η υλοποίηση αξιολογείται σε μία συσκευή FPGA χρησιμοποιώντας μετρήσεις όπως πόρους, καθυστέρηση, μέγιστη συχνότητα και κατανάλωση ενέργειας. Εάν χρειάζεται, τα βήματα βελτιστοποίησης που περιλαμβάνουν αλλαγές στον κώδικα μπορούν να εφαρμοστούν επαναληπτικά. Μόλις η βελτιστοποίηση ανταποκριθεί στις προσδοκίες, ο σχεδιασμός ενσωματώνεται ως IP, χρησιμοποιώντας πηγές RTL (VHDL ή Verilog).

Δ. ΔΙΑΔΙΚΑΣΙΑ ΒΕΛΤΙΣΤΟΠΟΙΗΣΕΩΝ

Οι μετρικές οι οποίες μας ενδιαφέρανε καθ' όλη τη διάρκεια της εργασίας αυτής ήταν οι εξής:

1. Initiation Interval: ο αριθμός κύκλος ρολογιού ώστε να ξεκινήσει η επεξεργασία ενός νέου δεδομένου.
2. Latency: ο αριθμός κύκλων ρολογιού ώστε να ολοκληρωθεί η επεξεργασία δεδομένων μίας συνάρτησης.
3. Resource Utilization: η κατανάλωση πόρων μίας συνάρτησης.

Αρχικά, αφότου υλοποιήσαμε τον αλγόριθμο μέσω του εργαλείου Vivado HLS, εντοπίσαμε τις συναρτήσεις οι οποίες ευθύνονται για την μεγάλη καθυστέρηση και για την μεγάλη κατανάλωση πόρων. Αυτό το πέτυχαμε παρατηρώντας τα αποτελέσματα μέσω του σταδίου C synthesis στο εργαλείο μας, και με την βοήθεια ενός επιστημονικού άρθρου. Καταλήξαμε ότι οι πιο χρονοβόρες συναρτήσεις ήταν αυτές που υλοποιούν τον αλγόριθμο Number Theoretic Transform, και οι συναρτήσεις με την μεγαλύτερη κατανάλωση πόρων οι hash συναρτήσεις.

Ο Number Theoretic Transform (NTT) είναι στην ουσία ο Discrete Fourier Transform (DFT) σε ένα πεπερασμένο πεδίο ή δαχτυλίδι, και υλοποιούνται οι πράξεις μεταξύ ακεραίων συντελεστών πολυωνύμων. Ο ευθύς NTT παράγει τις εξόδους ενός διανύσματος με τους δείκτες των διευθύνσεων να είναι bit-reversed, ενώ ο αντίστροφος NTT τις παράγει στην αρχική τους μορφή. Οι δύο αυτές συναρτήσεις, με την βοήθεια του Montgomery reduction υλοποιούν τον πολλαπλασιασμό μεταξύ ακεραίων συντελεστών πολυωνύμων.

Σαν πρώτη αντιμετώπιση των συναρτήσεων που υλοποιούν αυτόν τον αλγόριθμο χρειάστηκε να μετασχηματίσουμε κατάλληλα τα όρια των βρόχων, καθώς το εργαλείο δεν μπορούσε να προσδιορίσει το συνολικό

latency τους. Ύστερα, εφαρμόσαμε δύο από τις ντιρεκτιβες όπου διαθέτει το Vivado HLS. Πρώτα, εφαρμόσαμε την ντιρεκτίβα UNROLL στον εσωτερικό βρόχο, την οποία και απορρίψαμε καθώς είχε τεράστια αύξηση κατανάλωσης πόρων και αρκετά σημαντική αύξηση του latency. Κατά δεύτερον, εφαρμόσαμε την ντιρεκτίβα PIPELINE, η οποία αρχικά δεν είχε καμία επίδραση στο κύκλωμά μας.

Δεν είχε κάποια επίδραση λόγω του ότι το *Initiation Interval* (II) ισούταν με 8. Αυτό αναλύοντας τον κώδικα ίσχυε για δύο λόγους. Πρώτον, λόγω της ύπαρξης των anti-dependencies, και δεύτερον, λόγω της ύπαρξης των loop-carried dependencies.

Anti-dependencies υπάρχουν όταν, στην ίδια επανάληψη, πρώτα διαβάζεται μία διεύθυνση μνήμης και ύστερα γράφεται στην ίδια διεύθυνση μνήμης, και έτσι υπάρχει ο κίνδυνος να ξεκινήσει πρώτα να γράφει με αποτέλεσμα να υπάρχει αλλοίωση αποτελεσμάτων. Loop-carried dependencies υπάρχουν όταν μία επόμενη επανάληψη εξαρτάται από την ολοκλήρωση μίας ή περισσότερων προηγούμενων επαναλήψεων.

Για την αντιμετώπιση των anti-dependencies τοποθετήσαμε καταχωρητές ώστε να αποθηκεύεται η τιμή της διεύθυνσης μνήμης, ώστε σε περίπτωση που ξεκινήσει πρώτα η εγγραφή σε εκείνη την συγκεκριμένη διεύθυνση μνήμης, να υπάρχει αποθηκευμένη η τιμή. Για την αντιμετώπιση των loop-carried dependencies, αποφασίσαμε να εφαρμόσουμε μία τεχνική που ονομάζεται *Loop Splitting*, δηλαδή διασπάσαμε τον βρόχο, ο οποίος ευθύνεται για αυτές τις εξαρτήσεις, με τέτοιο τρόπο ώστε να δημιουργηθούν πλέον οκτώ διαφορετικά modules, ένα για κάθε στάδιο, από τα 8 του NTT. Έτσι επιτυγχάνεται η εξουδετέρωση αυτών των εξαρτήσεων

καθώς ξεκινάει η επεξεργασία ενός σταδίου, αφότου ολοκληρωθεί η επεξεργασία δεδομένων του προηγούμενου. Με τις δύο αυτές βελτιστοποιήσεις κατορθώθηκε η μείωση του II=2.

Μία άλλη τεχνική βελτιστοποίησης που εφαρμόσαμε ήταν ο συνδυασμός δύο τεχνικών. Η πρώτη είναι η *Loop Normalization*, μέσω της οποίας κανονικοποιήσαμε τα όρια των βρόχων. Η συγκεκριμένη τεχνική από μόνη της δεν επέφερε καμία βελτίωση, αλλά μας έκανε εφικτό την εφαρμογή της τεχνικής *Loop Coalescing*, δηλαδή να μετασχηματίσουμε τις nested loops, σε μία. Ο συνδυασμός αυτών των δύο τεχνικών έφερε μία σημαντική μείωση καθώς πλέον μειώθηκε το control overhead.

Εν συνεχεία, για να επιτευχθεί το II=1 για κάθε στάδιο, επιλέξαμε να ενσωματώσουμε μεταξύ του κάθε σταδίου συμπληρωματικές μνήμες, καθώς ο λόγος που δεν ήταν αρχικά εφικτό το II=1 ήταν το γεγονός ότι δεν μπορούσε η μνήμη να οργανώσει το store operation λόγω των περιορισμένων θυρών εισόδου/εξόδου.

Τέλος, για να περιορίσουμε την κατανάλωση πόρων της συγκεκριμένης συνάρτησης, εφαρμόσαμε την ντιρεκτίβα ALLOCATION, ώστε να περιορίσουμε τον αριθμό των πολλαπλασιαστών που χρησιμοποιούνται και τον αριθμό των Montgomery reduce modules.

Οι συναρτήσεις οι οποίες ευθύνονται για την μεγαλύτερη κατανάλωση πόρων είναι, όπως προαναφέρθηκε, οι hash συναρτήσεις. Στον αλγόριθμό μας αυτές οι συναρτήσεις είναι οι shake128 και shake256. Αυτές οι συναρτήσεις είναι βασισμένες στο sponge construction, όπου για οποιοδήποτε μήκος bit εισόδου παράγει μία έξοδο οποιοδήποτε επιθυμητού μήκους, και αποτελούνται από δύο στάδια. Το absorb, όπου

απορροφά όλα τα διαιρεμένα μπλοκ μίας state memory, αρχικοποιώντας την στο μηδέν, και το squeeze, όπου εξάγει την έξοδο, η οποία υφίσταται κάποιες μεταλλαγές μέσω της συνάρτησης KeccakF1600.

Σαν πρώτη βελτιστοποίηση εφαρμόστηκε κατάλληλα η ντιρεκτίβα `INLINE` στις συναρτήσεις, με σκοπό να «διαλυθούν» και να ανέβουν στην ιεραρχία. Έτσι, λόγω του ότι καλούνται πολλαπλές φορές αυτές οι hash συναρτήσεις, πλέον είναι εφικτή η κοινοποίηση πόρων. Επίσης, εφαρμόστηκε η ντιρεκτίβα `UNROLL` στις συναρτήσεις που είναι υπεύθυνες να αποθηκεύσουν και να φορτώσουν τα δεδομένα της state memory, με σκοπό την μείωση της καθυστέρησης που επέφεραν στο κύκλωμα.

Καθώς αναλύαμε τις συναρτήσεις αυτές, παρατηρήσαμε ότι στην συνάρτηση που γίνονται οι μεταλλαγές της state memory, υλοποιούνται δύο μεταλλαγές ανά επανάληψη, πράγμα το οποίο ευθυνόταν για μεγάλη κατανάλωση πόρων, και πιο συγκεκριμένα XOR, AND και shifts. Έτσι, ανακατασκευάσαμε την συνάρτηση με τέτοιο τρόπο ώστε να επιτυγχάνεται μία μεταλλαγή ανά επανάληψη, και πράγματι παρατηρήθηκε μεγάλη μείωση πόρων.

Στο αρχείο `polyvec` υλοποιούνται οι συναρτήσεις η οποίες από έναν βρόχο που κάνει `iterate` στο κάθε διάνυσμα πολυωνύμων και καλεί τις αντίστοιχες συναρτήσεις η οποίες υλοποιούν πράξεις μεταξύ των συντελεστών των πολυωνύμων. Οπότε σαν πρώτη βελτιστοποίηση εφαρμόσαμε αρχικά την ντιρεκτίβα `PIPELINE` στον βρόχο των συναρτήσεων του αρχείου `poly`, το οποίο είχε μία ικανοποιητική μείωση του latency, και σε δεύτερη εφαρμόσαμε την ντιρεκτίβα `UNROLL`, οποίο μείωσε ικανοποιητικά το latency αλλά με μεγάλη αύξηση πόρων.

Οπότε επιλέξαμε να ακολουθήσουμε μία διαδικασία συνδυασμού αυτών των δύο ντιρεκτίβων, και ακολουθήσαμε την παρακάτω επαναληπτική διαδικασία. Αρχικά εφαρμόσαμε στον ίδιο βρόχο τις ντιρεκτίβες `PIPELINE` και `UNROLL factor=2`. Όμως παρατηρήσαμε ότι για να επιτευχθεί το `II=1` χρειάστηκε να εφαρμόσουμε την ντιρεκτίβα `ARRAY PARTITION factor=2`, καθώς προηγουμένως δεν ήταν εφικτή η οργάνωση των load/store operations λόγω περιορισμένων θυρών εισόδου\εξόδου. Έτσι παρατηρήσαμε αρκετά ικανοποιητική μείωση του latency και αποφασίσαμε να την επαναλάβουμε για `factor=4, 8, 16, 32, 64` και `128` και στο τέλος συγκρίναμε ποια βελτιστοποίηση είχε την βέλτιστη σχέση latency-resource utilization.

Μία τελευταία βελτιστοποίηση εφαρμόστηκε στην συνάρτηση που ευθύνεται για τον πολλαπλασιασμό στοιχείο προς στοιχείο μεταξύ των συντελεστών δύο πολυωνύμων. Αυτή η συνάρτηση εφαρμόζεται αφότου τα πολυώνυμα υποστούν ευθύ NTT και πρωτού τους εφαρμοστεί ο αντίστροφος NTT. Αναλύοντας την συνάρτηση παρατηρήσαμε ότι δύο από τις συναρτήσεις όπου καλούνται, αποτελούνται από έναν βρόχο ιδίων ορίων, οπότε επιλέξαμε να τις κάνουμε χειροκίνητα `INLINE` και ύστερα να εφαρμόσουμε την τεχνική *Loop Fusion*, δηλαδή να μετασχηματίσουμε τους δύο βρόχους σε έναν. Και σας τελευταία αντιμετώπιση της εφαρμόσαμε την solution που εφαρμόστηκε και στις υπόλοιπες `polyvec` συναρτήσεις.

ΣΤ. ΣΥΝΔΥΑΣΜΟΣ ΒΕΛΤΙΣΤΟΠΟΙΗΣΕΩΝ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ

Αφότου ολοκληρώθηκε η επαναληπτική διαδικασία βελτιστοποιήσεων και ξεχωρίσαμε τις βέλτιστες, τις εφαρμόσαμε

στα τρία top functions για να πάρουμε τα τελικά μας αποτελέσματα.

Ο συνδυασμός αυτών των τεχνικών επέφερε μία ικανοποιητική μείωση καθώς συμπεράναμε ότι η συνάρτηση που υλοποιεί το Key Generation επιταχύνθηκε κατά 4.05x φορές, η συνάρτηση που υλοποιεί το Signature Generation κατά 4.82x και η συνάρτηση που υλοποιεί το Signature Verification κατά 2.88x, συγκριτικά με την υλοποίηση χωρίς εφαρμογή βελτιστοποιήσεων.

Με το πέρας της διπλωματικής εργασίας καταλήξαμε σε δύο ειδών συμπεράσματα. Συγκεκριμένα, το εργαλείο Vivado HLS πράγματι βοήθησε στην γρήγορη δημιουργία του RTL σχεδίου και μας έδωσε την δυνατότητα να επιταχυνθεί κατά πολλές φορές περισσότερο η επαναληπτική διαδικασία βελτιστοποιήσεων, μέσω των πολλαπλών ντιρεκτίβων που διαθέτει. Παράλληλα, μέσα από την υλοποίηση του Dilithium, καταλήξαμε στο ότι λόγω της μεγάλης πολυπλοκότητας του αλγορίθμου, των πολλών συναρτήσεων και των μεγάλων διανυσμάτων συντελεστών πολυωνύμων, καθίσταται δύσκολη η διαδικασία μείωσης του latency.

Ωστόσο, καταλήγοντας, σαν μελλοντική βελτιστοποίηση θα μπορούσε να μετασχηματιστεί ο αλγόριθμος του NTT με τρόπο ώστε να ακολουθεί το πρότυπο της ντιρεκτίβας DATAFLOW ώστε να επιτευχθεί παραλληλία όχι μόνο σε κάθε στάδιο, αλλά και μεταξύ των σταδίων, δηλαδή να ξεκινάει η επεξεργασία ενός σταδίου πριν την ολοκλήρωση του προηγούμενου.

TABLE OF CONTENTS

TABLE OF CONTENTS	xiv
LIST OF FIGURES.....	xvi
LIST OF TABLES.....	xx
Chapter 1 Diploma's objectives	1
A. Motivation and objectives	1
B. Summary of our developed architecture and optimizations.....	2
Chapter 2 CRYSTALS-Dilithium Algorithm.....	6
2.1 Introduction	6
2.2 Structure of Crystals-Dilithium	7
2.2.1 Key Generation	8
2.2.2 Signature Generation	9
2.2.3 Signature Verification	11
2.3 Basic Components of Dilithium.....	12
2.3.1 Number Theoretic Transform.....	12
2.3.2 Montgomery Reduction Algorithm	14
2.3.3 Sponge Construction	15
2.3.4 Keccak-f permutation	16
2.3.5 Secure Hash Algorithm KEccak (SHAKE)	21
2.3.6 Power2Round and Decompose	22
Chapter 3 High Level Synthesis	24
3.1 Introduction	24
3.2 Vivado High Level Synthesis.....	24
3.2.1 Vivado High Level Synthesis Design Flow	25
3.2.2 High Level Synthesis Benefits	26
3.2.3 Optimization methods of Vivado HLS tool	27
3.2.4 Optimization methods changing the source code.....	30
3.2.5 Vivado HLS limitations	34
Chapter 4 First Implementation of CRYSTALS-Dilithium	36
4.1 Initial Solution	36
4.1.1 Top-level function and verification code.....	36
4.1.2 Project and Solutions.....	37

4.1.3 C simulation	38
4.1.4 High Level Synthesis	38
4.1.5 RTL Co-Simulation.....	39
Chapter 5 Optimized versions of CRYSTALS-Dilithium	40
5.1 Profiling and function selection to optimize.....	40
5.2 Optimization of polyveck_ntt and polyvecl_ntt functions	42
5.2.1 Transformation to constant loop bounds	43
5.2.2 Optimization of the inner loop of the ntt() function	44
5.2.3 Elimination of the anti-dependencies	46
5.2.4 Elimination of loop-carried dependencies	48
5.2.5 Appliance of Loop Normalization technique in every stage of ntt() function	51
5.2.6 Appliance of Loop Coalescing technique in every stage of ntt() function....	52
5.2.7 Insertion of supplementary RAM	53
5.2.8 Comparison of optimizations and optimal selection.....	53
5.2.9 Reduction of resource utilization	54
5.3 Optimization of polyveck_invntt_montgomery() function.....	56
5.4 Optimizations of shake256() and shake128() functions	57
5.4.1 Reduction of resources consumed by the functions of the fips202.c file	59
5.4.2 Reduction of resources consumed by KeccakF1600_StatePermute function	59
5.4.3 Combination of the Solution_Inline_fips202 and the One_Permute_Per_Iter solutions	60
5.5 Optimizations of polyveck and polyvecl functions	61
5.6 Optimization of polyvecl_pointwise_acc_im() function	67
Chapter 6 Experimental results and comparisons.....	71
6.1 Combination of the optimal solutions to the Dilithium-CRYSTALS.....	71
6.2 Comparison of the results.....	72
Chapter 7 Conclusions and future research.....	74
7.1 Conclusions	74
7.2 Future research.....	75
BIBLIOGRAPHY	76

LIST OF FIGURES

Figure 1.1: FPGA architecture	1
Figure 2.1: a) deterministic bit b) probabilistic quantum bit	6
Figure 2.2: Structure of Crystals-Dilithium	8
Figure 2.3: Key generation algorithm	9
Figure 2.4: Signature generation algorithm	10
Figure 2.5: Signature verification algorithm	12
Figure 2.6: Bit reverse.....	13
Figure 2.7: NTT stages	14
Figure 2.8: Butterfly unit	14
Figure 2.9: The sponge construction of hash functions	15
Figure 2.10: Relation between capacity, rate and b	16
Figure 2.11: Parts of the state [15].....	17
Figure 2.12: Illustration of θ applied to a single bit [15]	18
Figure 2.13: Illustration of ρ [15].....	19
Figure 2.14: Illustration of π in a single bit [15]	20
Figure 2.15: Illustration of χ applied on a single row [15]	20
Figure 2.16: Illustration of hash function [12].....	22
Figure 2.17: Impact of different Byte (SHAKE256)	22
Figure 2.18: Impact of different Byte (SHAKE128)	22
Figure 2.19: Impact of different output size (SHAKE256)	22
Figure 2.20: Impact of different output size (SHAKE128)	22
Figure 3.1: Design flow of FPGA	25
Figure 3.2: Design flow of Vivado HLS tool [17]	25
Figure 3.3: The appliance of Pipeline directive	28
Figure 3. 4: Partially unrolling and fully unrolling	29
Figure 3.5: Array partition directive	30
Figure 3.6: Example of Loop Coalescing [22].....	31
Figure 3.7: Example of Loop Normalization	31
Figure 3.8: Example of Loop Splitting [22]	32
Figure 3.9: Example of Loop Fusion.....	32
Figure 3.10: Data-dependencies analysis. (a) Sequence of statements, (b) types of data dependencies, (c) data dependence graph.....	33

Figure 3.11: Elimination of data dependencies. (a) renamed sequence of statements, (b) data dependence graph after renaming	33
Figure 3.12: Example of data dependencies in nested loop [22]	34
Figure 4.1: crypto_sign() hierarchy	36
Figure 4.2: crypto_sign_open() hierarchy	37
Figure 4.3: crypto_sign_keypair() hierarchy.....	37
Figure 4.4: Synthesis setting.....	38
Figure 4.5: Inputs of testbench	38
Figure 4.6: Timing estimations of implementation of CRYSTALS-DILITHIUM without optimization	38
Figure 4.7: Utilization estimations of implementation of CRYSTALS-DILITHIUM without optimization	39
Figure 4.8: RTL Co-Simulation	39
Figure 5.1: Timing estimations of the instances of crypto_sign()	41
Figure 5.2: Resource estimations of the instances of crypto_sign()	42
Figure 5.3: NTT hierarchy	43
Figure 5.4: Reconstructed ntt() function with constant boundaries.....	44
Figure 5.5: PIPELINE directive to the inner loop.....	45
Figure 5.6: UNROLL directive to the inner loop	45
Figure 5.7: Anti-dependencies of the loop body.....	46
Figure 5.8: Insertion of intermediate registers	47
Figure 5.9: Analysis perspective before inserting registers.....	47
Figure 5.10: Analysis perspective after inserting registers	48
Figure 5.11: Loop-carried dependency analysis of a three-stage number theoretic transform	49
Figure 5.12: Reconstructed ntt() function after the appliance of loop splitting technique	50
Figure 5.13: PIPELINE and DEPENDENCE directive applied to every stage	50
Figure 5.14: Example of appliance of the Loop Normalization technique on second stage ...	51
Figure 5.15: Example of appliance of the Loop Coalescing technique on second stage.....	52
Figure 5.16: Combination of optimization to polyveck_ntt() function	54
Figure 5.17: Latency after the appliance of the S7_ntt_LSNCR_p and S8_alloc_LSNCR_p to the polyveck_ntt() function	55
Figure 5.18: Resource Utilization after the appliance of the S7_ntt_LSNCR_p and S8_alloc_LSNCR_p to the polyveck_ntt() function	55
Figure 5.19: Inverse NTT hierarchy.....	56

Figure 5.20: Latency after the appliance of the S7_ntt_LSNCR_p and S8_alloc_LSNCR_p to the invntt_frominvmont() function	57
Figure 5.21: Resource Utilization after the appliance of the S7_ntt_LSNCR_p and S8_alloc_LSNCR_p to the invntt_frominvmont() function	57
Figure 5.22: Hierarchy of shake128() function	58
Figure 5.23: Hierarchy of shake256() function	58
Figure 5.24: Resource Utilization of crypto_sign() function with the appliance of Solution_inline_fips202	59
Figure 5.25: KeccakF1600_StatePermute() function	60
Figure 5.26: Resource Utilization of the crypto_sign() with and without the optimization of the KeccakF1600_StatePermute() function	60
Figure 5.27: Resource Utilization of the crypto_sign() with and without the combination of the optimizations of the KeccakF1600_StatePermute() function	61
Figure 5.28: Hierarchies of the functions	61
Figure 5.29: Appliance of PIPELINE directive to the inner loop of poly_add() function	62
Figure 5.30: Appliance of UNROLL directive to the inner loop of poly_add() function	62
Figure 5.31: Appliance of ARRAY PARTITION complete directive to the inputs and output arrays of polyveck_add() function	62
Figure 5.32: Latency graph of the polyveck_add() function with the implementation of various optimizations	63
Figure 5.33: Resource Utilization graph of the polyveck_add() function with the implementation of various optimizations	63
Figure 5.34: Example of loop unrolling by a factor of two	63
Figure 5.35: Array partitioning in a cyclic manner	64
Figure 5.36: Latency graph of the polyveck_add() function with the implementation of further optimizations	65
Figure 5.37: LUT and FF Utilization graph of the polyveck_add() function with the implementation of further optimizations	65
Figure 5.38: Latency graph of the polyvecl_freeze() function with the implementation of further optimizations	66
Figure 5.39: LUT and FF Utilization graph of the polyvecl_freeze() function with the implementation of further optimizations	66
Figure 5.40: Latency graph of the polyvecl_add() function with the implementation of further optimizations	66
Figure 5.41: LUT and FF Utilization graph of the polyvecl_add() function with the implementation of further optimizations	66
Figure 5.42: Latency graph of the polyveck_freeze() function with the implementation of further optimizations	67

Figure 5.43: LUT and FF Utilization graph of the <code>polyveck_freeze()</code> function with the implementation of further optimizations	67
Figure 5.44: Latency graph of the <code>polyveck_power2round()</code> function with the implementation of further optimizations	67
Figure 5.45: LUT and FF Utilization graph of the <code>polyveck_power2round()</code> function with the implementation of further optimizations	67
Figure 5.46: Hierarchy of the <code>polyvecl_pointwise_acc_im()</code> function	68
Figure 5.47: Block diagram of <code>polyvecl_pointwise_acc_im()</code> function.....	68
Figure 5.48: Manually function inlining and Loop Fusion	68
Figure 5.49:Appliance of UNROLL, PIPELINE and ARRAY PARTITION directive to the <code>polyvecl_pointwise_acc_im()</code> function.....	69
Figure 5.50: Latency graph of the <code>polyvecl_pointwise_acc_im()</code> function with the implementation of a) Loop Fusion b) <code>S_pv_unroll_part4_Pipeline</code> and Loop Fusion	69
Figure 5.51: FF & LUT Utilization graph of the <code>polyvecl_pointwise_acc_im()</code> function with the implementation of a) Loop Fusion b) <code>S_pv_unroll_part4_Pipeline</code> and Loop Fusion	70
Figure 6.1Comparison of Latency and Resource Utilization of <code>crypto_sign()</code> function of No optimization and Our optimization	71
Figure 6.2: Comparison of Latency and Resource Utilization of <code>crypto_sign_keypair()</code> function of No optimization and Our optimization	71
Figure 6.3: Comparison of Latency and Resource Utilization of <code>crypto_sign_open()</code> function of No optimization and Our optimization	72
Figure 6.4: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the <code>crypto_sign()</code> function.....	73
Figure 6.5: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the <code>crypto_sign_keypair()</code> function	73
Figure 6.6: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the <code>crypto_sign_open()</code> function.....	73

LIST OF TABLES

Table 2.1: Security parameters, signature size, public key size and secret key size of Dilithium variants	8
Table 2.2: Width and related quantities.....	17
Table 2.3: The rotation offsets for b=1600	19
Table 2.4: Round constant.....	21
Table 2.5: Data correspondence for Decompose [16]	23
Table 5.1: Latency results of polyveck_ntt() function with variable boundaries	44
Table 5.2: Latency results of polyveck_ntt() function with constant boundaries.....	44
Table 5.3: Latency of the polyveck_ntt() function with the appliance of PIPELINE directive to the inner loop of the function	45
Table 5.4: Latency of the polyveck_ntt() function with the appliance of UNROLL directive to the inner loop of the function	46
Table 5.5: Resource utilization of the polyveck_ntt() function with no optimizations and with the UNROLL directive	46
Table 5.6: Latency of the polyveck_ntt() function with the insertion of intermediate registers	48
Table 5.7: Resource Utilization and Latency for a) no optimization, b) Loop splitting, c) loop splitting + PIPELINE, d) loop splitting + PIPELINE + DEPENDENCE.....	51
Table 5.8: Resource Utilization and Latency for a) no optimization, b) Loop normalization, c) loop normalization + PIPELINE, d) loop normalization + PIPELINE + DEPENDENCE	52
Table 5.9:Resource Utilization and Latency for a) no optimization, b) Loop normalization +Loop Coalescing, c) loop normalization +Loop Coalescing + PIPELINE, d) loop normalization +Loop Coalescing + PIPELINE + DEPENDENCE	53
Table 5.10:Resource Utilization and Latency for a) no optimization, b) Loop normalization + Loop Coalescing + Supplementary RAM and c) loop normalization +Loop Coalescing + Supplementary RAM + PIPELINE	53
Table 5.11: Naming solution of ntt() function.....	54
Table 5.12: Naming last solution of ntt() function	55
Table 5. 13: Solution naming for fips202.c.....	58
Table 5.14: Solution naming.....	62
Table 5.15: Naming of further optimizations	64

Chapter 1

Diploma's objectives

A. Motivation and objectives

Field Programmable Gate Array (FPGA) is an array of interconnected array of subcircuits that provides flexibility to implement the hardware design of any algorithm and the configuration is specified using a hardware description language (HDL). They contain an array of programmable logic blocks, and a hierarchy of reconfigurable interconnects that allow blocks to be connected. These blocks can perform either as complex combinational logic functions, or even act as simple logic gates. Furthermore, they contain memory elements, either flip-flops or even blocks of memory.

In our days, FPGAs usage is becoming more and more important, as they have a significant role in the development of the integrated and embedded systems due to their capability of developing the software and the hardware simultaneously. Someone could refer to them as the bridge between hardware and software developers. Their best advantage is that thanks to this simultaneously development, the possibility of simulations and optimizations of the design is created before the final architecture of the system.

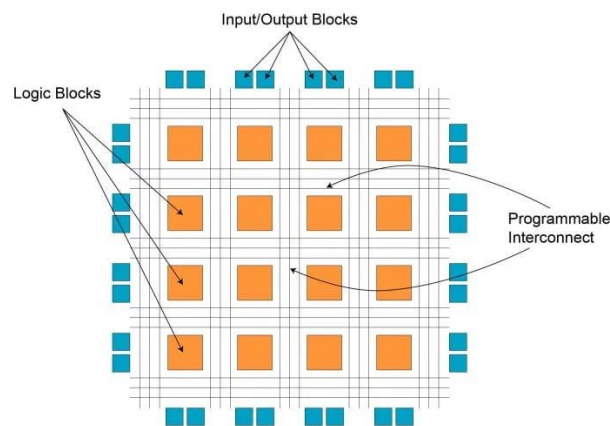


Figure 1.1: FPGA architecture

For many years, register-transfer level (RTL) has been the dominant method to describe a Very Large-Scale Integration (VLSI) system. But, although the RTL tools have developed incrementally, the complexity of the VLSI systems has raised exponentially, which has made the design and verification process a bottleneck. So, the desire for the development of a variety of many tools has risen, in order to face these problems and to make the process of the design faster, more flexible and easier. One of these tools has been created from Xilinx, the Vivado HLS. The creation of this tool has achieved to make the process of design easier and faster, as it made it possible to specify the configuration with higher level languages, such as C and C++.

This happens because the designer can concentrate on describing the behavioral of the system, without spending time on the implementation of the microarchitectures, the verification is accelerated, as the RTL output can be verified by the original behavioral test bench and some transformations of the source code can be applied without changing it, such as pipelining and unrolling.

The infrastructure of communication, all around the world, use cryptography algorithms to keep communication secure and private. As we approach the era of quantum computing, traditional cryptographic algorithms become vulnerable to attacks that could put in jeopardy the security and the privacy of sensitive information. So, the need of new cryptographic algorithms has emerged, the Post-Quantum Cryptography (PQC) algorithms, as the old ones can be easily broken from algorithms, such as Shor's, which is a quantum algorithm for finding prime numbers of an integer. These algorithms seek to develop encryption techniques that can withstand the computational power of the quantum computers, which help to provide long-term security of data transmissions and storage. By staying ahead of the quantum threats, PQC play significant role in preserving the confidentiality of our digital interactions.

The development of specialized accelerators for these PQC algorithms is significant due to the immense computational complexity of the algorithms. The Post-Quantum encryption methods rely on mathematical problems that are designed to be computationally demanding. As the development of these algorithm increases, the complexity of them increases too. This complexity is double-edged sword, as on one hand, it ensured resistance to quantum attacks, but on the other hand, it can lead to a slow performance when implemented on conventional hardware. Therefore, hardware accelerators specialized for these algorithms are essential to maintain acceptable levels of efficiency and performance. These accelerators can improve the speed and efficiency of the cryptographic operations, while ensuring that secure communication remains possible in the face of evolving threats posed by quantum computers. They impose a critical investment in the battle to protect sensitive data in an era of rapid technological advancement.

In essence, specialized hardware accelerators are not only convenience but a necessity. They are a critical component in the overall method of the resistance of quantum attacks. Without these accelerators, the adoption of Post-Quantum Cryptography could be hampered by prohibitive computational cost, limiting its practicality and effectiveness.

So, in this work, our motivation was, at first, to design a specialized hardware accelerator for a Post-Quantum Cryptography algorithm, named CRYSTALS-Dilithium (Cryptographic Suite for Algebraic Lattices). And lastly, to optimize it and provide security efficiently, with a smaller latency and using a good number of resources.

B. Summary of our developed architecture and optimizations

In this thesis, the post-quantum cryptography algorithm, Dilithium CRYSTALS (Cryptographic Suite for Algebraic Lattices), will be implemented, using high level synthesis method, and will be compared to our reference research paper [1]. Dilithium is a lattice-based algorithm, which is the generic term for constructions of cryptographic primitives (low-level cryptographic algorithms) that involve lattices. These type of cryptography uses well-studied lattice problems that cannot be solved, which leads to the assumption that, theoretically, Dilithium is secure and resistant to quantum algorithms.

In the Dilithium digital signature process, firstly a key pair is generated with a private key, generated randomly, kept secret and a corresponding public key shared openly, which are responsible for signing and verifying messages. When you want to send a message, you preprocess it and generate a digital signature using the private key, with the application of mathematical operations based on lattice-based cryptography. Randomization adds uniqueness to each signature, enhancing security. The resulted digital signature verifies the message's authenticity and integrity. Then the message and the signature are sent both to recipient. Public key is used to verify and check the signature's validity through mathematical operations, based on lattice-based cryptography, such as the Number Theoretic Transform. If the verification is successful, the recipient can trust the message.

Dilithium has four variants spanning four security levels, and for this work we chose to apply the directives, at first, at the security level 0 (Dilithium_Weak). The very first step was to insert the algorithm of Dilithium_Weak in Vivado HLS tool, and to choose the FPGA platform of *kintex-7* family. Then, after the simulation of the C code, with the help of a testbench, and the verification of the valid results, the synthesis of the algorithm took place. Dilithium has three top components, as shown in, so we did three separate syntheses, one for each top function. After the first results, we noticed that some functions had undefined latency and timing results, so after some tests we specified the loop trip-counts. Then, after synthesizing the algorithm again we noticed that some functions, that are common in both three top functions, caused a big delay of the implementation and big resource utilization. So, we followed an iterative optimization process.

The function with the most delay, in the top three functions of the hierarchy, was the Number Theoretic Transform (`polyveck_ntt` and `polyvecl_ntt`), which is in charge of bit reversing 256 elements of an array, and with the help of the `montgomery_reduce` and the `inv_fromntt` functions, the modular multiplication of these 256 polynomials is achieved. Our first step was to transform the boundaries of the loops into constant numbers, as due to its complexity and the nature of the NTT algorithm, the directive `LOOP_TRIPCOUNT` was not helpful. The next step was to apply the *PIPELINE* directive to the inner loop.

The most important problems that arose was that of the initiation interval, for which the loop-carried dependencies, the anti-dependencies and the read of only two values from memory, are responsible. For the loop-carried dependencies problem, we chose to use the *Loop Splitting* method, as after analyzing the code we found that by splitting stages of the NTT into 8 different components, they are eliminated. With this method we achieved to decrease the initiation interval in each stage dramatically. For the data dependencies, we inserted intermediate registers to store the values of the memory addresses, so even if the write completes first, the old value would still be available. Lastly, we created intermediate memories for each stage to eliminate the problem of the read of only two values of the memory. Even though it increased by a little the resource utilization, after applying both three optimizations we achieved an initiation interval of one (II=1) in every stage. A further optimization that we chose to apply was that of *Loop Normalization*, which transformed loop bounds to start from zero and to increment by one in every iteration. Although this increased the latency by a little, it enabled us to apply *Loop Coalescing*, which transformed every nested loop into one single loop and provided reduced overhead and better load balancing.

The inverse Number Theoretic Transform, after the modular multiplication between two arrays of 256 elements with the number theoretic transform, is responsible to bring the output array to its canonical form, because the elements were bit reversed. It is the same algorithm

with the NTT with the exact opposite steps. So that same problems with the NTT apply to `inv_fromntt` function too, and we followed the same process of optimizations. We transformed the loop boundaries in a way that the Vivado HLS can define the timings, we applied *PIPELINE* directive to the inner loop, we applied the *Loop Splitting* method, and we inserted intermediate registers and we created intermediate memories.

The hash functions `shake128()` and `shake256()` are those that consume the most resources. These functions produce hash values of variable lengths, which make it ideal for applications like data integrity verification. They employ a sponge construction, where it absorbs input data of any size and then generate hash values with the desired output length. To achieve that it uses a lot of logical operators, such as XORs and ANDs, which result in big resource utilization.

For our first approach we implemented the solution of our reference research paper. At first we applied the *INLINE* directive to the functions `shake128_absorb()`, `keccak_absorb()`, `shake128_squeezeblocks()`, `shake256_squeezeblocks()`, `keccak_squeezeblocks()`, `shake256_absorb()`, so that we dissolve these functions into the `shake128()` and `shake256()`. With this implementation we achieved resource sharing. Lastly, we applied *UNROLL* directive to the loops of `store64()` and `load64()` functions, to make the loading and storing operations to run concurrently and in that way the latency was reduced.

The function that consumes the most resources in this algorithm is this of `shake256` and `shake128`, which are the hash functions of the Dilithium algorithm, and more specifically the `KeccakF1600_StatePermutation`, which is responsible for the computational logic. This function implements the computations with many XORs and shift registers to 25 elements of 64 bit of a state memory. While analyzing the source code, we noticed that in each iteration two rounds of permutation take place, which implies that the resource utilization is twice as much as if one round was done per iteration. Indeed, after modifying the code accordingly we noticed that the resources have been decreased by a factor of two. A further optimization that we tried was to reconstruct the code and to create subfunctions, each of which were responsible for the creation of θ, χ, π, ι and ρ (they are analyzed in 2.3.4 Keccak-f permutation) using for loops, so that the usage of XORs was minimum. This succeeded in reducing the resource usage by fifty times, but unfortunately, it increased the delay by 100 times, so we rejected this optimization.

For the functions `polyveck-add()`, `polyvecl-add()`, `polyveck-freeze()`, `polyvecl-freeze()`, `polyveck-sub()`, `polyveck-neg()`, `polyveck-decompose()`, `polyveck-power2round()`, `polyveck-make-hint()` and `polyveck-use-hint()` we applied the *UNROLL* and *PIPELINE* directives to the inner loops, or to the loops of their subfunctions. We avoided to perform *PIPELINE* or *UNROLL*, to the outer loops or to the subfunctions, because each of these functions take as input two-dimensional arrays of $L*256$ or $K*256$ elements of 64 bit each element, as that would increase the resource utilization by a lot, and the FPGA did not have the resources needed. The first optimization was to apply *PIPELINE* directive in the inner loop. The next was to apply *UNROLL* by a factor of 2 and *PIPELINE* directives. This gave us an initiation interval greater than one because one value can be read from this memory. To eliminate this and to achieve $II=1$, we performed *ARRAY_PARTITION* directive by a factor of 2, in other words we partitioned the array of the first dimension in cyclic way into 2 arrays. This provided a better parallelism, as per clock cycle the loop processed two elements of the array but increased by a little the resource usage. We continued this process until the factor of *UNROLL* directive equals to 128 (we applied by a factor of 2, 4, 8, 16, 32, 64 and 128). Then we compared the results to choose the optimal. For the functions `polyveck_chknorm()` and `polyvecl_chknorm()`, firstly we inlined the

subfunction `chknorm` manually to flatten the loop and then we followed the exact same process with the previous functions.

Lastly, we optimized the function `polyvec1_pointwise_acc_im()`, where we ,at first, inlined manually the functions `poly_pointwise_invmontgomery()` and `poly_add()`, in order to apply loop fusion, as their outer loop has the same bounds, and to achieve a better parallelism of the data processing. Then we followed the same steps as in the other `polyvec` functions.

Finally, all the optimizations illustrated above, were combined, and gave the optimal solution of the CRYSTALS-Dilithium algorithm. Then we compared our results with that of our referenced paper and we concluded that all top three functions were 3 to 4 times faster with our implementation, with a small increase in the resource utilization.

Chapter 2

CRYSTALs-Dilithium Algorithm

2.1 Introduction

In the last decades, classical cryptography algorithms, such as Rivest-Shamir-Adleman (RSA), Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA), have been vitally important of our global communication digital infrastructure. They are keeping safe and private any type of communication between individuals, businesses and governments [2].

In our days, due to the exponential development of technology, humanity has achieved a quiet spectacular breakthrough, the creation of quantum computers. Quantum computing promises to revolutionize in various fields, bringing unparalleled computational power and capabilities. However, the technological achievement has brought the classical cryptographic algorithms, which rely on the hardness of certain mathematical problems, such as discrete logarithms, in jeopardy. Quantum algorithms, such as the widely known Shor's algorithm, can efficiently solve these mathematical problems, rendering most cryptographic algorithms insecure.

This happens because classical computers use deterministic bits, while quantum computers use probabilistic qubits, which means that while the classical bit exist in either 0 or 1, qubit's value depends only after the measurement by the observer. For example, considering a four classical bit processor, there are 2^4 different states, 0000, 0001, etcetera, that the binary computer can be once at a time. However, a four-quantum bit processor can be in all 2^4 states simultaneously. This can happen since an atom can exist in two stages, the so-called superposition, simultaneously, $|0\rangle$ and $|1\rangle$. The two states that can be seen in Figure 2.1 represent the two energy levels of an atom [3].

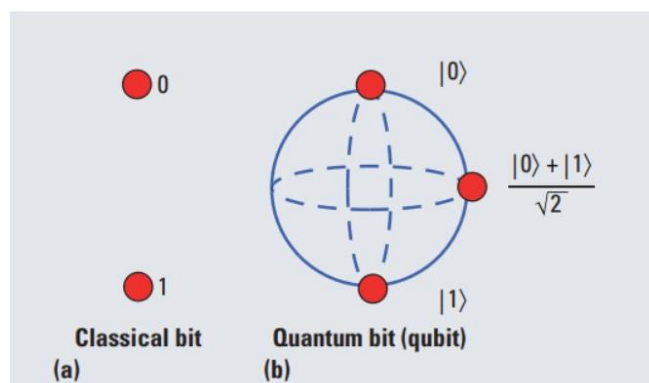


Figure 2.1: a) deterministic bit b) probabilistic quantum bit

So, the need of new algorithms has emerged, the post-quantum cryptography, which are capable of keeping safe both binary and quantum computers. Many scientists have already developed algorithms that are believed to resist classical and quantum computers. Every attempt of applying Shor's algorithm to these systems has been a failure, which establishes that these systems can keep the computers secured. The following are important classes of cryptographic systems:

- Hash-based cryptography: Hash-based cryptography are fast, and their security relies on the irreversibility of one-way functions, which are independent of any number theory problems. The uniqueness of this method is that these functions avoid collisions (matching digital signatures). However, these are not ideal cryptosystems due to their limited reuse of the private key [4].
- Code-based cryptography: Code-based cryptosystems use error correction codes to generate keys from private matrices with purposefully injected errors. Although they are quite fast and relatively safe thanks to the low complexity of the algorithm, they have a big drawback, that they need large keys, with public keys requiring millions of bits [4].
- Lattice-based cryptography: Lattice-based cryptography's speed depends on the difficulty of the complex mathematical problems. These systems, and especially the NTRU (Number Theory Research Unit), have relatively small public and private keys (Kbits) and promise quite good safety and performance, while they fit nicely with the existing global communication infrastructure [4].

2.2 Structure of Crystals-Dilithium

Dilithium is a lattice-based digital signature, built on the Learning with Errors (LWE) and Short Integer Solutions (SIS) problems. It is part of the Cryptographic Suite for Algebraic Lattices (CRYSTALS) submitted to the National Institute of Standards and Technology (NIST) PQC competition. Dilithium uses a uniform distribution to optimize the public key size instead of Gaussian sampling. This lattice-based cryptosystem has four variants spanning four security levels shown in Table 2.1. Dilithium consists of three basic components: a) key generation, b) signature generation and c) signature verification, with many subfunctions which are responsible for the computations, as shown in Figure 2.2 [1].

	Weak	Medium	Recommended	Very high
NIST security level	–	1	2	3
Parameter q	8380417	8380417	8380417	8380417
Parameter d	14	14	14	14
Parameter weight of c	60	60	60	60
Parameter $\gamma_1 = (q - 1)/16$	523776	523776	523776	523776
Parameter $\gamma_2 = \gamma_1/2$	261888	261888	261888	261888
Parameter (k, l)	(3,2)	(4,3)	(5,4)	(6,5)
Parameter η	7	6	5	3
Parameter β	375	325	275	175
Parameter ω	64	80	96	120
Signature size (bytes)	1387	2044	2701	3366
Public key size (bytes)	896	1184	1472	1760
Secret key size (bytes)	2081	2733	3348	3916

Table 2.1: Security parameters, signature size, public key size and secret key size of Dilithium variants

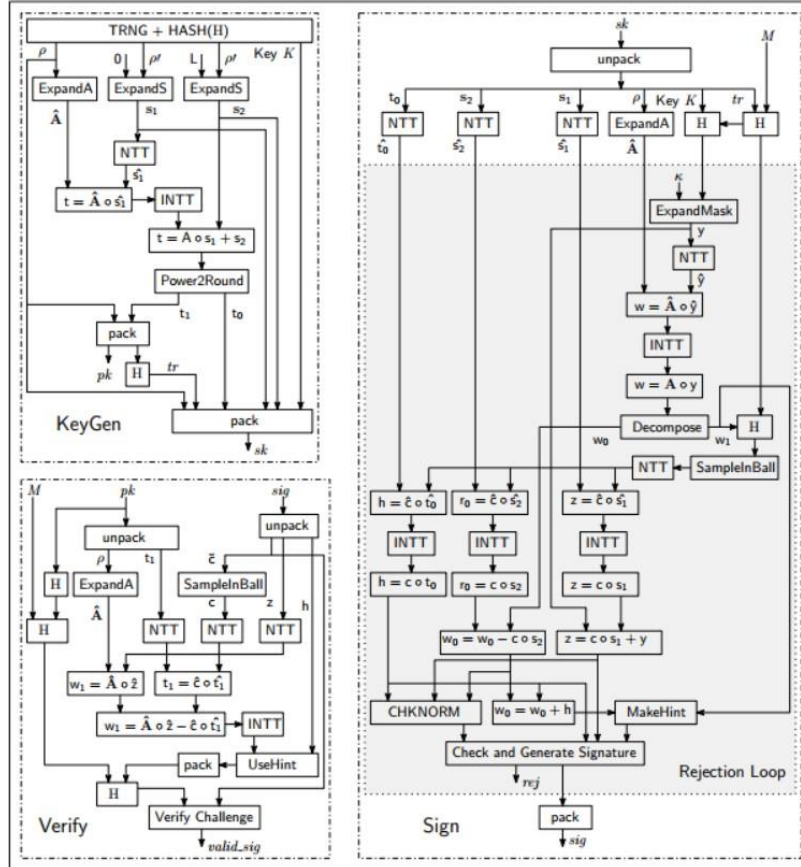


Figure 2.2: Structure of Crystals-Dilithium

2.2.1 Key Generation

The key generation component is responsible for the production of the public key, which will be used for the signature verification, and secret key, for the signature generation. At first, key generation component produces random bytes, that with the help of the hash function

shake256, which will be analyzed in section 2.2.1, the seeds ρ , ρ' and key will be created. Using ρ seed, key generation expands a $(k \times l)$ -size public matrix A . Each entry of the matrix is a polynomial of the ring $R_q = \mathbb{Z}_q[X] / (X^n + 1)$, with $q = 2^{23} - 2^{13} + 1$ and $n = 256$. The ρ' and nonce are the entries of the shake256 and are responsible for the production of the vectors s_1 sized l and s_2 sized with a uniform distribution. The value of nonce is updated with each key of the various expansion functions of matrix A . For the matrix-vector multiplication of A and s_1 , forward Number Theoretic Transform (NTT) is performed on s_1 and after each iteration a reduction of coefficients is performed to the row of the stored data. For the last step of the multiplication, inverse NTT is performed to the new vector t . Afterwards, vector t_1 stores the sum of the vector t and the uniform distributed vector s_2 , and then once again the coefficients will be reduced. Furthermore, t_1 is divided into t_0 and t_1 , and by combining t_1 and ρ seed the public key is produced. Lastly, shake256 function, with input the public key, produces tr and with the combination of the uniform distributed s_1 and s_2 , the vector t_0 , the seed ρ , key and tr the secret key is created [1].

Algorithm 1 CRYSTALS-Dilithium: Key generation

Input : No Input Required

Output : Secret Key $sk = (\rho, \text{key}, tr, s_1, s_2, t_0)$ and Public Key $pk = (\rho, t_1)$

```

1: procedure KEY GENERATION                                ▷ crypto_sign_keypair(sk, pk)
2:   nonce ← 0
3:   Generation of seed ( $\rho$ ,  $\rho'$ , key) using AES              ▷ randombytes
4:   Expand matrix  $A$  from  $\rho$ .                                ▷ expand_mat
5:   Generation of short vectors  $s_1$  and  $s_2$  from  $\rho'$ .          ▷ poly_uniform_eta
6:    $s_1 \leftarrow \text{NTT}(s_1)$ .                                  ▷ polyvecl_ntt
7:   for ( $k = 0; k < \text{lengthof } s_2; k++$ ) do
8:      $t \leftarrow A * s_1$                                      ▷ polyvecl_pointwise_acc_invmontgomery
9:     Reduce the coefficient of  $t$ .                             ▷ poly_reduce
10:     $t \leftarrow \text{NTT}^{-1}(t)$ .                               ▷ poly_invntt_montgomery
11:  end for
12:   $t \leftarrow t + s_2$                                        ▷ polyveck_add
13:  Reduce the coefficient of  $t$ .                               ▷ polyveck_freeze
14:   $(t_0 + t_1) \leftarrow t$                                    ▷ polyveck_power2round
15:   $pk \leftarrow (\rho, t_1)$                                    ▷ pack_pk
16:   $tr \leftarrow \text{shake256}(pk)$                                ▷ shake256
17:   $sk \leftarrow (\rho, \text{key}, tr, s_1, s_2, t_0)$              ▷ pack_sk
18:  return ( $sk, pk$ )
19: end procedure

```

Figure 2.3: Key generation algorithm

2.2.2 Signature Generation

Signature generation component produces the signature and its length, with inputs the secret key, the message and the message length (m_{len}), as shown in Figure 2.4. Firstly, the seeds ρ , key, tr , s_1 , s_2 and t_0 are detached from the secret key and the message is being copied to the last m_{len} inputs of sm . The function shake256 produces collision resistant hash μ , with the seed tr and the message as entries. As in key generation, matrix A is expanded using ρ and forward NTT is applied to s_1 , s_2 and t_0 . Afterwards, the algorithm will run in an infinite loop, until all the checks pass so that the signature can be written. At first, using the seed key and a

harmonized nonce an intermediate vector is generated. This nonce is incremented every time to ensure different values of the intermediate vector y for every iteration of the while loop. Then, matrix multiplication is applied, as explained in section 2.2.1, with intermediate vector y and expanded matrix A as inputs. The resultant w is decomposed in w_1 and tmp , in such way that the equality $w_1 * ALPHA + tmp = w$, applies for every element of the array. With the help of shake256, an array c is produced with 60 nonzero elements with values $[-1,1]$, and then the checking begins. The steps are as follows:

Algorithm 2 CRYSTALS-Dilithium: Signature generation

Input : Message m , Message Length m_{len} , and Secret Key $sk = (\rho, key, tr, s_1, s_2, t_0)$

Output : Signature $sm = (z, h, c)$, Signature Length s_{mlen}

```

1: procedure SIGNATURE GENERATION                                ▷ crypto_sign(sm, smlen, m, mlen, sk)
2:   nonce ← 0
3:   Extract  $\rho, key, tr, s_1, s_2, t_0$  from secret key                ▷ unpack_sk
4:   Store the input message at the end of  $sm$ .
5:    $\mu \leftarrow CRH(tr, m)$                                           ▷ shake256
6:   Expand matrix  $A$  from  $\rho$                                          ▷ expand_mat
7:    $(s_1, s_2, t_0) \leftarrow NTT(s_1, s_2, t_0)$                    ▷ polyvecl_ntt
8:   while 1 do
9:     Generate intermediate vector  $y$                                 ▷ poly_uniform_gamma1ml
10:     $w \leftarrow A * y$                                               ▷ polyvecl_ntt, polyvecl_pointwise_acc_invmontgomery,
    poly_invntt_montgomery, polyveck_freeze
11:    Decompose  $w$ ,  $(w_1, tmp) \leftarrow w$                             ▷ polyveck_decompose
12:     $c \leftarrow Hash(\mu, w_1)$                                        ▷ challenge
13:     $z \leftarrow y + c * s_1$                                          ▷ poly_ntt, polyvecl_pointwise_invmontgomery,
    poly_invntt_montgomery, polyvecl_add, polyvecl_freeze
14:    if  $(z > \gamma_1 - \beta)$  then Continue                             ▷ polyvecl_chknorm
15:    end if
16:     $w' \leftarrow w - cs_2$                                            ▷ polyvecl_pointwise_invmontgomery, poly_invntt_montgomery,
    polyvecl_sub, polyveck_freeze
17:    Decompose  $w'$ ,  $(w_{cs20}, tmp) \leftarrow w'$                      ▷ polyveck_decompose
18:    Reduce the coefficient of  $w_{cs20}$ .                               ▷ polyveck_freeze
19:    if  $(w_{cs20} > \gamma_2 - \beta) \vee (tmp \neq w_1)$  then Continue     ▷ polyvecl_chknorm
20:    end if
21:     $ct_0 \leftarrow c * t_0$                                           ▷ polyvecl_pointwise_invmontgomery, poly_invntt_montgomery,
    polyveck_freeze
22:    if  $(ct_0 > \gamma_2 - \beta)$  then Continue                             ▷ polyvecl_chknorm
23:    end if
24:     $tmp \leftarrow w_{cs20} + ct_0$                                      ▷ polyveck_add
25:    Reduce the coefficient of  $tmp$ .                                   ▷ polyveck_freeze
26:     $n \leftarrow (tmp, -ct_0)$                                          ▷ polyveck_neg, polyveck_make_hint
27:    if  $(n \neq \omega)$  then Continue
28:    end if
29:     $sm \leftarrow (z, h, c)$                                           ▷ pack_sig
30:     $s_{mlen} = m_{len} + CRYPTO\_BYTES$ 
31:    return ( $sm$ )
32:  end while
33: end procedure

```

Figure 2.4: Signature generation algorithm

1. Vector z is created ($z = y + c \times s_1$). If the infinity norm of z is higher of $\gamma_1 - \beta$, the algorithm starts all over again.

2. Similarly, vector w' is produced ($w'=w - c \times s_2$) and is decomposed into wcs_{20} and tmp . The coefficients of wcs_{20} are reduced and then if the reduced wcs_{20} is greater than $\gamma_2 - \beta$, the algorithm discards and starts a new iteration.
3. If tmp , that was created from the decomposition of w' , is not equal to w_1 , the loop starts again.
4. Then, $c \times t_0$ is generated and if it is higher than $\gamma_2 - \beta$, the while loop restarts.
5. Lastly, $tmp = w' + c \times t_0$ is calculated and then the coefficients are reduced. A vector n is produced with the help of tmp and $c \times t_0$. If n is not equal to ω , while loop jumps to the next iteration.

If all the above checks are satisfied, the infinity while loop stops, and the vectors z , h , and c are stored in the signature and the length of the signature is updated [1].

2.2.3 Signature Verification

The signature verification component verifies whether the signature is from an authenticated user that has the corresponding, signing and secret key, with the signature and the public key as inputs. At first, z, h, c and p, t_1 are extracted from the signature and the public key respectively. As happened in signature generation, the infinity norm of z is checked again and if it is higher than $\gamma_1 - \beta$ the signature is rejected. Otherwise, p , t_1 , and the message are provided as inputs to collision resistant hash function shake_{256} , which produces μ . Similarly, to the other algorithms, matrix A is expanded with p . Then, $tmp_1 = A \times z - c \times t_1$ is calculated in order to produce the hint vector w_1 . Once again, an array cp with 60 nonzero elements is created using μ and w_1 and if cp is not equal to c then the signature is rejected, otherwise the last m_{len} elements of the signature, which are the message, are copied to the output message and the signature is accepted.

Algorithm 3 CRYSTALS-Dilithium: Signature verification**Input :** Signature $sm = (z, h, c)$, Message m , Signature length $= smlen$ Public Key $pk = (\rho, t_1)$ **Output :** “Accept”/“Reject”, Message m , Message Length mle

```

1: procedure SIGNATURE VERIFICATION ▷ crypto_sign_open( $m, mlen, sm, smlen, pk$ )
2:   if  $smle < \text{CRYPTO\_BYTES}$  then return “Reject”
3:   end if
4:    $(\rho, t_1) \leftarrow pk$  ▷ unpack_pk
5:    $(z, h, c) \leftarrow sm$  ▷ unpack_sig
6:   if  $z > (\gamma_1 - \beta)$  then return “Reject” ▷ polyvecl_chknorm
7:   end if
8:    $\mu \leftarrow CRH(CRH(\rho, t_1), m)$  ▷ shake_256, shake_256
9:   Expand matrix  $A$ . ▷ expand_mat
10:   $tmp1 \leftarrow A * z$  ▷ polyvecl_ntt, polyvecl_pointwise_acc_invmontgomery
11:   $tmp2 \leftarrow c * t_1$  ▷ poly_ntt, polyveck_shiftl, polyveck_ntt,
    poly_pointwise_invmontgomery
12:   $tmp1 \leftarrow tmp1 - tmp2$  ▷ polyveck_sub, polyveck_freeze, polyveck_invntt_montgomery
13:   $w_1 \leftarrow (tmp1, h)$  ▷ polyveck_use_hint
14:   $cp \leftarrow H(\mu, w_1)$  ▷ challenge
15:  if  $(cp \neq c)$  then return “Reject”
16:  else
17:     $m \leftarrow sm[\text{CRYPTO\_BYTES}]$ 
18:    return “Accept”
19:  end if
20: end procedure

```

Figure 2.5: Signature verification algorithm

2.3 Basic Components of Dilithium

As seen in the key generation, the signature generation and the signature verification, Dilithium is based on modular multiplications, the NTT function and montgomery_reduce function, and on sponge construction functions and hash functions, the keccak-f permutation, SHAKE256 and SHAKE128.

2.3.1 Number Theoretic Transform

Number Theoretic Transform (NTT) is a Discrete Fourier Transform (DFT) over a finite field or ring. Therefore, NTT works with polynomials having (modulo) integer coefficients. The forward NTT converts an $n-1$ degree polynomial with the modulo p coefficients $(a_0, a_1, \dots, a_{n-1})$ having the generic form:

$$f(x) = a_0x^0 + a_1x^1 + \dots + a_{(n-1)}x^{(n-1)} \quad (1)$$

into a $n-1$ degree polynomial with modulo p coefficients $(A_0, A_1, \dots, A_{(n-1)})$ having the generic form:

$$F(X) = A_0X^0 + A_1X^1 + \dots + A_{(n-1)}X^{(n-1)} \quad (2)$$

The coefficients of the outcome of the forward NTT are computed as, using ω (twiddle factor), which is a primitive n -th primitive root over the polynomial ring:

$$A_i = \sum_{j=0}^{n-1} a_j \omega^{ij} \cdot \text{mod } p \quad (3)$$

And the coefficients of the outcome of the inverse NTT are computed as, using ω similarly with the forward NTT:

$$a_i = n^{-1} \sum_{j=0}^{n-1} A_j \omega^{i \cdot j} \bmod \rho \quad (4)$$

NTT exists if for some integer k , $p = kn + 1$, $\omega^n = 1 \pmod{p}$ (root of unity), and $\forall i < n$, $\omega^i \neq 1$. The prime numbers p for NTT-based lattice cryptography are chosen to satisfy these conditions[5].

The NTT-based polynomial multiplication is selected because it reduces the complexity from $O(n^2)$ to $O(n \log n)$. This reduction benefits the lattice-based cryptosystems since a typical value of n range from 256 to 1024[5].

NTT is a DFT over a finite field or ring, which means that after the calculations, the indexes of the output array will be bit-reversed (Figure 2.6). The exact opposite occurs to the inverse NTT where the input indexes are bit-reversed and the output returns the array in its canonical form. The Figure 2.7 shows the stages from the input till the output array, which will be $\log_2 N$, in our case $N=256$, so stages = 8 and Figure 2.8 depicts the butterfly unit that implements the calculations of the stages.

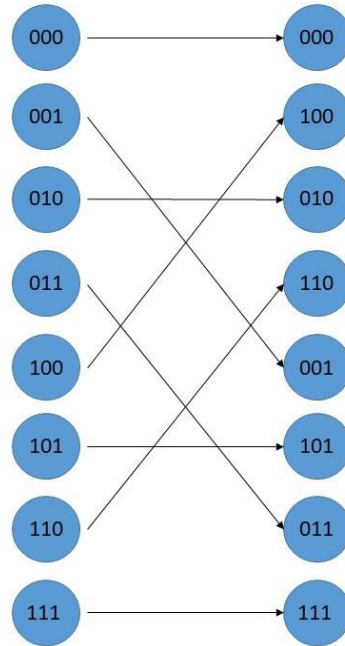


Figure 2.6: Bit reverse

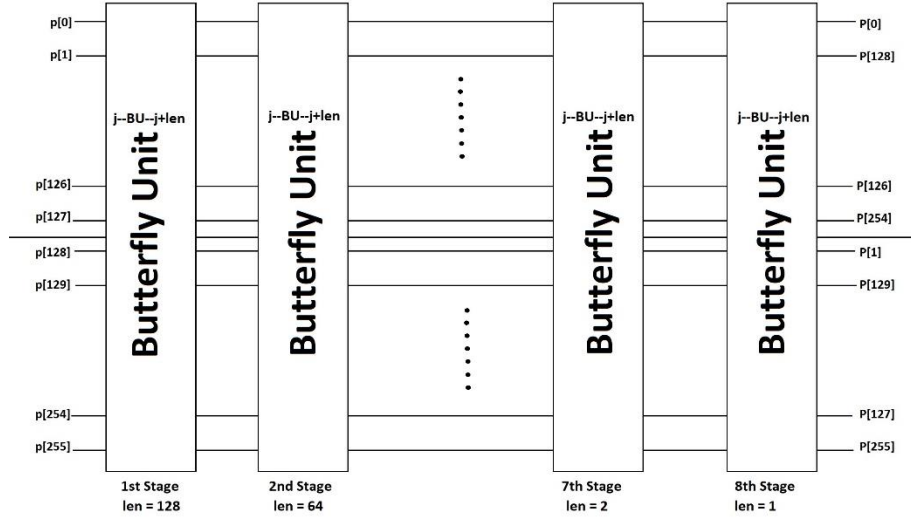


Figure 2.7: NTT stages

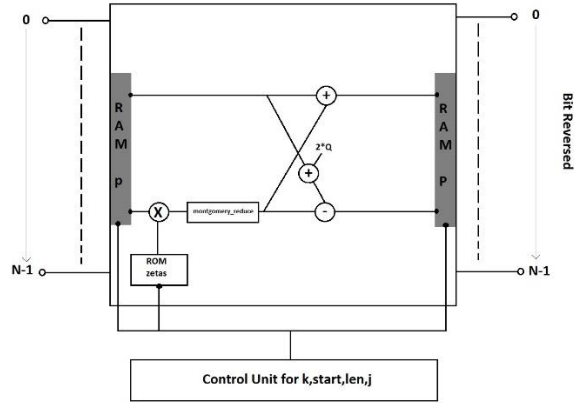


Figure 2.8: Butterfly unit

With NTT, multiplications in the finite ring R_q can be accomplished efficiently in hardware. NTT uses similar method with the Fast Fourier Transform and can be obtained by replacing $e^{-2\pi i k/N}$ with the respective powers of the N th primitive root of unity in Z_q denoted by ω_N , and doing transformations over the field Z_q . The result of the multiplication can be computed by performing forward NTT to the two polynomials a and b , then multiply the results and lastly by performing inverse NTT to the outcome of the multiplication, as shown in eq. (5).

$$c = \text{INT T}^{-1}(\text{NT T}(a) * \text{NT T}(b)) \quad (5)$$

With this method of multiplication, the complexity of the computation is $O(n \log n)$, as mentioned before [6].

2.3.2 Montgomery Reduction Algorithm

Many cryptosystems are based on arithmetic operations, such as multiplications, modulo a large number, which can be so expensive and time consumer computationally. The most common method is to multiply the two integers and then take the modulo of the result, but this method is as expensive as dividing two numbers.

In [7], Peter L. Montgomery, proposed a method for multiplying two integers, called N-residues, modulo N while avoiding any division by N. Essentially this method calculates the (6), where a , b and N are k -bit binary numbers.

$$c = a \times b \pmod{N} \quad (6)$$

This algorithm computes the outcome by replacing the division by n operation with division by a power of 2, which is easily accomplished by a classical computer[8].

The Montgomery reduction algorithm requires that n , which is a modulus k -bit number, i.e., $2^{k-1} \leq n < 2^k$, and r , which we assumed it is 2^k , are relatively prime, i.e., $\gcd(r, n) = 1$. This requirement is satisfied if n is odd number. Given an integer $a < n$, we define its n residue with respect to r as

$$\bar{a} = a \times r \pmod{n} \quad (7)$$

It can be easily understood that $\bar{a}$ contains all the numbers between 0 and $n-1$. The Montgomery reduction algorithm introduces a much faster multiplication method which computes the n -residue of the product of the two n -residue integers. Given the two n -residue integers $\bar{b}$ and $\bar{a}$, the Montgomery result is defined as,

$$\bar{c} = \bar{a} \times \bar{b} \times r^{-1} \pmod{n} \quad (8)$$

where r^{-1} is the inverse r modulo n and the quality (9) holds.

$$r \times r^{-1} = 1 \pmod{n} \quad (9)$$

2.3.3 Sponge Construction

A sponge function or sponge construction (see Figure 2.9) is a class of cryptographic algorithms with finite internal state that with an input bit stream of any length produces an output bit stream of any desired length. Sponge function consists of a state memory S , that is divided into r (the bitrate), and the remaining part is the capacity, a permutation function f that produces pseudorandom permutations and a padding function P that appends bits to the input bit stream so that the input string is a whole multiple of the bitrate r . The function, at first, “absorbs” all blocks of the padded string by initializing the S to zero and then, for each r -bit block B , of the message, by XORing R , r -bit block of the state, with B and by replacing S with $f(S)$, i.e., the message bits go through permutations. This procedure continues until all the inputs have been held. Lastly, to “squeeze” and produce the output, R , which is portion of S , is extracted and S is replaced by $f(S)$, repeatedly. If the output has less than r bits, then R will be truncated [9][10].

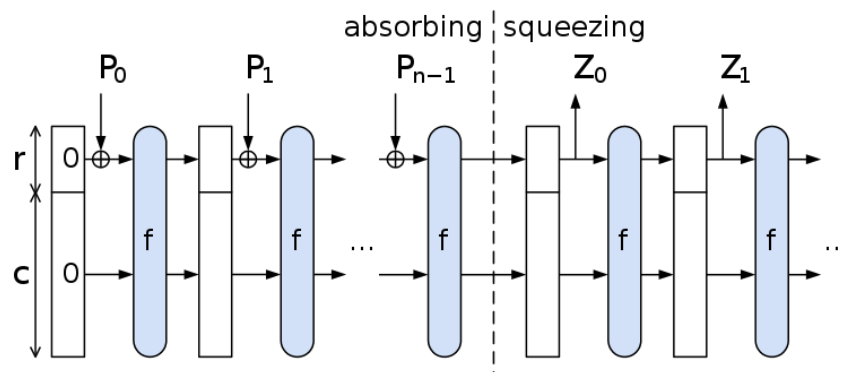


Figure 2.9: The sponge construction of hash functions

Padding

To ensure that the message can be evenly divided into r -bit blocks, the padding technique is required. In SHA-3 hash functions, the pattern 10^*1 is used, i.e., 1 bit, followed by zeros (maximum $r-1$) and a final 1 bit. The condition of $r-1$ zeros occurs when the last block is $r-1$ bits. Even when the length of the message is already divisible by r , the process of adding two 1 bits takes place. In this situation, an extra block is appended to the message, having the MSB and LSB 1 bit and the intermediate $r-2$ bits are zero. This step ensures that when the message's length is a multiple of r and ends with a pattern that of a padding, it will not yield the same hash as the message obtained by removing these bits. The first 1 bit is important because it prevents the generation of identical hashes for messages that vary only by a few zero bits. Additionally, the final 1 bit serves a purpose in indicating which rate r was utilized during the process of multi-rate padding. Without this distinction, various hash versions of small messages could become identical upon truncation [11,12].

Capacity

Is the area of bits that is untouched by the input or output, meaning new absorption of data will not change these bits, but the function on the newly absorbed data will. None of the capacity bits are used for the output of the hash function [11].

Rate

Rate, known as bit rate size, is the size of data blocks that will be absorbed by the function [11].

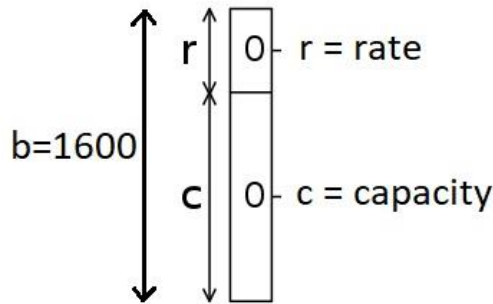


Figure 2.10: Relation between capacity, rate and b

2.3.4 Keccak-f permutation

Keccak is a family of hash functions that is based on the sponge construction that has been standardized in the form of SHAKE128 and SHAKE256 extendable output functions (XOF) hash functions in FIPS 202 [13].

The Keccak permutations are specified with two parameters: 1) the fixed length of the strings that are permuted, called the width, and 2) the number of iterations of an internal transformation, called round. The permutation is defined for any width b in $\{25, 50, 100, 200, 400, 800, 1600\}$ and any positive integer of round. The permutation is specified in terms of an

array of values for b bits that is being updated, called the state. This state contains two more quantities that are related to b , $w = b/25$ and $l = \log_2(b/25)$, and so we have seven possible values, as shown in Table 2.2 [14].

b	25	50	100	200	400	800	1600
w	1	2	4	8	16	32	64
l	0	1	2	3	4	5	6

Table 2.2: Width and related quantities

The Figure 2.11, as shown below, is illustrating parts of the state memory S , for the case of $b=200$ and $w=8$, so that it helps to analyze the step mappings, that were mentioned before.

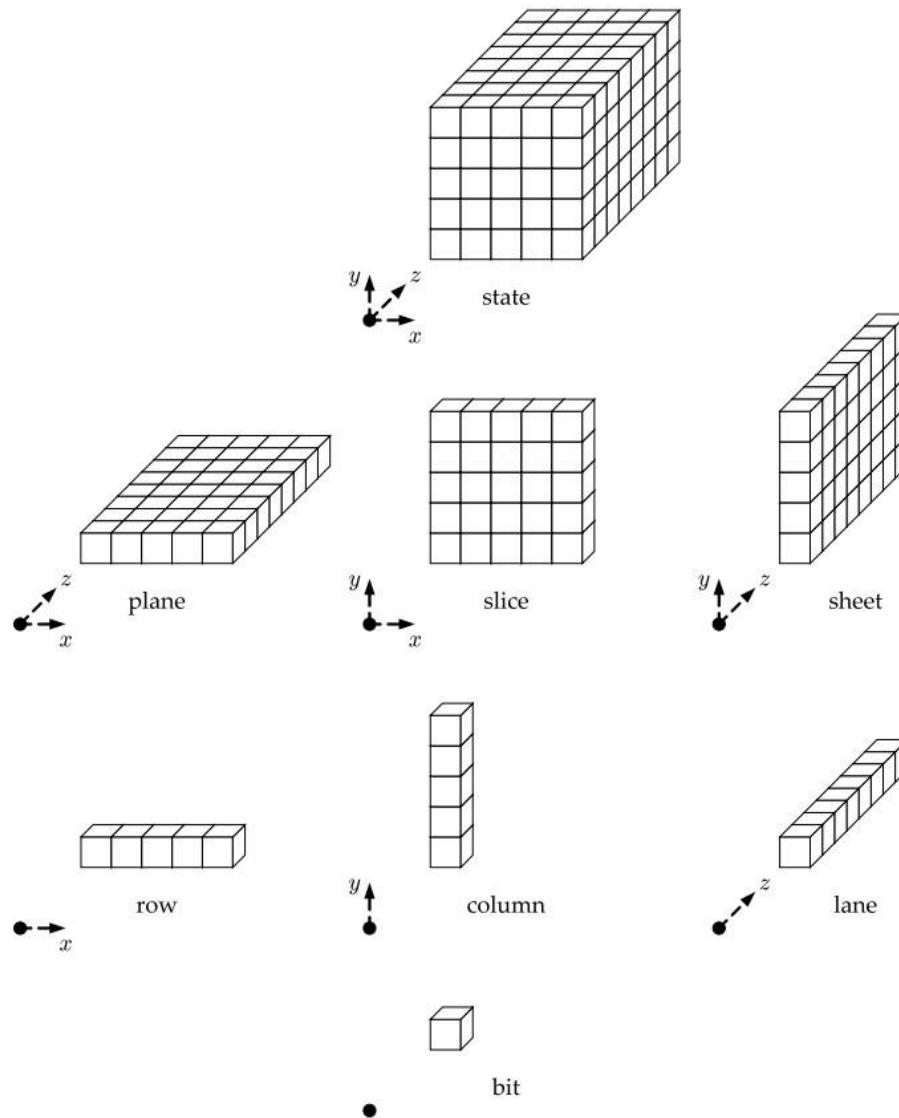


Figure 2.11: Parts of the state [15]

- *Row*: set of 5 bits with constant y and z coordinates.
- *Column*: set of 5 bits with constant x and z coordinates.
- *Lane*: set of w bits with constant y and x coordinates.

- *Sheet*: set of 5w bits with constant x coordinates.
- *Plane*: set of 5w bits with constant y coordinates.
- *Slice*: set of 25 bits with constant z coordinates.

Each round of Keccak-f permutation is composed from a sequence of mappings. The steps have a simple description leading to a specification that is compact and in which no trapdoor can be hidden. There are five steps that each one computes θ , ρ , π , χ and ι respectively, which will be discussed in the bellow sections.

1. Computation of θ

The θ mapping is linear and aims at the diffusion and its translation-invariant in all directions. It adds to each bit $a(x,y,z)$ the bitwise sum of the parities of two columns $a(x+1,y,z)$ and $a(x-1,y,z)$. The eq. (10), (12) and (14) [10] show the calculations needed for the θ computation. In the Figure 2.12 the symbol Σ indicates the XOR sum of all bits in the column [15].

$$C(x,z) = S(x,0,z) \oplus S(x,1,z) \oplus S(x,2,z) \oplus S(x,3,z) \oplus S(x,4,z) \quad (10)$$

$$0 \leq x < 5 \text{ and } 0 \leq z < w \quad (11)$$

$$D(x,y) = C[(x-1) \bmod 5, z] \oplus C[(x+1) \bmod 5, (z-1) \bmod w] \quad (12)$$

$$0 \leq x < 5 \text{ and } 0 \leq z < w \quad (13)$$

$$S'(x,y,z) = S(x,y,z) \oplus D(x,z) \quad (14)$$

$$0 \leq x < 5, 0 \leq y < 5 \text{ and } 0 \leq z < w \quad (15)$$

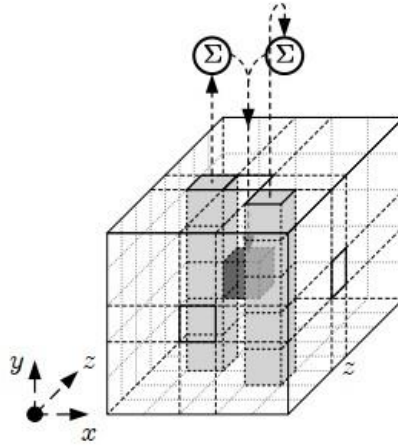


Figure 2.12: Illustration of θ applied to a single bit [15]

1. Computation of ρ

The variable ρ consists of translations within the lanes aimed at providing inter-slice dispersion. Without it, diffusion within the slices would be very slow. It is translation-variant in the z direction. The lane in coordinates (x,y) goes to position $(x,y)M^T$, with $M = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$. The eq. (16) and (19) [10] show the calculations needed for the θ computation. In the Figure 2.13 is being illustrated the effect of ρ , which is to rotate the bits of each lane by a length, called rotation

offset, which depends on the x, y coordinates and it is depicted in Table 2.3 for the case of $b=1600$ and $w=64$ [14,15].

$$A(0,0,z) = S'(0,0,z) \quad (16)$$

$$0 \leq z < w \quad (17)$$

$$\text{Let } (x,y) = (1,0) \quad (18)$$

$$A(x,y,z) = S'[x,z, \left(z - \frac{(t+1)(t+2)}{2}\right) \bmod w] \quad (19)$$

$$(x,y) = (y, (2x + 3y) \bmod 5) \quad (20)$$

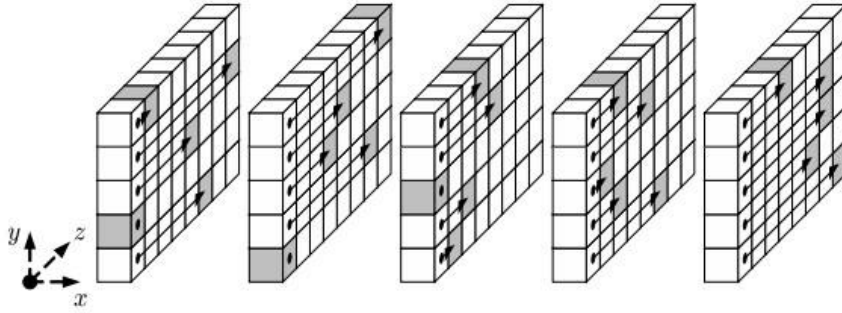


Figure 2.13: Illustration of ρ [15]

	$x=3$	$x=4$	$x=0$	$x=1$	$x=2$
$y=2$	25	39	3	10	43
$y=1$	55	20	36	44	6
$y=0$	28	27	0	1	62
$y=4$	56	14	18	2	61
$y=5$	21	8	41	45	15

Table 2.3: The rotation offsets for $b=1600$

There is $t \in [0,25)$ rotation offsets. The methodology to calculate the above numbers is the following:

- First, find the number of rotations: $\rho(t) = (t+1)(t+2)/2$.
- Then, compute x and y : $\begin{pmatrix} i \\ j \end{pmatrix} = M^t \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \bar{x} \\ \bar{y} \end{pmatrix}$
 $x = \bar{x} \bmod w$
 $y = \bar{y} \bmod w$

2. Computation of π

The variable π is a transposition of the lanes that provides dispersion aimed at long-term diffusion. It operates in a linear way on the x, y coordinates and it is translation-invariant in the z direction. Essentially it is responsible for the rearrangement of the positions of the lanes, as shown in Figure 2.14, where this bit at the center of the slice has $(x,y) = (0,0)$ [14,15]. The eq. (21) shows the calculations needed for the π computation.

$$A'(x, y, z) = A[(x + 3y) \bmod 5, x, z] \quad (21)$$

$$0 \leq x < 5, 0 \leq y < 5, \text{ and } 0 \leq z < w \quad (22)$$

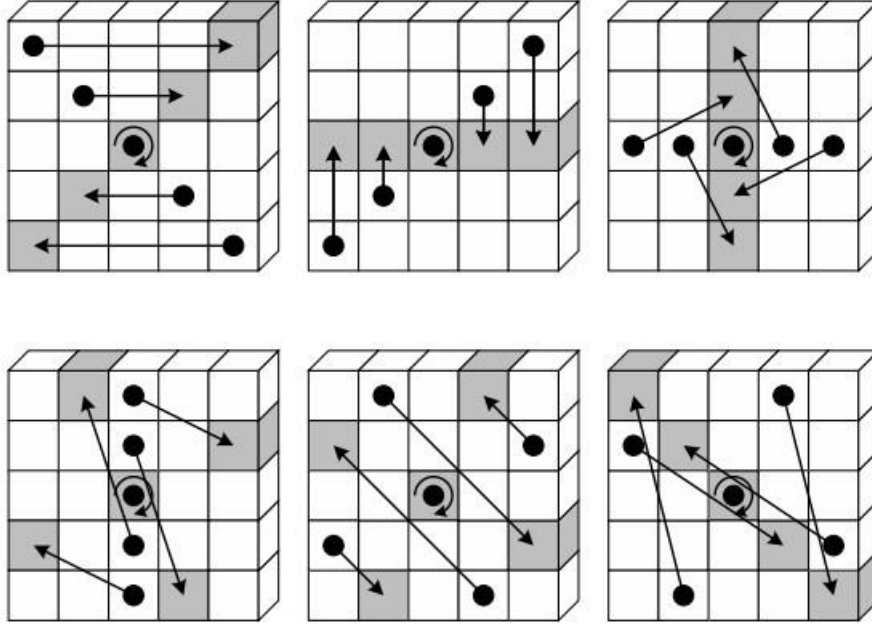


Figure 2.14: Illustration of π in a single bit [15]

3. Computation of χ

The variable χ is the only non-linear step in the permutation process. It can be seen as the parallel application of $5w$ S-boxes operating on 5-bit rows. It is translation-invariant in all directions and has algebraic degree of two. The eq. (23) shows the calculations needed for the χ computation. As in Figure 2.15, χ is XORing each bit with a non-linear function of two other bits in its row [14,15].

$$S(x, y, z) = A'(x, y, z) \oplus [\text{NOT } [A'((x + 1) \bmod 5, y, z)] \text{ AND } A'((x + 2) \bmod 5, y, z)] \quad (23)$$

$$0 \leq x < 5, 0 \leq y < 5 \text{ and } 0 \leq z < w \quad (24)$$

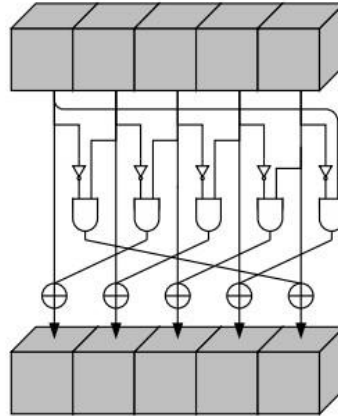


Figure 2.15: Illustration of χ applied on a single row [15]

4. Computation of ι

The variable ι consists of the addition of round constants and is aimed at disrupting symmetry. The bits of the round constants differ from round to round and are taken as the output of a maximum-length linear feedback shift register (LFSR). The constants are added in a single lane of the state and because of this, the disruption diffuses through θ and χ to all lanes of the state after a single round. The impact of ι is to alter some of the bits of lane (0,0), depending of the round index, without changing the remaining 24 lanes [14,15]. The eq. (25) shows the calculations needed for the ι computation, with the help of the round constants, which are shown in Table 2.4.

$$S(0,0,z) = S(0,0,z) \oplus RC(z) \quad (25)$$

$$0 \leq z < w \quad (26)$$

RC[0] 0x0000000000000001	RC[12] 0x000000008000808b
RC[1] 0x0000000000008082	RC[13] 0x800000000000008b
RC[2] 0x000000000000808a	RC[14] 0x8000000000008089
RC[3] 0x8000000080008000	RC[15] 0x8000000000008002
RC[4] 0x000000000000808b	RC[16] 0x800000000000808b
RC[5] 0x0000000080000001	RC[17] 0x8000000000000080
RC[6] 0x8000000080008081	RC[18] 0x000000000000800a
RC[7] 0x8000000000008081	RC[19] 0x800000008000000a
RC[8] 0x000000000000008a	RC[20] 0x8000000080008081
RC[9] 0x0000000000000088	RC[21] 0x8000000000008080
RC[10] 0x0000000000008082	RC[22] 0x0000000080000001
RC[11] 0x000000008000000a	RC[23] 0x8000000080008008

Table 2.4: Round constant

2.3.5 Secure Hash Algorithm KEccak (SHAKE)

Hash functions play a pivotal role in modern cryptography and information security. They are important components for various applications, such as password storage, digital signature etc. They transform input data of arbitrary size (e.g., text message) to a result of a fixed size, known as hash value.

SHA-3 (Secure Hash Algorithm 3) was released by NIST on August 5, 2015. SHA-3 is a subset of the broader cryptographic primitive family Keccak. The SHAKE function is an Extendable-Output function (XOF), which belongs to the SHA-3 family, but with an infinite output. It is based on the sponge construction and keccak-f permutation, that were analyzed in 2.3.3 Sponge Construction and 2.3.4 Keccak-f permutation.

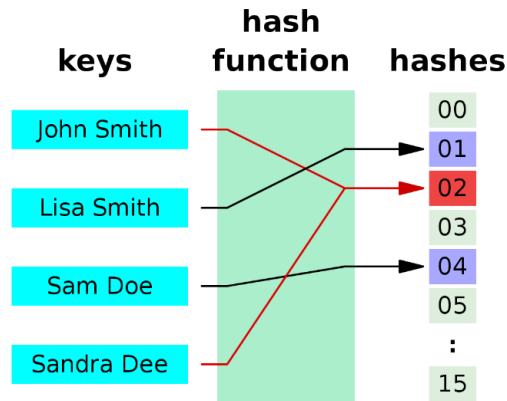


Figure 2.16: Illustration of hash function [12]

In Figure 2.17, Figure 2.18, Figure 2.19 and Figure 2.20, can be understood the impact of two texts that differ only by a letter, which can cause the output to change with 50% probability [12], demonstrating the avalanche effect, the impact of different output byte size and the difference between SHAKE128 and SHAKE256.

```
Bytes = 8
my name is marios: 5d652a50039e6dc2
my name is mario: 9d3ae7cb9e03c0ed
```

Figure 2.17: Impact of different Byte (SHAKE256)

```
Bytes = 8
my name is marios: 881c223255188ac8
my name is mario: 2592a3ffb1482db4
```

Figure 2.18: Impact of different Byte (SHAKE128)

```
Bytes = 64
my name is marios: 5d652a50039e6dc2d2382a9b41f3ef7c705e5c5792c6ffc719a749e66232ea110
b4dbbd78de181fd8c9aef5124a1f8f64d2bada36ea1f48e48e575b5324f0047
my name is mario: 9d3ae7cb9e03c0ed4d007ca605ad8b325d74a32b771076be3dac1131a357512e13
c7d4c2528b742bb7483ece5bd702045f4445414001e72b73534a51f8446267
```

Figure 2.19: Impact of different output size (SHAKE256)

```
Bytes = 64
my name is marios: 881c223255188ac877dd06b40e8b4073a1503031d4e9ba3808f24a28916
e2889082ec5d4289d8a06daf7481aab1ece324f5359609b5c5fffb6f908d91e019749
my name is mario: 2592a3ffb1482db46610dc795a00ddc1fa2b9a317c08be8472792f6070a1
c4b9224a37cbc701e869ca2010764f29c576e6621433fdd0818474e134d768e1c0d9
```

Figure 2.20: Impact of different output size (SHAKE128)

2.3.6 Power2Round and Decompose

Power2Round and Decompose are used to break up elements in $\mathbb{Z}_q$ into their “high-order” bits and “low-order” bits. The former is a straightforward bitwise “Break” of an element r into $r = r_1 \times 2^d +$

r_0 , with $r_0 = r \bmod \pm 2^d$ and $r_1 = \frac{r-r_0}{2^d}$. This function is quite simple and is used only in the key generation function.

The latter computes high and low bits r_1 and r_0 such that $r = r_1 \times ALPHA + r_0$, with $\frac{-ALPHA}{2} < r_0 < \frac{ALPHA}{2}$, except for the border case. For the border case, when $Q - \frac{ALPHA}{2} < r < Q - 1$, the high bits are cleared to zero and the low bits to one, as shown in Table 2.5 [16].

Input r	0	1		ALPHA/2	ALPHA/2+1		3*ALPHA/2	Q-ALPHA/2		Q-1
								BORDER CASE		
Output r0	0	1		ALPHA/2	-ALPHA/2+1		ALPHA/2	-ALPHA/2		-1
Output r1	0	0		0	1	1		1	0	0

Table 2.5: Data correspondence for Decompose [16]

Chapter 3

High Level Synthesis

3.1 Introduction

High Level Synthesis, also referred to as Electronic System Level Synthesis (ESL) or algorithmic synthesis, is an automated design process that translates a circuit description into a level behavior or algorithm to a structural description that performs the given operation.

The material can be designed at various levels of abstraction. The most common abstraction layers are the gate level, the register transport level (RTL) and the algorithmic level.

Although logical synthesis uses an RTL description of the design, for high-level synthesis a higher level of abstraction is used, which includes an algorithmic description in a high-level language such as SystemC, Matlab and ANSI C/C++. High-level tools explore the architecture and convert the functional code into RTL implementations with details of each execution cycle. The RTL implementations are then used, directly, in a conventional flow of logic synthesis to create an implementation at the gate level.

In this way, a design flow that includes synthesis of high layer allows fast implementation of algorithms. Thanks to automated algorithms, HLS tools create an RTL architecture that respects the system constraints and which is more reliable (introduces less error) than manual implementation. The acceleration of the design time is enhanced compared to manual development of code level transport. High level synthesis can also restrict the complexity of the material (the number of resources allocated) and to achieve a given implementation speed.

3.2 Vivado High Level Synthesis

The Xilinx High-Level Synthesis tool transforms a C specification into a register transfer level (RTL) implementation that you can synthesize into Xilinx field programmable gate array (FPGA). The tool receives a C/C++ or SystemC code and transforms it immediately into RTL so that the synthesis can be achieved into a FPGA, as shown in Figure 3.1. FPGA provides a parallel architecture with benefits in performance, cost and power over traditional processors [17].

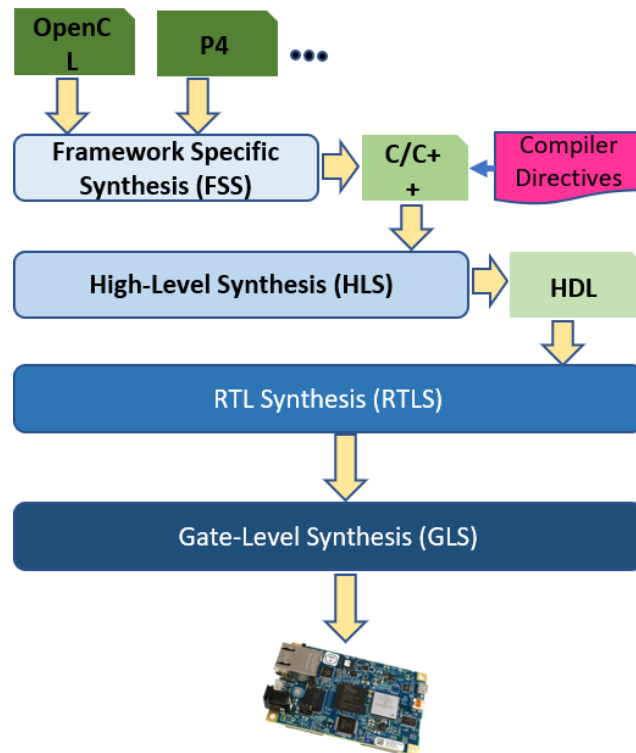


Figure 3.1: Design flow of FPGA

3.2.1 Vivado High Level Synthesis Design Flow

The Figure 3.2 below shows in detail the steps that the Xilinx Vivado HLS tool follows from the implementation of the high-level code to the gateway level.

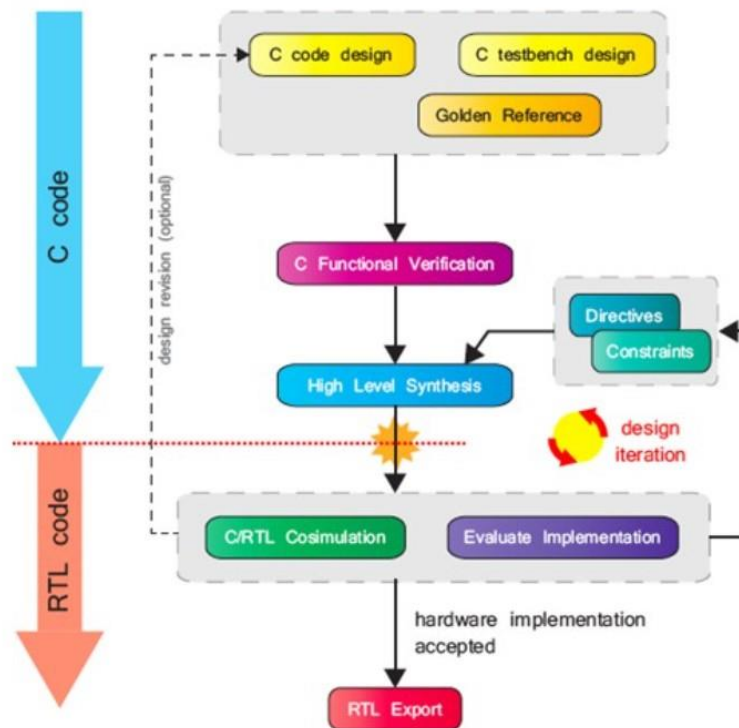


Figure 3.2: Design flow of Vivado HLS tool [17]

1. Inputs

Languages, such as C/C++ and SystemC, are used as inputs of the HLS tool, which give a description of the desired implementation. This description is linked directly with a testbench file, through which the correct operation of the implementation can be confirmed.

2. Functional Verification

C code, together with the testbench, are compiled to verify the correct production of the output data.

3. High Level Synthesis

The necessary constraints and directives are introduced for the description of the RTL circuit. Before starting the HLS process, the option to choose the preferred HDL language is given to the user, and the desired simulator. When the process is completed, the tool produces an IP (Intellectual Property) circuit with a set of output files including the design files in RTL language. IP is essentially the design that originally described at a high (algorithmic, behavioral) level, which can be used in each design as a separate block.

4. C/RTL Co-Simulation

After the IP is created, the next step is the verification of the design using C/RTL Co-simulation in Vivado HLS. The inputs provided by testbench are given to the RTL code that has been produced immediately, and the simulator is being used to verify the output data of the design.

5. Evaluation of implementation

After the IP is created and the simulation through the C/RTL Co-simulation, the design will be constrained (HLS constraints and directives) and will be implemented on an FPGA device. This step of the implementation will be evaluated continuously using some metrics, such as resources, delay, max frequency, energy consumption, etc.

6. Optimization

This step has to do with the optimization of the timing and resources of the implementation. If the performance of the implementation does not meet the desired expectations, in this step a new optimization through changing the C code can be implemented. In order to obtain the desired results, there is a possibility to repeat the optimization process quite a few times using new directives and constraints.

7. Export to RTL

When the optimizations meet the desired expectations, the design will be integrated as IP in another circuit. This integration can be achieved by either using directly the RTL sources (VHDL or Verilog) or packaging the design in order to be integrated with libraries of other Xilinx tools, such as Vivado IDE and System Generator [18].

3.2.2 High Level Synthesis Benefits

High Level Synthesis offers a bridge between the hardware and software domains, yielding a range of key advantages. For hardware designers, it enhances productivity by enabling work

at a more abstract level while still producing high-performance hardware. Concurrently, software developers benefit from heightened system performance by efficiently accelerating computationally intensive segments of their algorithms on a novel compilation target, the FPGA. The high-level synthesis design approach affords the ability to conceptualize algorithms at the C-level, liberating developers from the intricacies of implementation details that traditionally consume significant development time. Furthermore, functional validation can be expedited at the C-level, surpassing the time required when using conventional Hardware Description Languages, such as VHDL or Verilog. The process of C synthesis can be finely controlled through optimization directives, as was mentioned before, thereby creating high-performance custom hardware implementations. Taking advantage of these optimization directives makes it more feasible to create multiple implementations from C-code, thus facilitating an extensive exploration of the design space and increasing the probability of discovering the optimal implementation. Additionally, this approach provides readable and portable C code that can be easily targeted for various devices and seamlessly integrated into new projects [17].

3.2.3 Optimization methods of Vivado HLS tool

As mentioned before, Vivado HLS provides many and different choices of optimizations to achieve the desired resources and timing performance that the designer has set.

The directives that will be used to optimize the algorithm of Dilithium are the following:

-  **Function Inline**
-  **Pipeline**
-  **Unroll**
-  **Allocation**
-  **Dependence**
-  **Array partition**

Function Inline

This particular directive removes a function as a separate entity in the hierarchy. After inlining, the function is dissolved into the calling function and no longer appears as a separate level of hierarchy in the RTL. In some cases, inlining a function allows operations within the function to be shared effectively. An inlined function cannot be shared. This can cause the increase of resources required. It can be used to bring logic or loops up one level of hierarchy. Sometimes it is preferable to pipeline the logic in a function by including the logic in the function above it [19,20].

Pipeline

The PIPELINE is one of the most commonly used directives applied in the design. This is because it can provide dramatic results in terms of reducing the latency [19,20].

This directive can be applied in both loops and in functions and is based on the three stages of data processing:

- Read (RD)
- Compute (CMP)
- Write (WR)

Therefore, the optimization is based on the autonomy and separation of the above stages. Different data have the ability to be read, to be processed and to be written at the same time, which leads to parallelism of the different processes, and it is no longer necessary to complete the previous process for the next one to start its operations. The Figure 3.3 is illustrating the benefits of the parallelism.

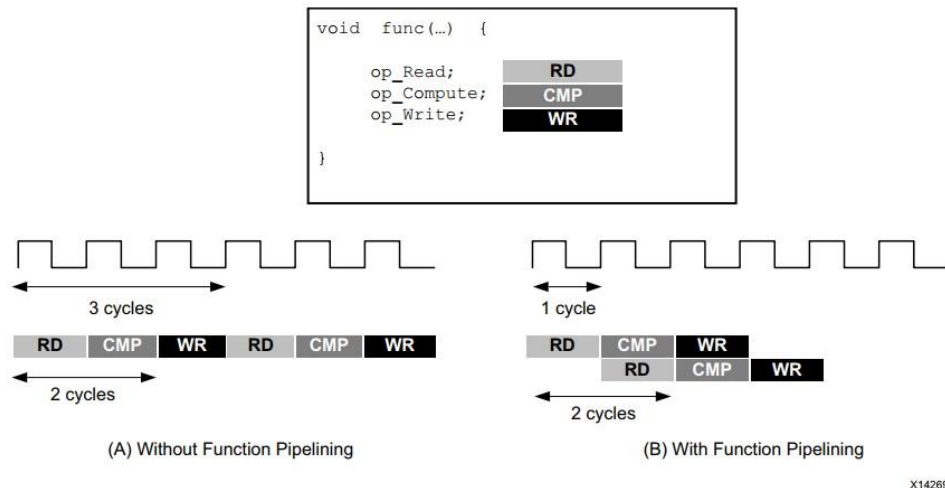


Figure 3.3: The appliance of Pipeline directive

The above figure illustrates that without pipelining, the operations execute sequentially until the function completes. Then the next execution of the function is performed. With pipelining, the next execution starts as soon as the hardware resource becomes available. This can be achieved with the help of intermediate registers that are inserted between the stages, which keep the data, until they can be reused.

The outcome of the appliance of the PIPELINE directive is the reduction of the clock cycles that the function or the loop is needed to start processing new data (Initiation Interval). The optimal is always to achieve $II=1$, which means that the function needs one clock cycle to start processing new data. But it is not always achievable due to dependencies [19,20,21].

Unroll

Loops in Vivado HLS, without the appliance of the UNROLL directive, reuse the same resources in each iteration. Specifically, the synthesis of the design creates the logic for one iteration and the RTL circuit reuses this logic for all iterations.

The UNROLL directive allows the loop to be unrolled, which essentially creates many copies of the loop body in RTL level and allows the loop to be executed concurrently. In other words, unrolling loops increase data accessing and throughput.

The loops can be either partially unrolled or fully unrolled. Partially unrolling a loop lets you specify a factor N , to create N copies of the body and reduce the loop iterations accordingly. The only condition to fully unroll a loop is the loop bounds to be known, and to have the optimal results is the loop to be freed of dependencies.

The Figure 3. 4 is illustrating the data accessing in a rolled loop, in a partially unrolled loop and in a fully unrolled loop [19,20,21].

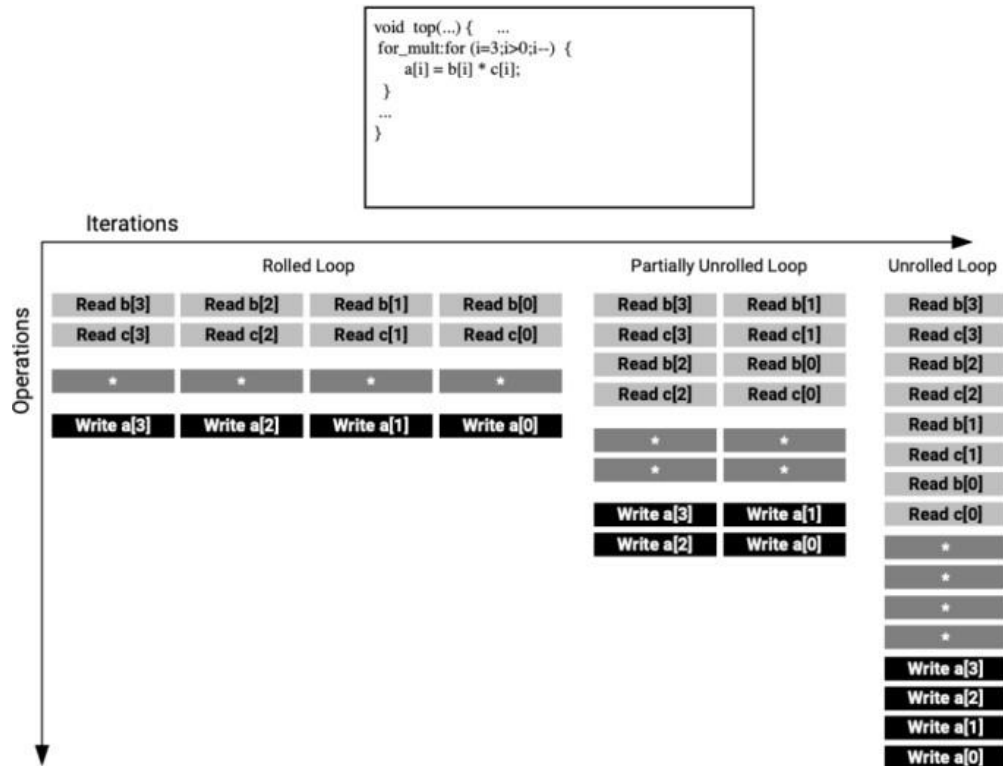


Figure 3. 4: Partially unrolling and fully unrolling

Allocation

The ALLOCATION directive specifies instance restrictions to limit the resource allocation in the implemented kernel. With the help of this directive, the number of RTL instances and hardware resources that are used to implement functions, loops or cores, are restricted. For example, if the C source has N instances of a function, the ALLOCATION pragma will ensure that in the final RTL there will be only one instance of this function, which means that all N instances will be implemented using the same RTL block. This helps for the reduction of the resource utilization.

Operations in C code, such as additions, multiplications, array read, and writes, can be restricted by the ALLOCATION directive. Cores, which operators are mapped to during synthesis, can be limited in the same way as the operators [19,20,21].

Dependence

The DEPENDENCE directive is used to provide further information to the Xilinx tool, to whether a loop-carried dependency can be overcome or not. This depends on whether an iteration depends on another iteration or not. If it depends, the DEPENDENCE TRUE directive provides the information of an existence of a dependency and if not, the DEPENDENCE FALSE provides the opposite information.

The `DEPENDENCE` directive is applied in loops, usually when the index of an array cannot be specified by the Vivado HLS tool, so it, spontaneously, thinks that a loop-carried dependency exists [19,20,21].

Array Partition

The `ARRAY_PARTITION` directive makes it possible to partition an array in smaller arrays, or even in individually elements. So, in the RTL, multiple small memories or multiple registers are created, instead of one big memory.

This directive is so important because memories have limited accesses for reads and writes, which restricts the load and store operations of a function and leads to value of Initiation Interval greater than 1. So, by partitioning an array, the accesses of a memory are increased.

The `PARTITION` directive can be applied to an array in three different ways, as shown in Figure 3.5:

- ❖ Complete: the initial array divides into individual elements. For an array of one dimension, the memory, technically means that it is partitioned in individual registers.
- ❖ Block: the array is partitioned into blocks of the same size, where the elements are placed serially.
- ❖ Cyclic: the array is partitioned into blocks of the same size, where the elements are placed alternately

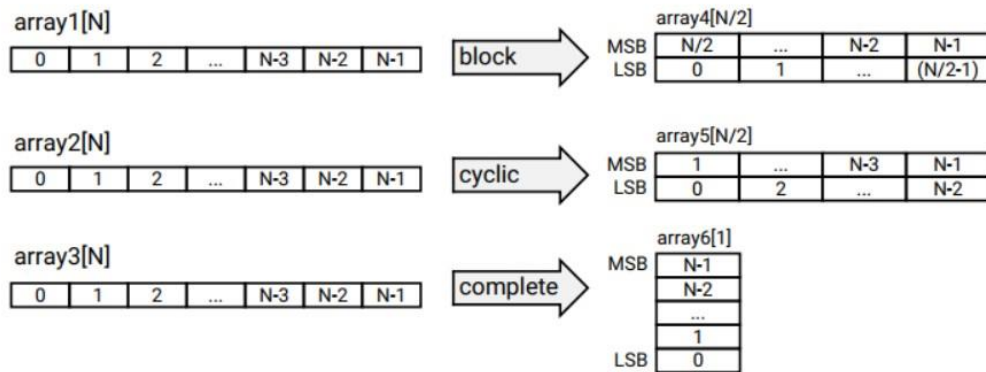


Figure 3.5: Array partition directive

The `ARRAY_PARTITION` directive can provide parallelism to a great degree and can lead to an important optimization for the performance. For this reason, it can be combined with the `UNROLL` directive, so `II=1` can be achieved [19,20,21].

3.2.4 Optimization methods changing the source code

Applications tend to spend a significant amount of their execution time in a few sections dominated by loops or nested loops. Consequently, loops usually represent the hotspots in an application, i.e., significant areas that has the highest impact on performance, energy consumption, and power usage. The list of loop transformations that Vivado HLS can apply is very extensive and only support a few [22].

Some of the loop transformations that were used to optimize the algorithm of Dilithium are:

- ✚ Loop Coalescing
- ✚ Loop Normalization
- ✚ Loop Splitting
- ✚ Loop Fusion

Loop Coalescing

Loop Coalescing combines nested loops into a single loop, reducing to a single induction variable, and computing the indices from the variable. This reduces overhead and allows better load balancing. It also can improve the instruction scheduling. The con of this technique is that it adds instruction complexity as it introduces the modulo and division to compute the array indexes [22,23].

<pre>for(j=0; j<N; j++) for(i=0; i<M; i++) sum += A[j][i];</pre>	<pre>for(t=0; t<N*M; t++) { j = t/M; i = t%M; sum += A[j][i]; }</pre>
--	--

Figure 3.6: Example of Loop Coalescing [22]

Loop Normalization

Loop Normalization alters the loop bounds in way such that the lower bound is zero and the variable increments by one. This simplifies the code by making the bound a known constant and by enabling the ability to perform more optimization techniques, such as Loop Fusion, that we were unable to do [22,23].

```
//A simple loop example
for(i=1;i<N+1;i++){
  b[i] = a[i-1] * b[i];
}

//A loop after loop normalization
for(i=0;i<N;i++){
  b[i+1] = a[i] * b[i+1];
}
```

Figure 3.7: Example of Loop Normalization

Loop Splitting

Loop Splitting partitions a loop or a nested loop in various smaller loops, each one traversing different iteration space. This technique can be useful in data dependencies, as with the right partitioning it can eliminate the loop-carried dependencies [22].

<pre>for(j=0; j<M; j++) for(i=0; i<N; i++) sum += A[j][i];</pre>	<pre>for(j=0; j<M; j++) for(i=j; i<N+j; i++) sum += A[j][i-j];</pre>
--	--

Figure 3.8: Example of Loop Splitting [22]

Loop Fusion

Loop Fusion merge a sequence of loops into one loop. In order to perform this kind of optimization, any data dependencies must be preserved. With respect to data dependencies, loop fusion legal only when it does not introduce anti-dependencies. It benefits in increasing the level of parallelism, data locality and in decreasing the loop control overhead.

```
//A simple example of two loops
for(i=0; i<N; i++){
  b[i] = a[i] * 2;
}
for(i=0; i<N; i++){
  c[i] = a[i] + 2;
}

//Applied loop fusion
for(i=0; i<N; i++){
  b[i] = a[i] * 2;
  c[i] = a[i] + 2;
}
```

Figure 3.9: Example of Loop Fusion

Another important thing that changing the source code provides, and the Xilinx tool cannot, is the elimination of dependencies, that makes it impossible to achieve II=1 when pipelining a loop without having eliminated any loop-carried dependencies.

Dependency analysis

A dependence is a relationship between two computations that places constraints on their execution order. There are two types of dependencies, the data dependencies, which means that there is a dependency between two states of the body in the same iteration, and loop-carried dependencies, which means that there is a dependency between two states of two different iterations.

Data dependencies are when two statements cannot be executed simultaneously due to conflicting uses of the same variable. There are four types of data dependencies, flow dependence (also known as true dependence or Read-After-Write), anti-dependence (also known as Write-After-Read), output dependence (also known as Write-After-Write) and input dependence (also known as Read-After-Read)[24].

In Figure 3.10, an example of both three types of data dependencies can be seen. For example, there is a flow dependency between the operations in statement s1 and s0, where the value of f is read in s1 and written in s0. Also, there is an anti-dependence between s0 and s1 where the value of g is written in s1 and read in s0. There is also depicted an input and output

dependence between s0 and s2, where the values of f and h are written and read in s0, respectively, and are written and read again in s2, respectively. Lastly, there is an anti-dependence between s1 and s2 statements, where the value of f is written in s2 and read in s1. Figure 3.10 (c), depicts the data dependencies of the example as a data dependence graph (DDG). Both anti-dependence and output dependence can be eliminated by renaming the variables, accordingly to the Figure 3.11, which results in the DDG of figure. Data-independent operations can be freely interchanged in terms of their execution order, while data-dependent operations must be executed in the same order. For this reason, we chose to rename the variables f and g, so that the compiler can change the relative order of any operations [22,24].

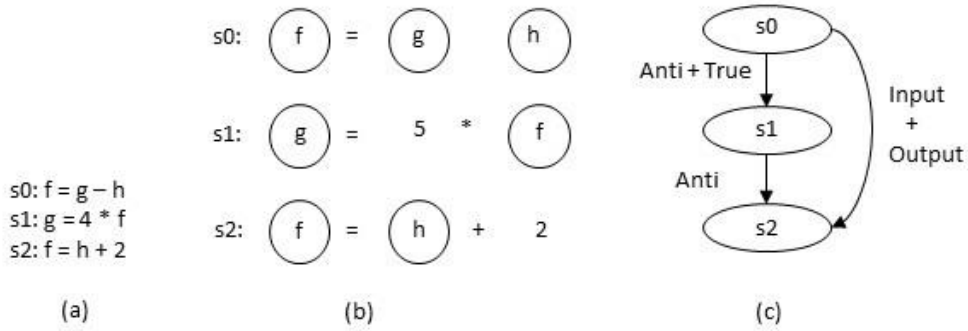


Figure 3.10: Data-dependencies analysis. (a) Sequence of statements, (b) types of data dependencies, (c) data dependence graph

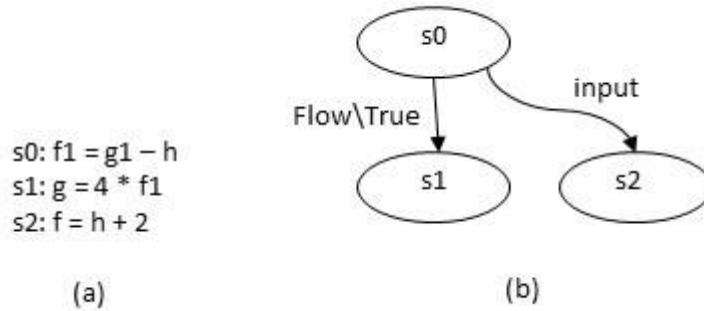


Figure 3.11: Elimination of data dependencies. (a) renamed sequence of statements, (b) data dependence graph after renaming

Loop-carried dependencies, as mentioned before, are the dependencies that one or more statements of one iteration depend on one or more statements from a different iteration. If such a dependency exists, it prevents a loop to be parallelized.

In the realm of analyzing data dependencies of loop nests and arrays with affine index functions, researchers developed the terms of “iteration space” and “dependence vector”. The iteration space and its vector define the space of iterations of the loops as an integer Cartesian space with as many dimensions as there are nested loops in the code section under analysis. The dependence vector determines which iterations of the iteration space depend on which iteration of the space. The Figure 3.12 illustrates an example of a nested loop with a loop-carried dependency and its dependency analysis. Given the iteration space as a vector space $i=(i_1, ..., i_k)$, where i_1 and i_k represent the base vectors of this space and correspond to loop index control variables, the data analysis aims to establish whether two distinct iterations (i_1 and i_2)

exist where the array elements accessed by the operations in these iterations are non-disjoint. This analysis examines if, for every pair of array elements, there exists i_1 and $i_2 \in I$ such that the affine indexing functions $F(i_1)$ and $F(i_2)$ evaluate to the same values of these iterations. Once the data dependence is established, a further analysis is required to determine the type of data dependence. In our example there is a data dependence between distinct iterations of the loop denoted by the distance vector $D=(1,1)$. In other words, there is a dependency between iterations (i_1, j_1) and (i_2, j_2) , where $i_2=i_1+1$ and $j_2=j_1+1$.

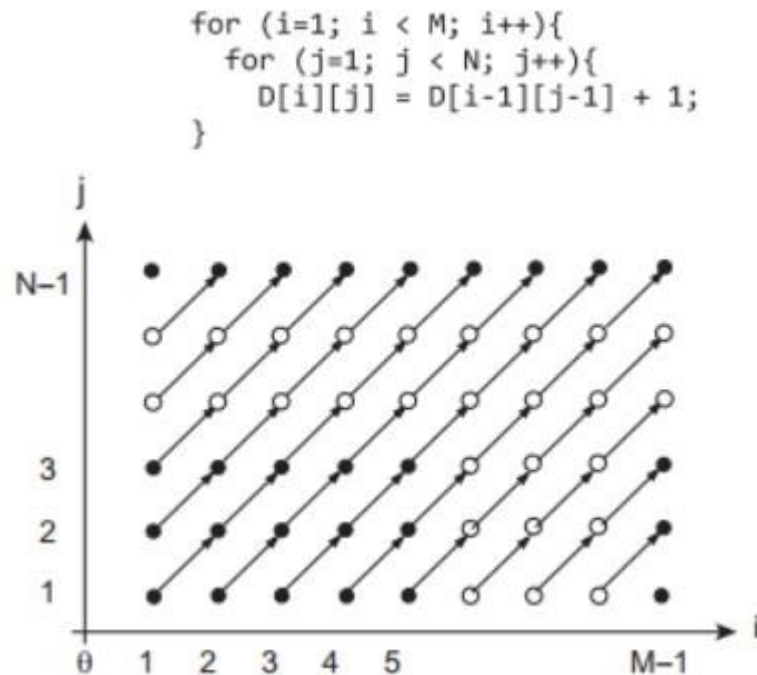


Figure 3.12: Example of data dependencies in nested loop [22]

3.2.5 Vivado HLS limitations

Although Vivado HLS supports a wide range of the C language, there are some constructs that either are not synthesizable or will eventually result in errors further in the design flow. In order to have a synthesizable design the following four statements must be true:

- The C function must contain the entire functionality of the design.
- None of the functionality can be performed by system calls to the operating system.
- The C constructs must be fixed or bounded size.
- The implementation of those constructs must be unambiguous.

System Calls

System calls cannot be synthesized as they are actions that interact with the operating system while a C program is running. These actions perform tasks within the operating system. So, functions as `getc()`, `time()`, `sleep()`, `printf()` and `fprintf()` must be deleted before synthesizing the code. Even if they are not deleted, Vivado HLS ignores most of them and depicts them as a warning, specifically the functions that do not interact with the algorithmic behavior.

Dynamic Memory Usage

Any system calls, like `malloc()`, `alloc()`, `realloc()` and `free()`, are using resources that exist in the memory of the operating system and are created and released during run time. So, these system calls must be removed and transform into equivalent bounded representations before synthesizing, because to be able to synthesize a hardware implementation the design must have specified sizes of arrays etc.

Pointer Limitations

Vivado HLS does not support general pointer casting but supports pointer casting between native C types. Also, it supports pointer arrays with one condition, that every pointer will point at a scalar or at an array out of scalars. Lastly, the function pointers are not supported.

Recursive Functions

Vivado HLS does not support recursive functions. In C++, templates can implement tail recursion.

Standard Template Libraries

Some of the C++ Standard Template Libraries (STLs) contain function recursion and use dynamic memory allocation. For this reason, STLs cannot be synthesized. But a local function can be created with identical functionality that does not exhibit these characteristics of recursion and dynamic memory allocation.

Chapter 4

First Implementation of CRYSTALS-Dilithium

4.1 Initial Solution

4.1.1 Top-level function and verification code

A top-level function of a code is defined as the function that goes through synthesis stage. Any other function is included in the top function, as they are at a lower level of hierarchy.

In our case, the top function of the implementation of Dilithium-CRYSTALS, will be the functions `crypto_sign()`, `crypto_sign_open()`, and `crypto_sign_keypair()`, which are the three top functions (as can be seen in Figure 4.1, Figure 4.2 and Figure 4.3 respectively) and include the entire Dilithium implementation. For simplicity reasons we will be synthesizing the top function `crypto_sign()` and we will implement the other two functions at the end of the chapter 5 with all the optimizations. Furthermore, due to the fact the top functions call too many functions, from lower level of hierarchy, we will depict the hierarchy of these functions in chapter 5, in every optimization.

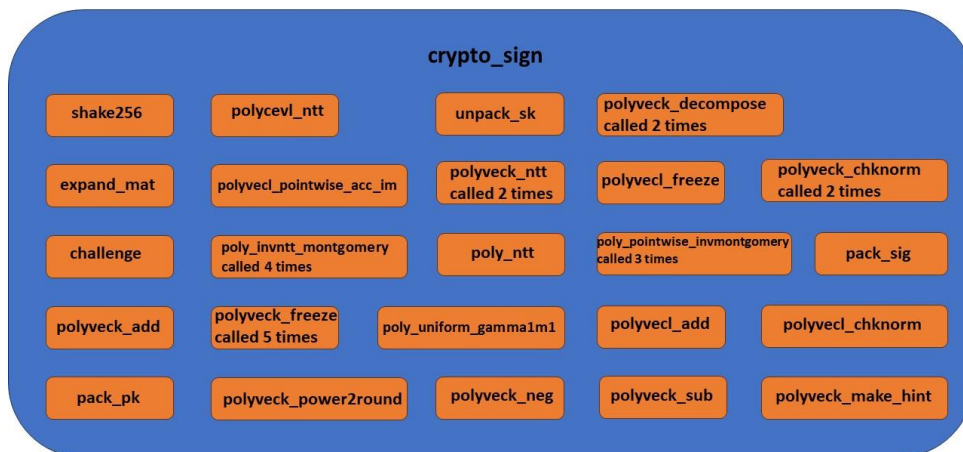
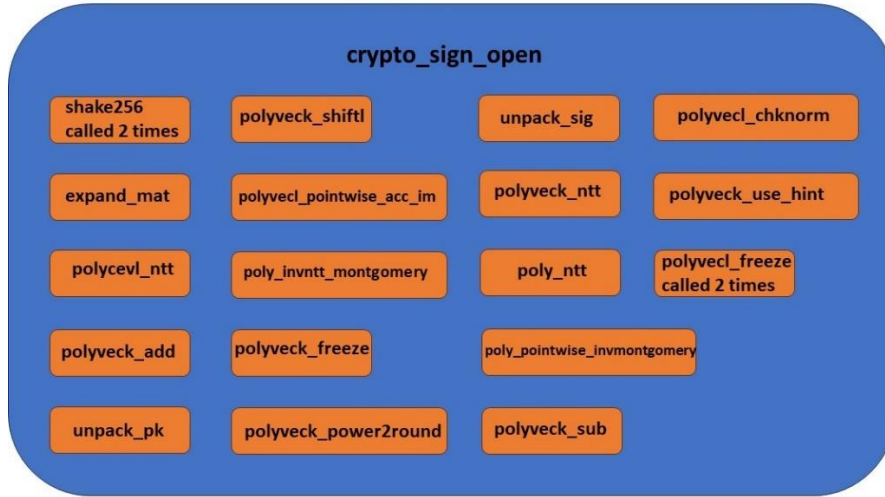
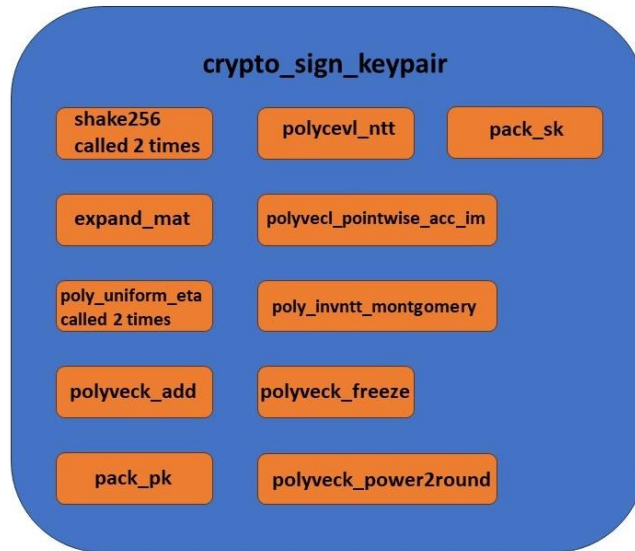


Figure 4.1: `crypto_sign()` hierarchy

Figure 4.2: `crypto_sign_open()` hierarchyFigure 4.3: `crypto_sign_keypair()` hierarchy

To verify the correct operation of the synthesized code, it is necessary to develop a verification file, known as testbench. This file contains the `main()` function, and its purpose is to correlate the hardware results with that of the software.

Our testbench is named ***newtest_sign.c*** and its inputs are the length of the message, a seed of 48 random bytes and the message. It creates the appropriate data structures reading the file with the inputs and then feeds the synthesizable functions with the appropriate data, resulting in the creation and storage of the public key, secret key, signed message, and the length of the signed message.

4.1.2 Project and Solutions

With the appropriate files, we create a new project in Vivado HLS by following a certain step process. We insert to source code with the filename ***sign.c*** and the testbench with the filename ***newtest.c***. Afterwards, the function `crypto_sign()` is defined as top functions. Finally, we

selected the FPGA of the Artix-7 family, and defined the clock period as 10ns, as shown in Figure 4.4.

Synthesis Settings

Clock

Period: Uncertainty:

Part Selection

Part:

Figure 4.4: Synthesis setting

4.1.3 C simulation

The inputs of the testbench are shown in Figure 4.5.

```
seed = 061550234D158C5EC95595FE04EF7A25767F2E24CC2BC479D09D86DC9ABCFDE7056A8C266F9EF97ED08541DBD2E1FFA1
mlen = 33
msg = D81C4D8D734FCBFBEADE3D3F8A039FAA2A2C9957E835AD55B22E75BF57BB556AC8
```

Figure 4.5: Inputs of testbench

By running C simulation, we get the results from running the testbench file and we are able to see the resulted keys and signed message. Lastly, the results are compared to that of the stored data. Since the results are the same, we move on to the next stages.

4.1.4 High Level Synthesis

In the synthesis stage, a VHDL and a Verilog code are produced, and a report for the implementation showing the delay and the resource consumption for each part of the code. Additionally, important information is available, such as the type of memory used by certain arrays, the way that some parts of the code are implemented etc.

The synthesis stage is the stage that we will use repeatedly, applying the optimizations to the selected functions, until we get the desired outcome.

The Figure 4.6 and Figure 4.7 depict the results of the initial implementation of CRYSTALS-DILITHIUM, without the appliance of any optimization.

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00 ns	8.738 ns	1.25 ns

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
238438	694549	2.384 ms	6.945 ms	238438	694549	none

Detail

Instance

Loop

Figure 4.6: Timing estimations of implementation of CRYSTALS-DILITHIUM without optimization

Utilization Estimates					
Summary					
Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	1543	-
FIFO	-	-	-	-	-
Instance	34	112	27253	123788	0
Memory	71	-	18	26	0
Multiplexer	-	-	-	5679	-
Register	-	-	941	-	-
Total	105	112	28212	131036	0
Available	730	740	269200	134600	0
Utilization (%)	14	15	10	97	0

Figure 4.7: Utilization estimations of implementation of CRYSTALS-DILITHIUM without optimization

4.1.5 RTL Co-Simulation

After the code has been successfully compiled and optimized, a simulation at RTL level takes place. Although the code is simulated using testbench, in this stage an evaluation is made of the material produced by the previous step of the synthesis, either of the VHDL or the Verilog code.

With the appliance of this step, a report is produced that informs about the failure or the success of the simulation, while also showing its clock cycles that our implementation, with the exact inputs, took.

In addition, the option to observe the signals of the implementation is given through Waveform. So that the behavior of the material that has been produced through synthesis can be understood.

Cosimulation Report for 'crypto_sign'							
Result							
RTL	Status	Latency			Interval		
		min	avg	max	min	avg	max
VHDL	Pass	352674	352674	352674	NA	NA	NA
Verilog	NA	NA	NA	NA	NA	NA	NA

Figure 4.8: RTL Co-Simulation

The Figure 4.8 shows that the RTL Co-Simulation informs us that the implementation of `crypto_sign()` is a success.

Chapter 5

Optimized versions of CRYSTALS-Dilithium

In sections 3.2.3 and 3.2.4, an extensive analysis was made of the optimization methods that we will apply to our post-quantum cryptography algorithm, Dilithium. These methods will aim at the reduction of the delay and the reduction of the resource utilization, and to acquire a good balance between these two.

Initially, we will identify the functions that are responsible for the delay and for the consumption of the resources. Then we will apply the optimizations that the Vivado HLS tool provides and will modify the source code of the algorithm for a further optimization.

We gave special importance, as they play a decisive role to optimize the algorithm, to the following factors:

1. **Initiation Interval (II):** The number of clock cycles needed so that the circuit starts processing a new input. It is one of the most critical factors to achieve the optimal delay.
2. **Latency:** The number of clock cycles needed so that the circuit computes all the values of the output. In other words, it is the time needed from the appliance of the first input till the computation of the last output.
3. **Resource Utilization:** The percentage number of the resources that the circuit will consume.

All the optimization results that will be analyzed below, will be applied to the functions of the `crypto_sign()` function as the top function. At the next chapter, all of them will be applied together to the other two functions (`crypto_sign_keypair` and `crypto_sign_open`) to get the final results of the algorithm.

5.1 Profiling and function selection to optimize

For the selection of the functions that will be optimized we followed the following steps. At first, we synthesized the top functions for several different inputs, to ensure that in every case there will be one or two functions that will differ from the others in terms of delay and resource consumption. The Figure 4.6 and Figure 4.7 depict the results of one of the inputs that we tested.

Optimized versions of CRYSTALS-Dilithium

The Figure 5.1 implies that the functions with the most impact of the total latency of the top function `crypto_sign()` is the `polyveck_ntt()`, `polyvecl_ntt()` and `invntt_frominvmont()`. While, the Figure 5.2 implies that the functions with the most resource utilization are `expand_mat()`, `keccak_absorb()`, `keccak_squeezeblocks()` and `challenge()`. The culprit function of this resource utilization is `KeccakF1600_StatePermute()`, as after pressing the Analysis tab and then the resource profile, it was quite noticeable that this function consumes the most resources.

Instance

Instance	Module	Latency (cycles)		Latency (absolute)		Interval (cycles)		Type
		min	max	min	max	min	max	
grp_polyveck_ntt_fu_1257	polyveck_ntt	33803	33803	0.338 ms	0.338 ms	33803	33803	none
grp_polyvecl_ntt_fu_1282	polyvecl_ntt	18437	18437	0.184 ms	0.184 ms	18437	18437	none
grp_challenge_fu_1299	challenge	3424	47284	34.240 us	0.473 ms	3424	47284	none
grp_keccak_absorb_fu_1308	keccak_absorb	491	3017	4.910 us	30.170 us	491	3017	none
grp_expand_mat_fu_1318	expand_mat	17232	17232	0.172 ms	0.172 ms	17232	17232	none
grp_keccak_squeezeblocks_1_fu_1326	keccak_squeezeblocks_1	241	1201	2.410 us	12.010 us	241	1201	none
grp_keccak_squeezeblocks_2_fu_1337	keccak_squeezeblocks_2	241	241	2.410 us	2.410 us	241	241	none
grp_invntt_frominvmont_1_fu_1345	invntt_frominvmont_1	11794	11794	0.118 ms	0.118 ms	11794	11794	none
grp_invntt_frominvmont_fu_1359	invntt_frominvmont	11794	11794	0.118 ms	0.118 ms	11794	11794	none
grp_pack_sig_fu_1371	pack_sig	4205	4205	42.050 us	42.050 us	4205	4205	none
grp_ntt_1_1_fu_1383	ntt_1_1	11265	11265	0.113 ms	0.113 ms	11265	11265	none
grp_ntt_2_fu_1395	ntt_2	11265	11265	0.113 ms	0.113 ms	11265	11265	none
grp_unpack_sk_fu_1405	unpack_sk	2491	2491	24.910 us	24.910 us	2491	2491	none
grp_polyvecl_pointwise_a_fu_1439	polyvecl_pointwise_a	4614	4614	46.140 us	46.140 us	4614	4614	none
grp_polyveck_make_hint_fu_1451	polyveck_make_hint	4615	4615	46.150 us	46.150 us	4615	4615	none
grp_rej_gamma1m1_fu_1461	rej_gamma1m1	641	646	6.410 us	6.460 us	641	646	none
grp_keccak_absorb_1_fu_1476	keccak_absorb_1	653	653	6.530 us	6.530 us	653	653	none
grp_poly_pointwise_invmo_1_fu_1482	poly_pointwise_invmo_1	1537	1537	15.370 us	15.370 us	1537	1537	none
grp_poly_pointwise_invmo_fu_1504	poly_pointwise_invmo	1537	1537	15.370 us	15.370 us	1537	1537	none
grp_polyveck_decompose_fu_1522	polyveck_decompose	3847	3847	38.470 us	38.470 us	3847	3847	none
grp_poly_neg_fu_1532	poly_neg	767	767	7.670 us	7.670 us	767	767	none
grp_polyveck_freeze_1_fu_1542	polyveck_freeze_1	2314	2314	23.140 us	23.140 us	2314	2314	none
grp_poly_sub_fu_1550	poly_sub	769	769	7.690 us	7.690 us	769	769	none
grp_poly_freeze_fu_1564	poly_freeze	769	769	7.690 us	7.690 us	769	769	none
grp_polyveck_freeze_fu_1574	polyveck_freeze	2311	2311	23.110 us	23.110 us	2311	2311	none
grp_poly_add_fu_1579	poly_add	769	769	7.690 us	7.690 us	769	769	none

Figure 5.1: Timing estimations of the instances of `crypto_sign()`

Instance

Instance	Module	BRAM_18K	DSP48E	FF	LUT	URAM
grp_challenge_fu_1299	challenge	11	0	7011	36629	0
crypto_sign_mux_4dEe_U227	crypto_sign_mux_4dEe	0	0	0	17	0
crypto_sign_mux_4dEe_U228	crypto_sign_mux_4dEe	0	0	0	17	0
crypto_sign_mux_4dEe_U229	crypto_sign_mux_4dEe	0	0	0	17	0
crypto_sign_mux_4dEe_U230	crypto_sign_mux_4dEe	0	0	0	17	0
grp_expand_mat_fu_1318	expand_mat	8	0	3462	18451	0
grp_invntt_frominvmont_fu_1359	invntt_frominvmont	1	18	790	1405	0
grp_invntt_frominvmont_1_fu_1345	invntt_frominvmont_1	1	18	802	1405	0
grp_keccak_absorb_fu_1308	keccak_absorb	3	0	3596	18422	0
grp_keccak_absorb_1_fu_1476	keccak_absorb_1	1	0	136	665	0
grp_keccak_squeezeblocks_1_fu_1326	keccak_squeezeblocks_1	2	0	3214	17317	0
grp_keccak_squeezeblocks_2_fu_1337	keccak_squeezeblocks_2	2	0	3209	17316	0
grp_ntt_1_1_fu_1383	ntt_1_1	1	10	705	1085	0
grp_ntt_2_fu_1395	ntt_2	1	10	695	1053	0
grp_pack_sig_fu_1371	pack_sig	0	0	363	1544	0
grp_poly_add_fu_1579	poly_add	0	0	88	215	0
grp_poly_freeze_fu_1564	poly_freeze	0	0	87	308	0
grp_poly_neg_fu_1532	poly_neg	0	0	65	213	0
grp_poly_pointwise_invmo_fu_1504	poly_pointwise_invmo	0	10	304	263	0
grp_poly_pointwise_invmo_1_fu_1482	poly_pointwise_invmo_1	0	10	287	279	0
grp_poly_sub_fu_1550	poly_sub	0	0	90	239	0
grp_polyveck_decompose_fu_1522	polyveck_decompose	0	0	148	471	0
grp_polyveck_freeze_fu_1574	polyveck_freeze	0	0	71	285	0
grp_polyveck_freeze_1_fu_1542	polyveck_freeze_1	0	0	99	356	0
grp_polyveck_make_hint_fu_1451	polyveck_make_hint	0	0	322	995	0
grp_polyveck_ntt_fu_1257	polyveck_ntt	1	10	721	1133	0
grp_polyvecl_ntt_fu_1282	polyvecl_ntt	1	10	625	923	0
grp_polyvecl_pointwise_a_fu_1439	polyvecl_pointwise_a	1	16	706	1262	0
grp_rej_gamma1m1_fu_1461	rej_gamma1m1	0	0	299	591	0
grp_unpack_sk_fu_1405	unpack_sk	0	0	244	1204	0
Total		30	34	112	28139	124097

Figure 5.2: Resource estimations of the instances of `crypto_sign()`

To ensure and verify our assumptions that these are the functions that are responsible for the big latency and resource consumption, we referred to [25], where they imply the exact same statement, that the `ntt()` function is the most time consuming functions and the `KeccakF1600_StatePermute()` function is the most resource consuming.

5.2 Optimization of `polyveck_ntt` and `polyvecl_ntt` functions

The `polyveck_ntt()` and `polyvecl_ntt()` consists of a loop that iterates K and L times, respectively. The body of the loop is the `poly_ntt()` function, which assigns the input of the `ntt()` function. To optimize these two functions, we chose to perform optimizations only to the `ntt()` function.

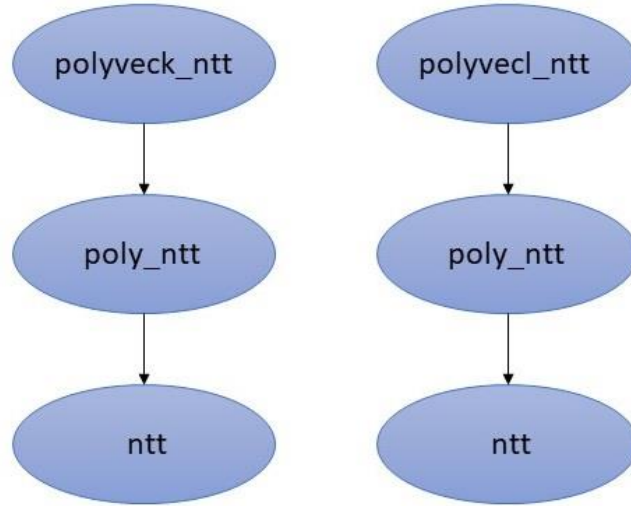


Figure 5.3: NTT hierarchy

The function `ntt()` is the Discrete Fourier Transform over a finite field or ring and works with polynomials. It basically performs fast convolutions, and with the help of the `inv_fromntt()` function, the multiplication of two polynomials is performed.

The input is a polynomial of 256 coefficients of 64 bit each, and a ROM which has stored the 256 values of the primitive roots of unity. The `ntt()` function consists of one nested loop, inner, middle and outer loop. The inner and the middle loop are responsible for the 128 iterations and the computations of the 256 elements of the polynomial, while the outer loop is responsible for the 8 iterations of the 8 different stages of the number theoretic transform. In other words, in each iteration of the outer loop, there are 128 iterations of the inner and middle loops.

In the middle and inner loop, at first, the modulo multiplication of the root of unity and the $i + \text{len}$ ($\text{len}=128$ at the beginning) element of the polynomial is computed. Then it computes the i and the $i + \text{len}$ element of the polynomial by increasing and decreasing the previous stored value of the polynomial by the output of the modulo multiplication. Then the variable `len` is divided by 2 and the process of computing the second stage begins. This happens until `len` reaches the value of 1. So, the `ntt()` function iterates 128×8 times to perform all the calculations needed.

5.2.1 Transformation to constant loop bounds

At first, we synthesized the design with `polyveck_ntt()` as the top function. As can be seen in Table 5.1, the latency of the function is unidentified, as the two inner loops have variable loop bounds. So, the very first step we had to do was to reconstruct the code in such a way that the architecture of the function would not change but the boundaries of the loops would transform to constant numbers, as in Figure 5.4.

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
?	?	?	?	?	?	none

Table 5.1: Latency results of `polyveck_ntt()` function with variable boundaries

Furthermore, we created the if conditions, as shown in Figure 5.4, to flatten the loop and to allow us to apply *PIPELINE* directive properly to the inner loop.

After synthesizing the design again, the tool can identify the latency of the function, as shown in Table 5.2, which from now on will allow us to perform optimization such as *UNROLL*.

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
27706	27706	0.277 ms	0.277 ms	27706	27706	none

Table 5.2: Latency results of `polyveck_ntt()` function with constant boundaries

```

void ntt(uint32_t *p) {
    unsigned int start, intermediate_point, j, k, i, h;
    unsigned int len = 128;
    uint32_t zeta, t;
    uint32_t temp2, temp1;
    intermediate_point = len;
    start = 0;
    k = 1;
    zeta = zetas[k++];
ntt_outer_loop:
    for (len = 128; len >= 1; len >>= 1) {
ntt_inner_middle_loop_combined:
        for (int j = 0; j < len; j++) {
            t = montgomery_reduce(p[start + len] * zeta);
            p[start + len] = p[start] + 2 * Q - t;
            p[start] = p[start] + t;
            start++;
            if (start == intermediate_point) {
                start += len;
                intermediate_point += (len << 1);
                zeta = zetas[k++];
            }
            if (j == len - 1) {
                intermediate_point = len >> 1;
                start = 0;
            }
        }
    }
}

```

Figure 5.4: Reconstructed `ntt()` function with constant boundaries

5.2.2 Optimization of the inner loop of the `ntt()` function

Applying the directive *PIPELINE* to the inner loop, as shown in Figure 5.5, we allow the loop to execute its operations concurrently, corresponding to different iterations.

```

v ● ntt
  v nt_outer_loop
    v nt_inner_middle_loop_combined
      # HLS PIPELINE

```

Figure 5.5: PIPELINE directive to the inner loop

As expected, the Xilinx tool flagged some warnings, showing as that the value of initiation interval is equal to 8 (II=8). This happens because, as we analyzed the function, we noticed that loop-carried and data dependencies (more specifically anti-dependencies) exist, which were analyzed in section 3.2.4. Furthermore, memory is unable to perform load and store operation due to the two reads and two writes in the same iteration.

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		Type
min	max	min	max	min	max	
27661	27661	0.277 ms	0.277 ms	27661	27661	none

Table 5.3: Latency of the `polyveck_ntt()` function with the appliance of PIPELINE directive to the inner loop of the function

In Table 5.3 can be seen that there was not even a slightly decrease of the latency, and that is because not even one of the statements of the loop body could be pipelined, as every statement depends from either another statement of the same iteration or from another statement of the same iteration. Also, due to the limited accesses of the memory, load and store operations could not be pipelined either.

Another appliance of optimization of the inner loop was the `UNROLL` directive, as shown in Figure 5.6, which leads to the unfolding of the inner loop. This directive offered even greater delay than before and increased the resource consumption of DSP48E, FF and LUT by a factor of 38,9, 67,6 and 79,4 respectively, as can be seen in Table 5.4 and in Table 5.5.

```

v ● ntt
  v nt_outer_loop
    v nt_inner_middle_loop_combined
      # HLS UNROLL

```

Figure 5.6: UNROLL directive to the inner loop

Opening the Analysis Perspective tab, we can see that the use of this directive does not offer any parallelism of the design because of one thing. Firstly, a memory can be accessed two times at most, and in every unrolled component of the 128, there is one read of the `i+len` coefficient, two reads of the `I` coefficient, one write of the `i+len` coefficient and one write of the `I` coefficient of the polynomial. So, this is a bottleneck to our code.

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
30730	30730	0.307 ms	0.307 ms	30730	30730	none

Table 5.4: Latency of the `polyveck_ntt()` function with the appliance of `UNROLL` directive to the inner loop of the function

	BRAM_18K	DSP48E	FF	LUT	URAM
No optimization	1	10	398	888	0
Unroll directive	1	389	26907	70540	0

Table 5.5: Resource utilization of the `polyveck_ntt()` function with no optimizations and with the `UNROLL` directive

This could be dissolved by applying the `ARRAY_PARTITION` directive, but we rejected it. The `ntt()` function appears one time through `polyveck_ntt()` function, two times through `polyveck_ntt()` function and one time through `poly_ntt()` function, so by partitioning the array it will increase even more the resource utilization.

Due to this rapidly increase of the resource utilization we chose not to perform any other optimization that could unfold the inner loop of the `ntt()` function, as it will lead the final design of the `crypto_sign()` function to exceed the number of resources that our FPGA platform has.

5.2.3 Elimination of the anti-dependencies

As said before, anti-dependencies appear in `ntt()` function and this happens because the loop body, at first it reads one coefficient and then writes a new value at the same address of memory. This is one of the reasons that the `PIPELINE` directive cannot be applied to, as there is a hazard that at some point, first it will write a new value to the memory address and then it will read it.

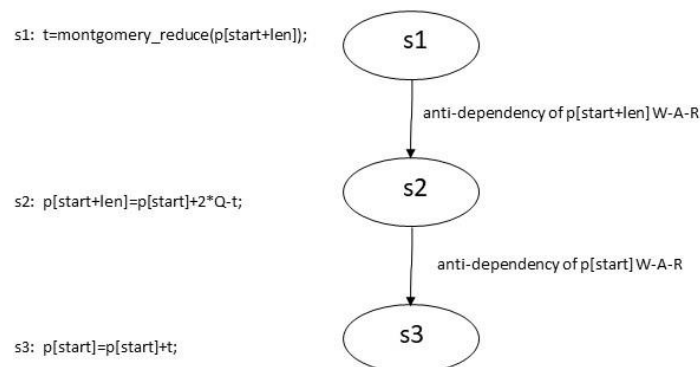


Figure 5.7: Anti-dependencies of the loop body

To eliminate these dependencies we inserted intermediate registers, temp1 and temp2. Their purpose is to read and store the value of the address, so that even if the write operation begins first, the value would not be lost and would still be available in temp1 or temp2. So, now the loop body of the ntt() function would be transformed as shown in Figure 5.8. A further purpose of these two registers was to reduce the read operations.

```
temp1 = p[start];
temp2 = p[start + len];
t=montgomery_reduce(temp2*zeta);
p[start + len] = temp1+ 2*Q - t;
p[start] = temp1+ t;
if (start == intermediate_point) {
    start += len;
    intermediate_point += (len << 1);
    zeta = zetas[k++];
}
if(j == 128 - 1) {
    intermediate_point = len>>1;
    start = 0;
}
```

Figure 5.8: Insertion of intermediate registers

Before inserting these intermediate registers, by opening the Analysis Perspective tab we can see that there are three reads per iteration, one for the $i+len$ coefficient and two for the i coefficient of the polynomial, as shown in Figure 5.9. After inserting these registers, not only were the dependencies eliminated, but the reads from the memory were also reduced, from three reads to two read per iteration, as shown in Figure 5.10, because now it needs to read the i and the $i+len$ coefficients once.

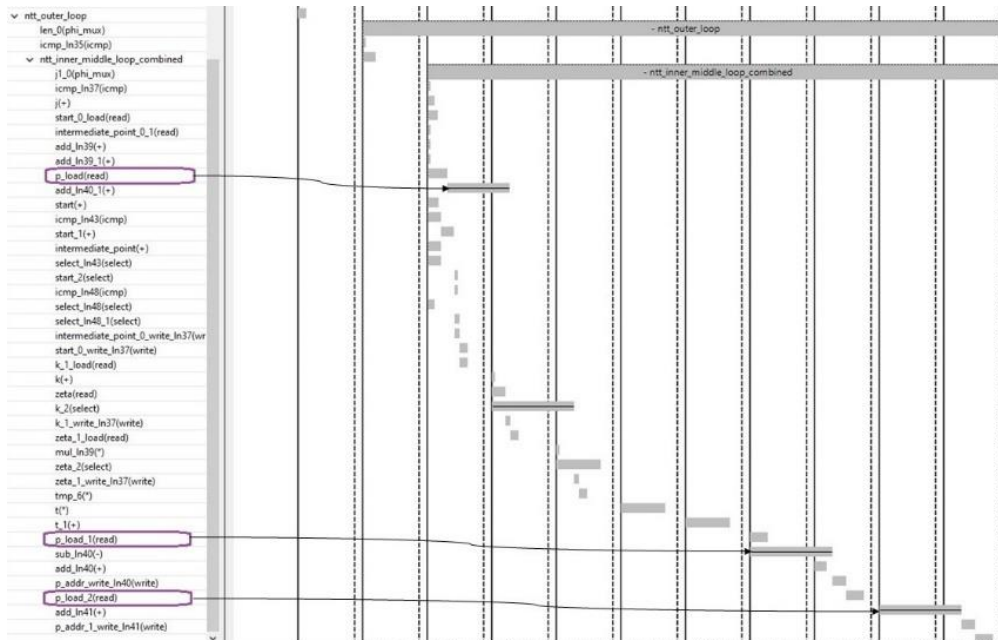


Figure 5.9: Analysis perspective before inserting registers

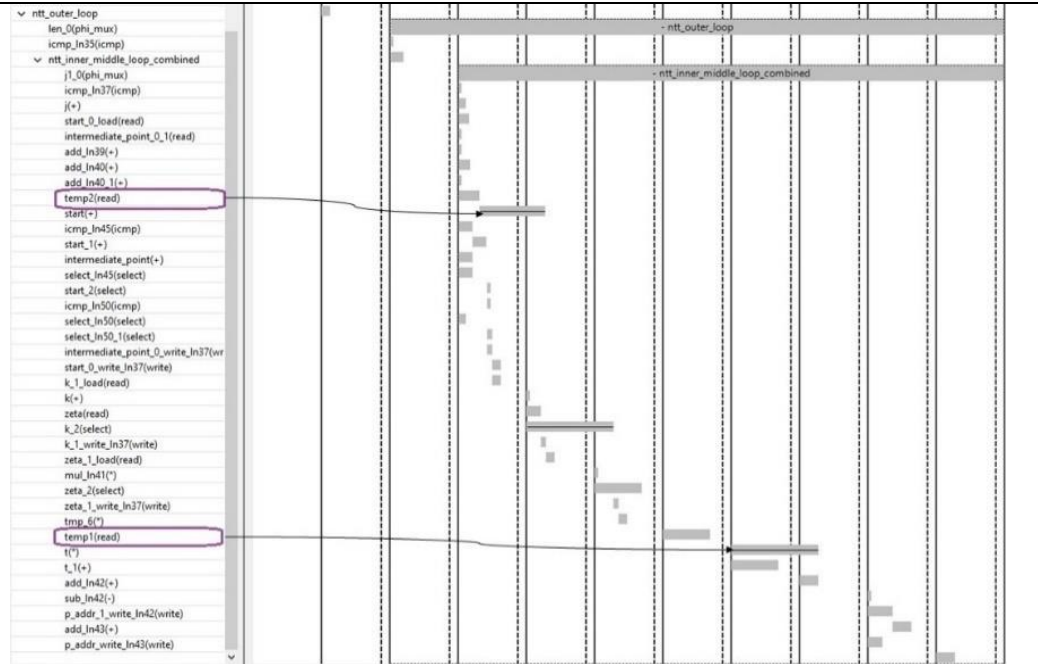


Figure 5.10: Analysis perspective after inserting registers

So, as it is expected, the latency of the `polyveck_ntt()` function was reduced, as shown in because in total the read operations were decreased by 128, which are the iterations of one stage of number theoretic transform, times 8, which are the total stages of the `ntt()` function.

Latency

Summary

Latency (cycles)		Latency (absolute)		Interval (cycles)		
min	max	min	max	min	max	Type
24634	24634	0.246 ms	0.246 ms	24634	24634	none

Table 5.6: Latency of the `polyveck_ntt()` function with the insertion of intermediate registers

5.2.4 Elimination of loop-carried dependencies

The biggest and most important problem of the number theoretic algorithm is the loop-carried dependencies, and due to this problem, the initiation interval cannot be equal to one. Each stage depends on the previous stage, so it must wait the computations to be completed from the one stage to start processing the next one. In other words, for the outer loop to go to the next iteration, it must wait the 128 iterations of the inner loop to be completed. The Figure 5.11 shows an example of the existence of loop-carried dependencies of a three-stage number theoretic transform with 2^3 coefficients.

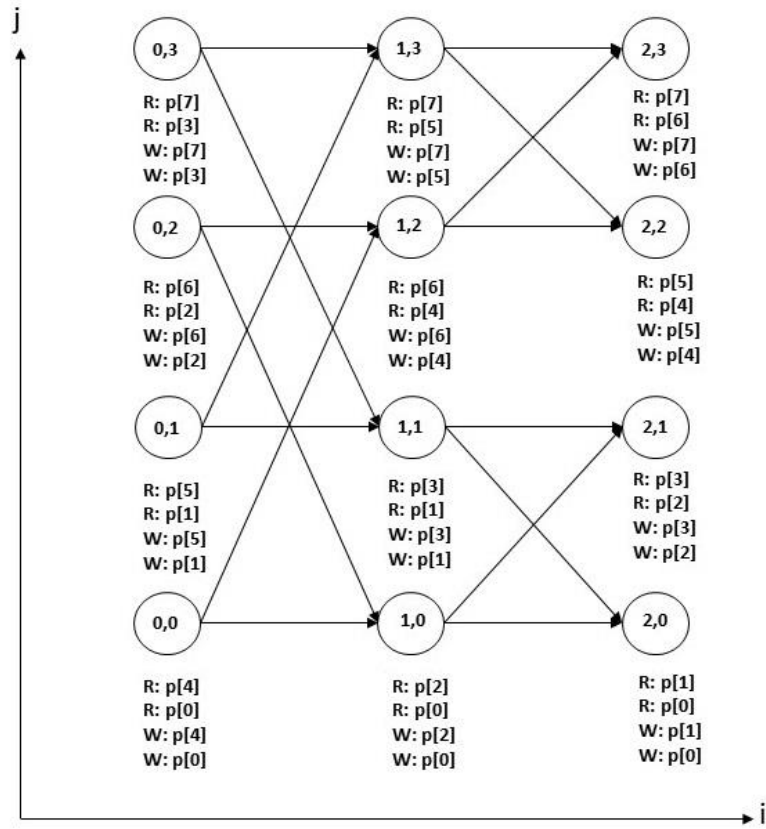


Figure 5.11: Loop-carried dependency analysis of a three-stage number theoretic transform

In the above example there are three different distance vectors. The first vector points to +1 in the direction of i and 0 in the j direction, which is the same for every stage. The second and the third vector points to +1 in the direction of i . For the j direction, it begins by pointing to $+(\text{number of coefficients of polynomial})/4$ and $-(\text{number of coefficients of polynomial})/4$, respectively. Every time it moves to the next stage, the divisor shifts to the left by one. To simplify the vectors, we can summarize and save only the direction the dependence is in. So, the first vector is $(i, j) = (+, 0)$, the second is $(i, j) = (+, +)$ and the third is $(i, j) = (+, -)$.

To eliminate the loop-carried dependencies, we applied the loop splitting technique, e.g., we reconstructed the code, as in Figure 5.13 and built separately loops for each stage. This does

```

void ntt(uint32_t *p) {
    unsigned int len, start, j, k, i;
    uint32_t zeta, t;
    //zeta = zetas[1];
    k = 1;
    zeta = zetas[k];

    // 1st Stage
ntt_label1011:
    for(j=0; j<128; j++) {
        t=montgomery_reduce((uint64_t)zeta*p[j+128]);
        p[j+128]=p[j]+(Q<<1)-t;
        p[j]=p[j]+t;
    }
    k++;
    zeta = zetas[k];

    // 2nd Stage
ntt_label11:
    for(i=0; i<256; i=i+128) {
ntt_label18:
        for(j=0; j<64; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+64]);
            p[j+i+64]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==64-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 3rd Stage
ntt_label12:
    for(i=0; i<256; i=i+64) {
ntt_label19:
        for(j=0; j<32; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+32]);
            p[j+i+32]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==32-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 4th Stage
ntt_label13:
    for(i=0; i<256; i=i+32) {
ntt_label20:
        for(j=0; j<16; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+16]);
            p[j+i+16]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==16-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 5th Stage
ntt_label14:
    for(i=0; i<256; i=i+16) {
ntt_label21:
        for(j=0; j<8; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+8]);
            p[j+i+8]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==8-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 6th Stage
ntt_label15:
    for(i=0; i<256; i=i+8) {
ntt_label22:
        for(j=0; j<4; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+4]);
            p[j+i+4]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==4-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 7th Stage
ntt_label16:
    for(i=0; i<256; i=i+4) {
ntt_label23:
        for(j=0; j<2; j++) {
            t=montgomery_reduce((uint64_t)zeta*p[j+i+2]);
            p[j+i+2]=p[j+i]+(Q<<1)-t;
            p[j+i]=p[j+i]+t;
            if(j==2-1) {
                k++;
                zeta = zetas[k];
            }
        }
    }

    // 8th Stage
ntt_label17:
    for(i=0; i<256; i=i+2) {
        zeta = zetas[k];
        k++;
        t=montgomery_reduce((uint64_t)zeta*p[i+1]);
        p[i+1]=p[i]+(Q<<1)-t;
        p[i]=p[i]+t;
    }
}

```

Figure 5.12: Reconstructed `ntt()` function after the appliance of loop splitting technique

n

Figure 5.13

re

in each stage and parallelize the computations of the loop body.

After applying *PIPELINE* directive in every stage, we noticed that the latency was decreased by a little, and that the tool still warns as that it is unable to enforce carried dependence. As a result, the initiation interval equals to 8. That is happening because the indices of the accessed array `p` are complicated, and the tool does not know that the same address will not be accessed in next iterations.

To deal with that, there is the *DEPENDENCE* directive, which gives us the ability to inform the tool if a dependence is true or false. So, we applied this directive by specifying that `p` is “responsible” for the dependence, that there is not loop-carried dependence, and that `p` is pointer array. Figure 5.13 depicts the way these three statements are specified to the tool.

```

v ntt
v ntt_label1011
  # HLS DEPENDENCE variable=p pointer inter false
  # HLS PIPELINE
v ntt_label11
  v ntt_label18
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label12
  v ntt_label19
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label13
  v ntt_label20
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label14
  v ntt_label21
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label15
  v ntt_label22
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label16
  v ntt_label23
    # HLS DEPENDENCE variable=p pointer inter false
    # HLS PIPELINE
v ntt_label17
  # HLS DEPENDENCE variable=p pointer inter false
  # HLS PIPELINE

```

Figure 5.13: *PIPELINE* and *DEPENDENCE* directive applied to every stage

In Table 5.7 the results of the loop splitting, loop splitting plus *PIPELINE* directive and loop splitting plus *PIPELINE* directive plus *DEPENDENCE* directive are depicted. It can be easily noticed that the reduction of the latency came with the price of the resource consumption increase.

	BRAM_18K	DSP48E	FF	LUT	URAM	Latency (Cycles)
No optimization	1	10	398	888	0	27706
Loop Splitting	1	76	2676	4225	0	26899
Loop splitting + PIPELINE	1	76	3647	4999	0	22738
Loop splitting + PIPELINE+ DEPENDENCE	1	76	4127	5231	0	9022

Table 5.7: Resource Utilization and Latency for a) no optimization, b) Loop splitting, c) loop splitting + PIPELINE, d) loop splitting + PIPELINE + DEPENDENCE

5.2.5 Appliance of Loop Normalization technique in every stage of ntt() function

Loop Normalization technique, as mentioned in section 3.2.4, alters the loop bounds in way such that the lower bound is zero and the variable increments by one. Sometimes, this has the risk of increasing the latency of the function because the computations of the indexes of the array is now more complicated. However, it makes the nested loops easier to handle, and offers the ability to perform further optimization techniques.

Figure 5.14 shows an example of the code transformations needed, of the second stage, to be performed to normalize every loop.

```

// 2nd Stage
ntt_label11:
for(i=0; i<256; i=i+128) {
ntt_label18:
for(j=0; j<64; j++) {
t=montgomery_reduce((uint64_t)zeta*p[j+i+64]);
p[j+i+64]=p[j+1]+(Q<<1)-t;
p[j+1]=p[j+i]+t;
if(j==64-1) {
k++;
zeta = zetas[k];
}
}
}
}

// 2nd Stage
ntt_label11:
for(i=0; i<2; i++) {
ntt_label18:
for(j=0; j<64; j++) {
t=montgomery_reduce((uint64_t)zeta*p[j+i*128+64]);
p[j+i*128+64]=p[j+i*128]+2*Q-t;
p[j+i*128]=p[j+i*128]+t;
if(j==64-1) {
k++;
zeta = zetas[k];
}
}
}
}

```

Figure 5.14: Example of appliance of the Loop Normalization technique on second stage

With this technique, as can be seen in Table 5.8, did not offer any optimization to ntt() function. To be more precise, it resulted in a slightly increase of resource utilization when we applied the *PIPELINE* and *DEPENDENCE* directive. But now, another technique can be applied, the Loop Coalescing, which will be discussed in the next section. All of the optimizations bellow were applied to the already split ntt() function.

	BRAM_18K	DSP48E	FF	LUT	URAM	Latency (Cycles)
--	----------	--------	----	-----	------	---------------------

<i>No optimization</i>	1	10	398	888	0	27706
<i>Loop Normalization</i>	1	76	2628	4120	0	26899
<i>Loop Normalization + PIPELINE</i>	1	76	3444	4583	0	22738
<i>Loop Normalization + PIPELINE+ DEPENDENCE</i>	1	76	5007	5347	0	9037

Table 5.8: Resource Utilization and Latency for a) no optimization, b) Loop normalization, c) loop normalization + PIPELINE, d) loop normalization + PIPELINE + DEPENDENCE

5.2.6 Appliance of Loop Coalescing technique in every stage of ntt() function

As mentioned before, we performed Loop Normalization to be able to perform Loop Coalescing too. With this technique the nested loops transform into one single loop. The pros of the Loop Coalescing are that it reduces the loop control overhead and improves instruction scheduling, and the cons are that it adds some instruction complexity, due to the introduction of modulo and division operations.

Figure 5.15 shows an example of the code transformations needed, of the second stage, to be performed to coalesce every loop.

```

// 2nd Stage
ntt_label11:for(i=0; i<256; i=i+128) {
    ntt_label18:for(j=0; j<64; j++) {
        t=montgomery_reduce((uint64_t)zeta*p[j+i+64]);
        p[j+i+64]=p[j+i]+(Q<<1)-t;;
        p[j+i]=p[j+i]+t;
        if(j==64-1) {
            k++;
            zeta = zetas[k];
        }
    }
}

// 2nd Stage
ntt_label11:for(h=0; h<128; h++) {
    i=h/64;
    j=h%64;
    t=montgomery_reduce((uint64_t)zeta*p[j+i*128+64]);
    p[j+i*128+64]=p[j+i*128]+2*Q-t;
    p[j+i*128]=p[j+i*128]+t;
    if(j==64-1) {
        k++;
        zeta = zetas[k];
    }
}

```

Figure 5.15: Example of appliance of the Loop Coalescing technique on second stage

Once again, we applied at first the *PIPELINE* directive and then the *DEPENDENCE* directive. Thanks to the reduction of the loop control overhead, it is obvious that the latency would decrease too. Table presents the results of the appliance of this technique.

	<i>BRAM_18K</i>	<i>DSP48E</i>	<i>FF</i>	<i>LUT</i>	<i>URAM</i>	<i>Latency (Cycles)</i>
<i>No optimization</i>	1	10	398	888	0	27706
<i>Loop Normalization + Loop Coalescing</i>	1	76	3134	3691	0	24223
<i>Loop Normalization + Loop Coalescing + PIPELINE</i>	1	76	3341	3894	0	20818
<i>Loop Normalization + Loop Coalescing +</i>	1	76	5272	4658	0	7120

PIPELINE+
DEPENDENCE

Table 5.9: Resource Utilization and Latency for a) no optimization, b) Loop normalization +Loop Coalescing, c) loop normalization +Loop Coalescing + PIPELINE, d) loop normalization +Loop Coalescing + PIPELINE + DEPENDENCE

5.2.7 Insertion of supplementary RAM

With all the above optimizations we have achieved an initiation interval equal to two to every stage. To further the optimization and achieve II=1 we inserted supplementary RAMs between every stage (the output array of one stage is the input array of the next stage). With this optimization the HLS tool now can schedule the load and the store operations in a way to achieve the initiation interval equal to 1.

	BRAM_18K	DSP48E	FF	LUT	URAM	Latency (Cycles)
No optimization	1	10	398	888	0	27706
Loop Normalization + Loop Coalescing + Supplementary RAM	9	68	3038	3567	0	19999
Loop Normalization + Loop Coalescing + Supplementary RAM + PIPELINE	27	68	4607	4363	0	3616

Table 5.10: Resource Utilization and Latency for a) no optimization, b) Loop normalization + Loop Coalescing + Supplementary RAM and c) loop normalization +Loop Coalescing + Supplementary RAM + PIPELINE

As can be understood, these intermediate RAMs, provide efficient parallelism and pipelining, by storing the intermediate values in separate arrays. It also provides a better scheduling for the store and load operations as mentioned before.

5.2.8 Comparison of optimizations and optimal selection

Firstly, we will name every solution for obvious reasons in Table 5.11.

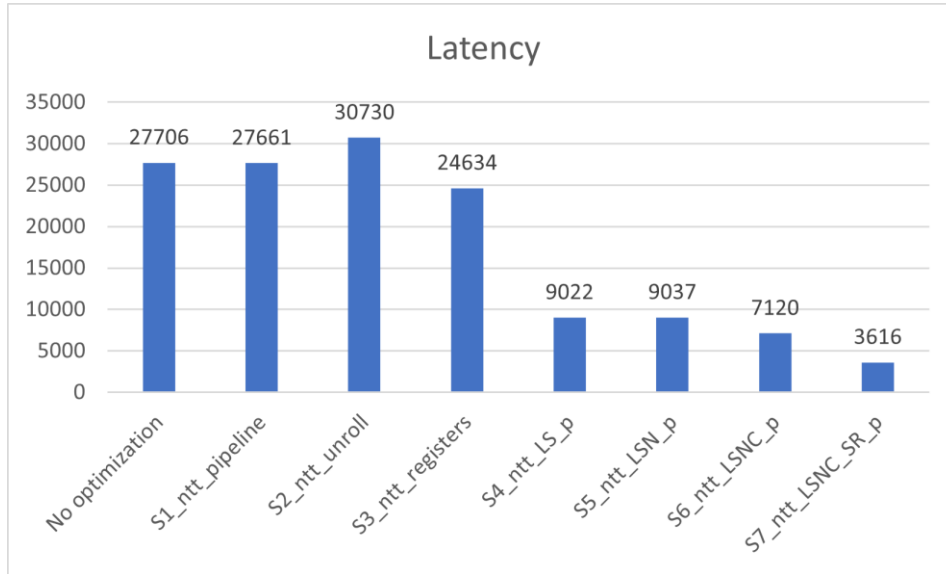
Solution names	Techniques
S1_ntt_pipeline	Pipeline the inner loop of ntt() function
S2_ntt_unroll	Unroll the inner loop of ntt() function
S3_ntt_registers	Insertion of intermediate registers
S4_ntt_LS_p	Splitting the ntt() function into 8 stages and pipeline every inner loop of the stages

S5_ntt_LSN_p	Normalization of the loops of the 8 stages and pipeline every inner loop of the stages
S6_ntt_LSN_p	Coalescing of the loops of the 8 stages and pipeline every inner loop of the stages
S7_ntt_LSN_SR_p	Insertion of supplementary RAM

Table 5.11: Naming solution of *ntt()* function

It is quite obvious that S4_ntt_LS, S5_ntt_LSN and S6_ntt_LSN_p solutions should be combined together, because to normalize the bounds the *ntt()* should be split, and to coalesce the inner and outer loop of every stage, the bounds should be normalized. Also, the

In Figure 5.16 and the comparison of the results in terms of latency and resource utilization are depicted. From the figures can be understood that the S7_ntt_LSN_SR_p solution made the *polyveck_ntt()* function 7.66 times faster at the price of the resources. But due to the facts that the number theoretic transform is the most time-consuming function and that is being used many times in each of the three top functions, we decided that this solution the best among the others.

Figure 5.16: Combination of optimization to *polyveck_ntt()* function

5.2.9 Reduction of resource utilization

After the selection of the optimal optimization of the *ntt()* function we tried to find a way to reduce the resource utilization because the *ntt()* function is called many times by the top functions. To achieve it we applied the *ALLOCATION* directives inside the *ntt()* function.

The first optimization that we implemented was to inline and allocate the *montgomery_reduce()* function one time inside the *ntt()* function. We implemented it that way because the implementation of the *ntt()* uses 8 *montgomery_reduce* modules, one for each stage. By allocating it once, the *ntt()* functions shares the *montgomery_reduce* module in both stages.

Another optimization that we implemented was to allocate the *mul* operation to 4, as each stage uses this operator four times. So now we have achieved to restrict the *mul* operators to

4 for the whole `ntt()` function. Of course, these allocation techniques slightly increased the latency, but reduced a significant amount of the resources.

Lastly, we applied `INLINE` directive to `ntt()` to dissolve the function logic into the calling function, `poly_ntt()`, then we applied `INLINE` directive to `poly_ntt()` to dissolve the function logic into the `polyveck_ntt()` and `polyvecl_ntt()` and then we applied `ALLOCATION limit=1` to the top function `crypto_sign()`.

Solution name	Technique name
S8_alloc_LSNC_SR_p	S7_ntt_LSNC_SR_p solution with the <code>ALLOCATIONS</code> and <code>INLINE</code> directives

Table 5.12: Naming last solution of `ntt()` function

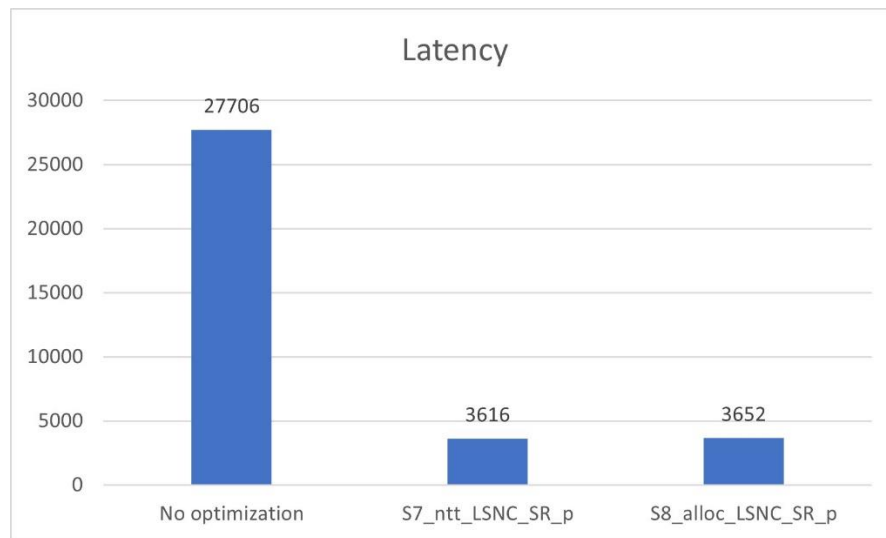


Figure 5.17: Latency after the appliance of the `S7_ntt_LSNC_SR_p` and `S8_alloc_LSNC_SR_p` to the `polyveck_ntt()` function

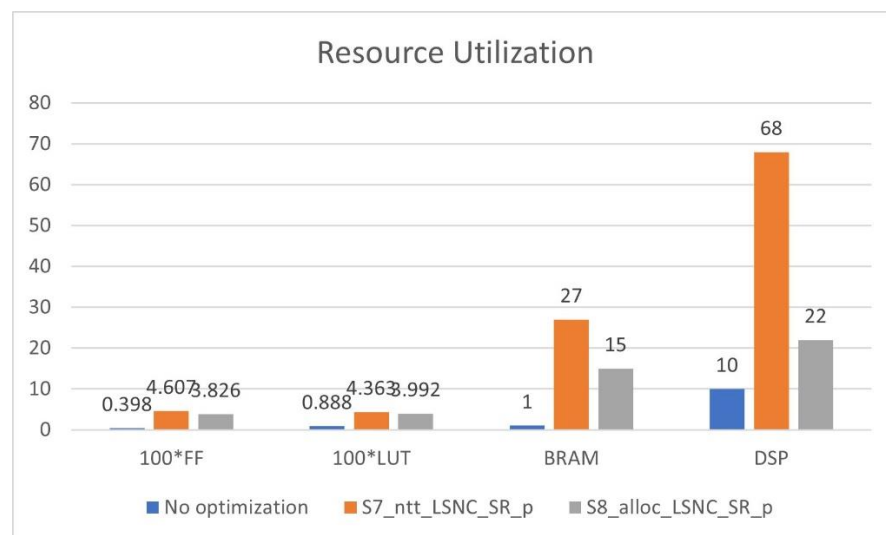


Figure 5. 18: Resource Utilization after the appliance of the `S7_ntt_LSNC_SR_p` and `S8_alloc_LSNC_SR_p` to the `polyveck_ntt()` function

5.3 Optimization of polyveck_invntt_montgomery() function

The `polyveck_invntt_montgomery()` function is the inverse Number Theoretic Transform, which is responsible to obtain the original order by reversing the modular operations that the NTT performed. To reverse these modular operations, inverse NTT uses the exact opposite algorithm of `ntt()` function.

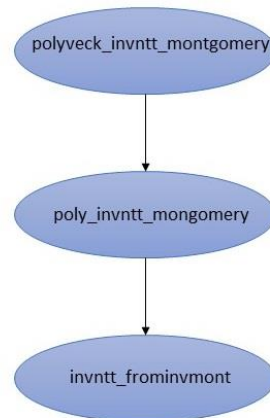


Figure 5.19: Inverse NTT hierarchy

The input is a polynomial of 256 coefficients of 32 bit each, and a ROM which has stored the 256 values of the primitive roots of unity. The `invntt_frominvmont()` function consists of one nested loop, inner, middle and outer loop. The inner and the middle loop are responsible for the 128 iterations and the computations of the 256 elements of the polynomial, while the outer loop is responsible for the 8 iterations of the 8 different stages of the number theoretic transform. In other words, in each iteration of the outer loop, there are 128 iterations of the inner and middle loops.

In the middle and inner loop, at first, the modulo multiplication of the root of unity and the $i+len$ ($len=1$ at the beginning) element of the polynomial is computed. Then it computes the i and the $i+len$ element of the polynomial by increasing and decreasing the previous stored value of the polynomial by the output of the modulo multiplication. Then the variable len is multiplied by 2 and the process of computing the second stage begins. This happens until len reaches the value of 128. So, the `invntt_frominvmont()` function iterates $128*8$ times to perform all the calculations needed. Lastly, after the 8 stages are completed, the 256 coefficients of the polynomial are multiplied by a scaling factor f . Thanks to the fact that it is the opposite algorithm, we were able to perform the same optimizations as with the `polyveck_ntt()` function.

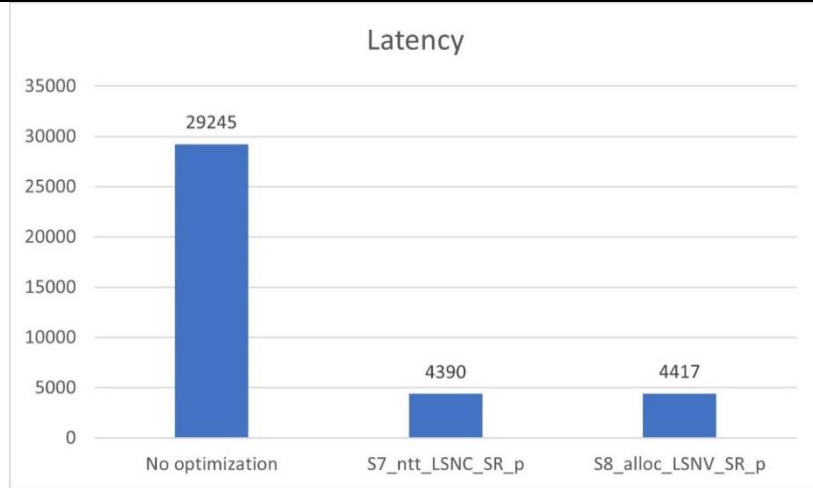


Figure 5.20: Latency after the appliance of the *S7_ntt_LSNC_SR_p* and *S8_alloc_LSNV_SR_p* to the *invntt_frominvmont()* function

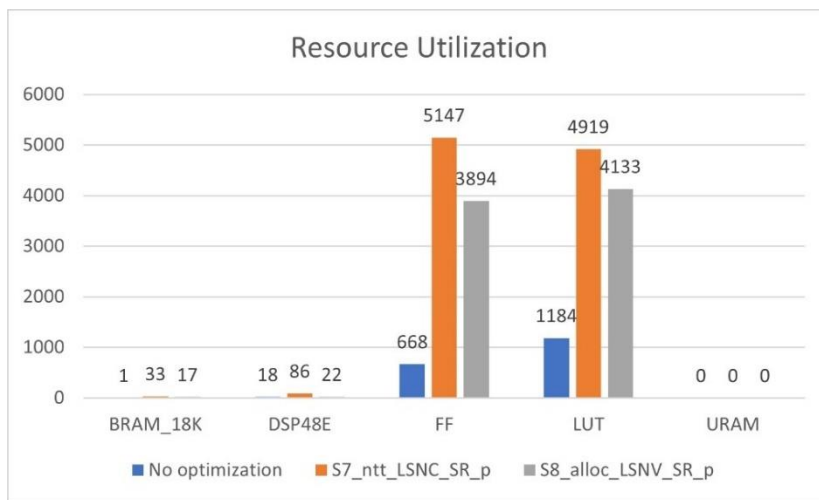


Figure 5.21: Resource Utilization after the appliance of the *S7_ntt_LSNC_SR_p* and *S8_alloc_LSNV_SR_p* to the *invntt_frominvmont()* function

5.4 Optimizations of shake256() and shake128() functions

As shown in Figure 5.2, the functions that consume the most resources of the FPGA platform are the *expand_mat()*, *challenge()*, *keccak_absorb()* and *keccak_squeezeblock()* functions.

The *fips202.c* file contains the functions *shake128()* and *shake256()*. The hierarchy of these two functions are depicted in and . These two functions consist of two significant phases, the absorption and the squeeze.

The absorption phase is the process of taking the input data and incorporating it into the internal state of the hash function. At first it pads the input message if it is necessary (if the input data are not multiple of the rate), and then the input data are divided into blocks, each of which is XORed with a corresponding portion of the internal state.

The squeeze phase is the process of extracting the output data from the internal state. Initially the internal state undergoes 24 rounds of permutations through the

KeccakF1600_StatePermute() function. Then, blocks of data are extracted from the state, equal to the rate of the hash function, and each block of data is appended to the output.

Solution name	Technique name
Solution_Inline_fips202	Inline the functions mentioned above and unroll the loops of the load64 and store64 functions
One_Permute_Per_iter	Reconstruction of the code to effectuate one permutation per iteration

Table 5. 13: Solution naming for fips202.c

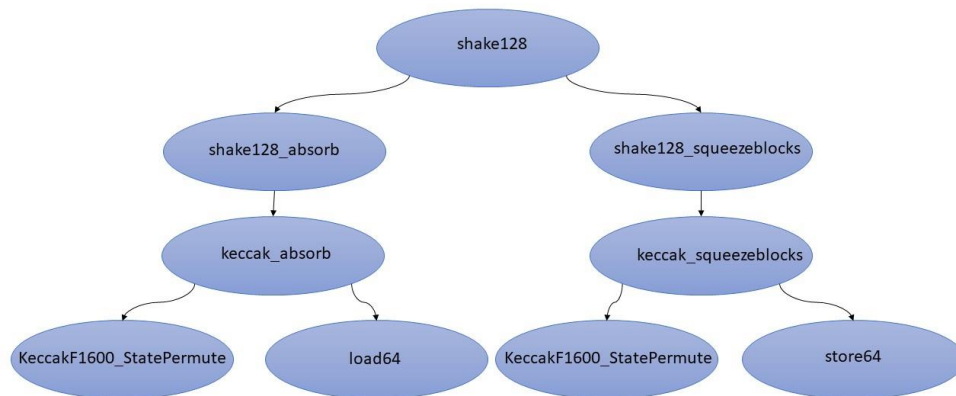


Figure 5.22: Hierarchy of shake128() function

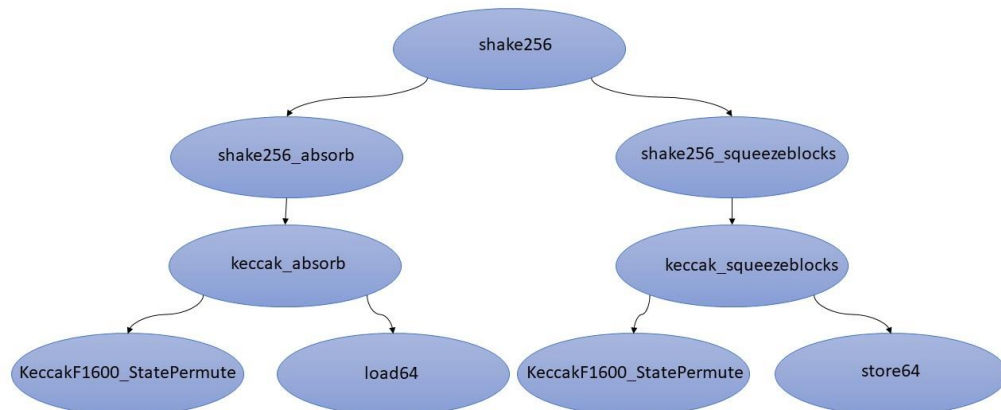


Figure 5.23: Hierarchy of shake256() function

5.4.1 Reduction of resources consumed by the functions of the fips202.c file

Our first approach of the reduction of the resources was that of our referenced implementation. At first, the *UNROLL* directive was applied to the functions `store64()` and `load64()`, which are responsible for the storing and loading of 64 bit unsigned integer from and into byte arrays in little-endian order, to be able to store and load concurrently, respectively.

Additionally, the *INLINE* directive was applied to the functions `keccak_absorb()` and `keccak_squeezeblocks()` to dissolve the function logic into the functions `shake128_absorb`, `shake256_absorb()`, `shake128_squeezeblocks` and `shake256_squeezeblocks()`. Then we applied the *INLINE* directive to the functions we just mentioned to dissolve the function logic into the functions `shake128()` and `shake256()`.

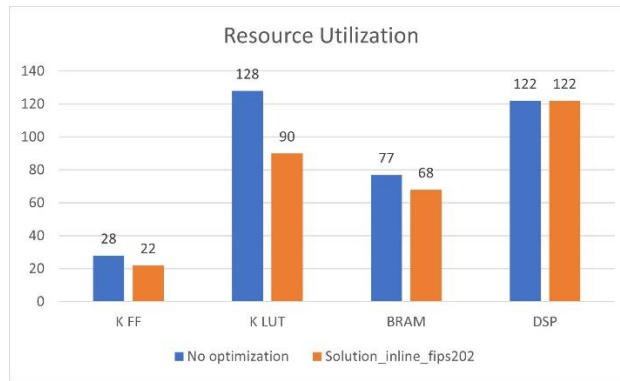


Figure 5.24: Resource Utilization of `crypto_sign()` function with the appliance of `Solution_inline_fips202`

5.4.2 Reduction of resources consumed by KeccakF1600_StatePermute function

By opening the Analysis Perspective tab, we noticed that the function that consumes the most resources of the `expand_mat()`, `challenge()`, `keccak_absorb()` and `keccak_squeezeblock()` functions is the `KeccakF1600_StatePermute` function.

This function is responsible for the creation of π , ι , ρ , θ and χ , that was analyzed in section 2.3.4. The input is a state memory of 25 elements of 64 bit each. After copying the state memory to 25 registers it starts iterating 12 times and creating the π , ι , ρ , θ and χ two times in each iteration, by performing many XOR and AND operations. And after the completion of the iterations, it copies the new values of the registers to the state memory.

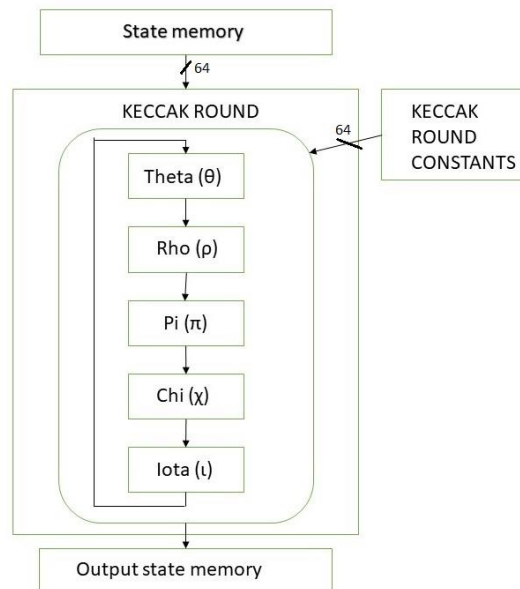


Figure 5.25: *KeccakF1600_StatePermute()* function

After analyzing the code, we noticed that the Keccak effectuates two steps of permutation at once. So, we adapted the code in a way to effectuate one permutation per iteration, which will decrease the resource consumption almost by 2 times.

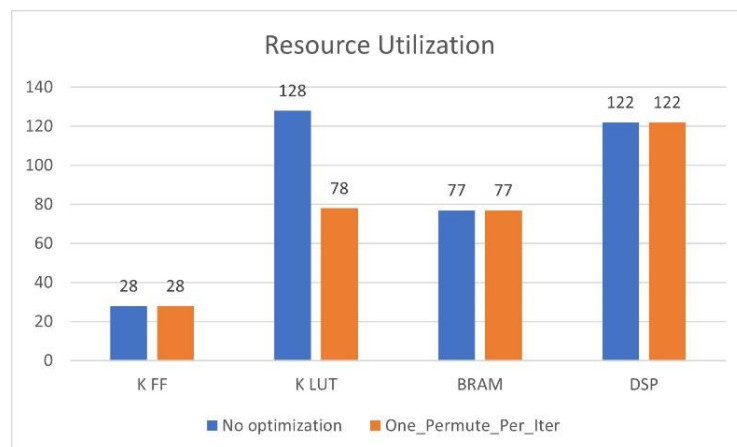


Figure 5.26: *Resource Utilization of the crypto_sign() with and without the optimization of the KeccakF1600_StatePermute() function*

From the Figure 5.26 it is noticeable that the effect of the One_Permute_Per_Iter solution is quite big, as it decreases the LUT consumption from **128 10³** to **78 10³** LUTs.

5.4.3 Combination of the Solution_Inline_fips202 and the One_Permute_Per_Iter solutions

After combining the solutions from the section 5.4.1 and 5.4.2 we achieved a quite significant reduction to LUT and FF usage, as shown in Figure 5.27, which was very important for our optimizations implemented on the next section. This implementation did not have any affection to the latency.

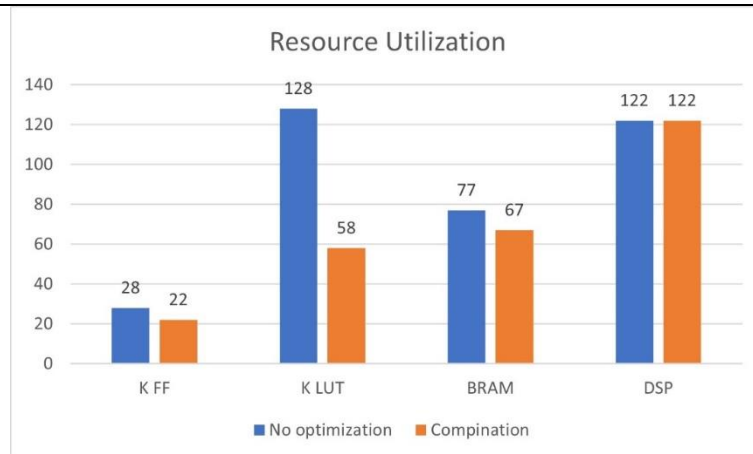


Figure 5.27: Resource Utilization of the `crypto_sign()` with and without the combination of the optimizations of the `KeccakF1600_StatePermute()` function

5.5 Optimizations of polyveck and polyvecl functions

For the functions `polyveck_add()`, `polyvecl_add()`, `polyveck_freeze()`, `polyvecl_freeze()`, `polyveck_sub()`, `polyveck_neg()`, `polyveck_decompose()`, `polyveck_power2round()`, `polyveck_make_hint()`, `polyveck_use_hint()` and `poly_pointwise_invmontgomery` we followed the same steps of optimizations as explained bellow.

In Figure 5.28 the hierarchies of the functions that will be optimized can be seen. In both of them we followed the bottom-up flow. We will apply the optimizations on `polyveck_add()` to show and explain the process of optimizations and then we will apply them to all the others functions and depict the results.

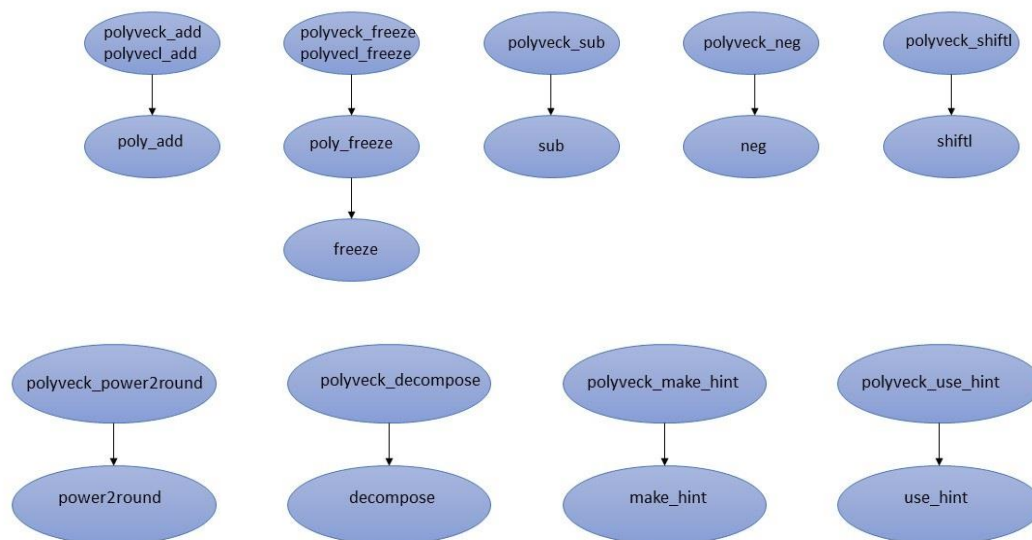


Figure 5.28: Hierarchies of the functions

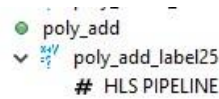
The Table 5.14 bellow names the directives that we applied to these functions.

Solution names	Technique names
----------------	-----------------

S_pv_pipeline_inner	pipeline inner loop
S_pv_unroll_inner_K	unroll inner loop by factor 256
S_pv_array_partition_K	array partition by factor 256
S_pv_pipeline_outer	pipeline outer loop
S_pv_unroll_outer	unroll outer loop
S_pv_arr_plus_pipe_plus_unroll_K	unroll and pipeline inner loop and partition the array by a factor 256

Table 5.14: Solution naming

At first we applied *PIPELINE* directive to the inner loop of the function `poly_add()`, as shown in Figure 5.29. This achieves an $II=1$, meaning that it will process a new addition per one clock cycle.



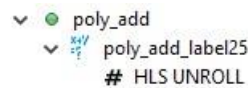
```

poly_add
└─ poly_add_label25
   └─ # HLS PIPELINE

```

Figure 5.29: Appliance of *PIPELINE* directive to the inner loop of `poly_add()` function

A second optimization that can be applied is the use of the *UNROLL* directive to the inner loop of the `poly_add()` function, as shown in Figure 5.30.



```

poly_add
└─ poly_add_label25
   └─ # HLS UNROLL

```

Figure 5.30: Appliance of *UNROLL* directive to the inner loop of `poly_add()` function

This brings the exact same results in terms of the latency with the appliance of the *PIPELINE* directive to the loop of the `polyveck_add()` function. This happens because by pipelining the outer loop, the inner loop is being fully unrolled. But the latter can be optimized further because the pipeline has achieved an initiation interval of 128. So, by applying the *ARRAY_PARTITION completely* directive, the latency can be radically reduced, as it achieves $II=1$. Figure 5.32 shows the results of these optimizations in terms of latency, and Figure 5.33 in terms of utilization.

```

#pragma HLS ARRAY_PARTITION variable=w->vec->coeffs complete dim=1
#pragma HLS ARRAY_PARTITION variable=u->vec->coeffs complete dim=1
#pragma HLS ARRAY_PARTITION variable=v->vec->coeffs complete dim=1

```

Figure 5.31: Appliance of *ARRAY PARTITION complete* directive to the inputs and output arrays of `polyveck_add()` function

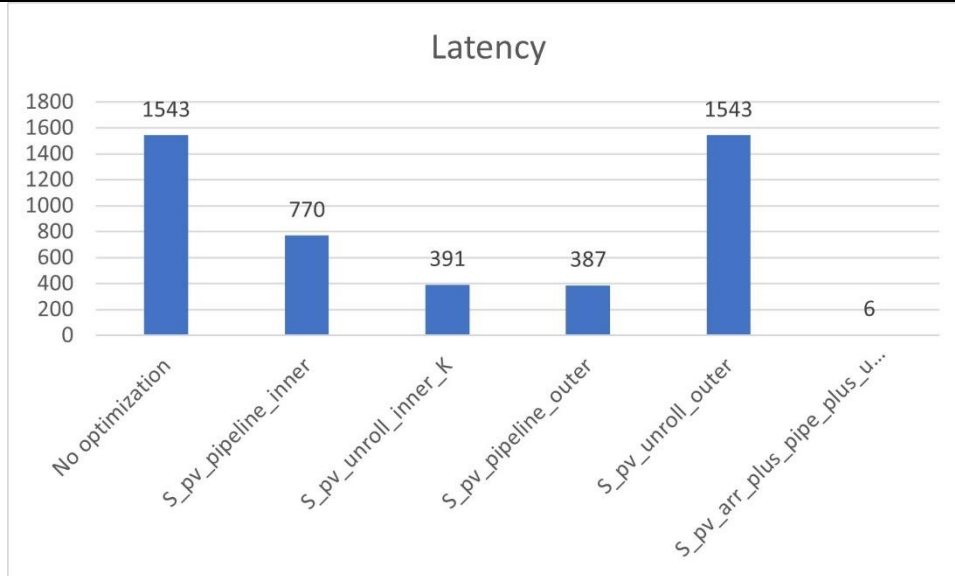


Figure 5.32: Latency graph of the `polyveck_add()` function with the implementation of various optimizations

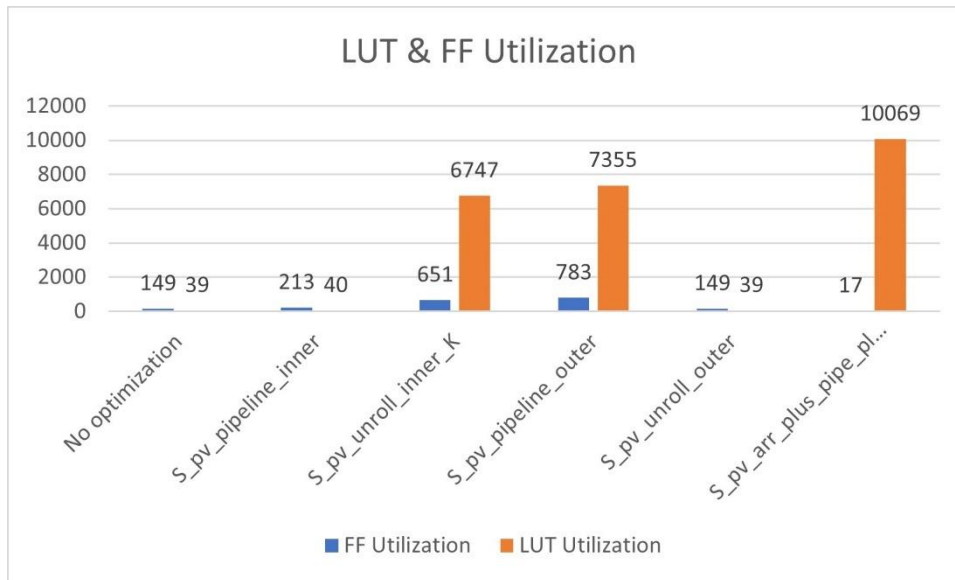


Figure 5.33: Resource Utilization graph of the `polyveck_add()` function with the implementation of various optimizations

For further optimizations we decided to combine the `UNROLL`, `PIPELINE` and `ARRAY_PARTITION` directive, by changing each time the factor `K` that the loop will be unrolled and that the array will be partitioned. So, once again we will be naming the solutions in `t`. It is important to clarify that the arrays will be partitioned in cyclic manner because when unrolling a loop, for example by a factor of 2, the code “transforms” as shown in, so in order have simultaneously access to the memories, the arrays must be partition as in Figure 5.35.

```

for(int i = 0; i < X; i++) {
    pragma HLS unroll factor=2
    a[i] = b[i] + c[i];
}

```

➔

```

for(int i = 0; i < X; i += 2) {
    a[i] = b[i] + c[i];
    if (i+1 >= X) break;
    a[i+1] = b[i+1] + c[i+1];
}

```

Figure 5.34: Example of loop unrolling by a factor of two

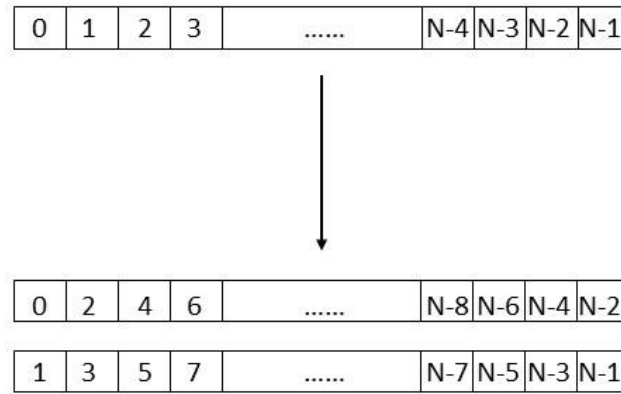


Figure 5.35: Array partitioning in a cyclic manner

Solution Names	Techniques
S_pv_unroll_part2_Pipeline	Unrolls the loop of poly_add() by a factor of 2, Partitions the arrays by a factor of 2 and pipelines the loop of poly_add()
S_pv_unroll_part4_Pipeline	Unrolls the loop of poly_add() by a factor of 4, Partitions the arrays by a factor of 4 and pipelines the loop of poly_add()
S_pv_unroll_part8_Pipeline	Unrolls the loop of poly_add() by a factor of 8, Partitions the arrays by a factor of 8 and pipelines the loop of poly_add()
S_pv_unroll_part16_Pipeline	Unrolls the loop of poly_add() by a factor of 16, Partitions the arrays by a factor of 16 and pipelines the loop of poly_add()
S_pv_unroll_part32_Pipeline	Unrolls the loop of poly_add() by a factor of 32, Partitions the arrays by a factor of 32 and pipelines the loop of poly_add()
S_pv_unroll_part64_Pipeline	Unrolls the loop of poly_add() by a factor of 64, Partitions the arrays by a factor of 64 and pipelines the loop of poly_add()
S_pv_unroll_part128_Pipeline	Unrolls the loop of poly_add() by a factor of 128, Partitions the arrays by a factor of 128 and pipelines the loop of poly_add()

Table 5.15: Naming of further optimizations

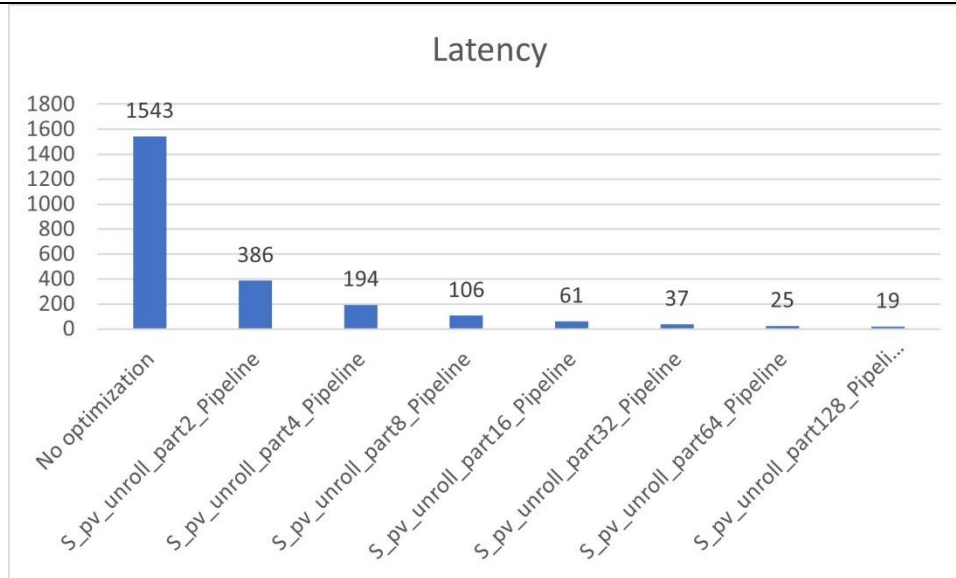


Figure 5.36: Latency graph of the `polyveck_add()` function with the implementation of further optimizations

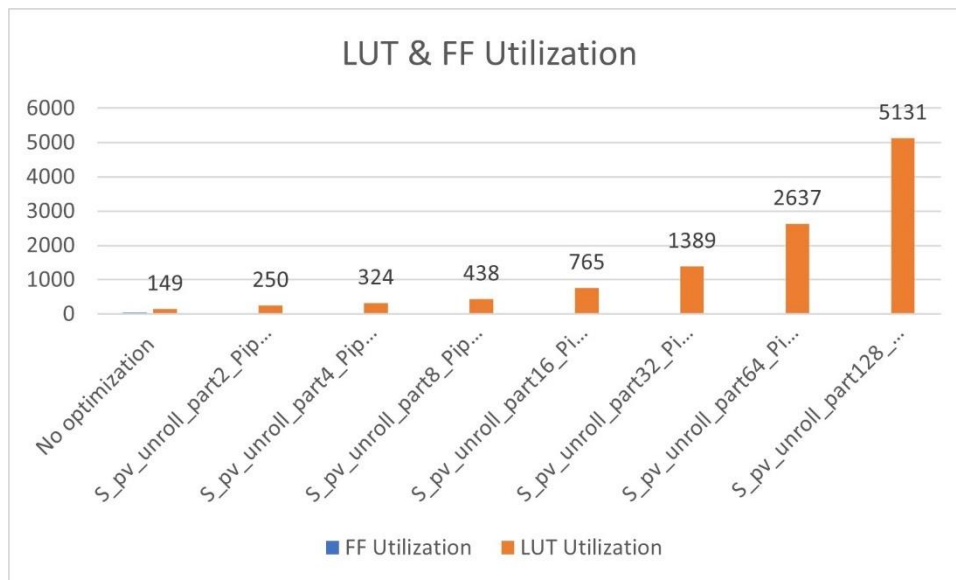


Figure 5.37: LUT and FF Utilization graph of the `polyveck_add()` function with the implementation of further optimizations

These further optimizations, except for the `S_pv_unroll_part128_Pipeline` due to their big resource consumption, were applied to all the other functions that were mentioned before, and the results of latency and resource utilization can be seen in the figures below for some of the functions separately (see Figure 5.38, Figure 5.39, Figure 5.40, Figure 5.41, Figure 5.42, Figure 5.43, Figure 5.44 and Figure 5.45).

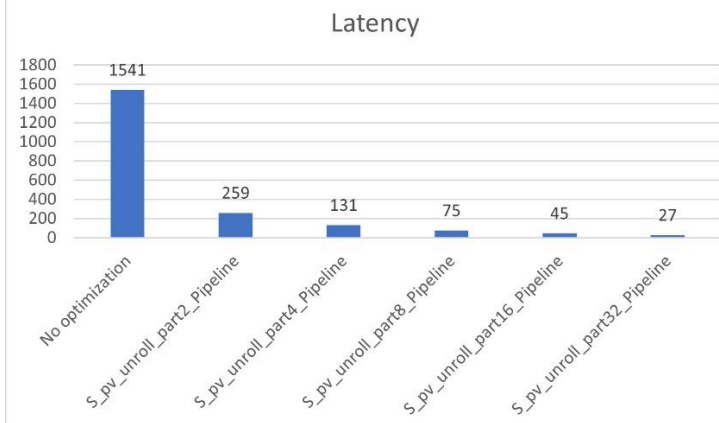


Figure 5.38: Latency graph of the `polyvecl_freeze()` function with the implementation of further optimizations

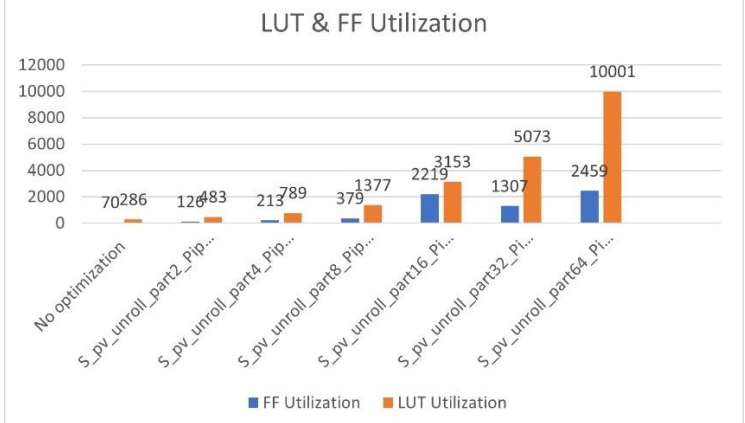


Figure 5.39: LUT and FF Utilization graph of the `polyvecl_freeze()` function with the implementation of further optimizations

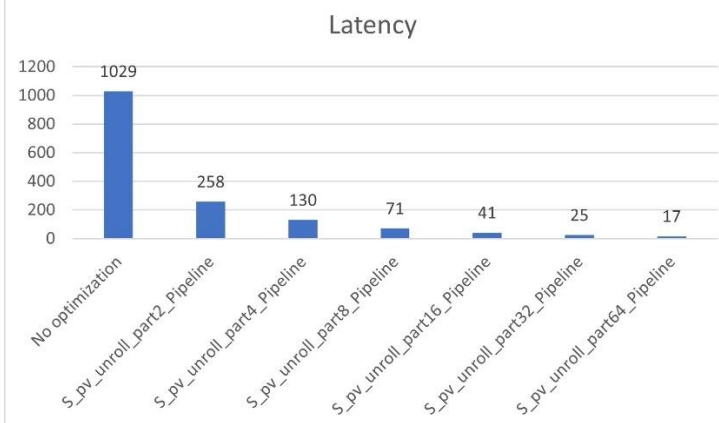


Figure 5.40: Latency graph of the `polyvecl_add()` function with the implementation of further optimizations

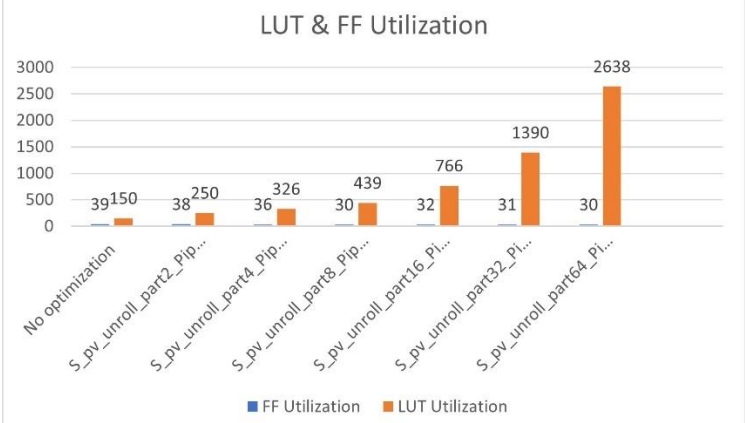


Figure 5.41: LUT and FF Utilization graph of the `polyvecl_add()` function with the implementation of further optimizations

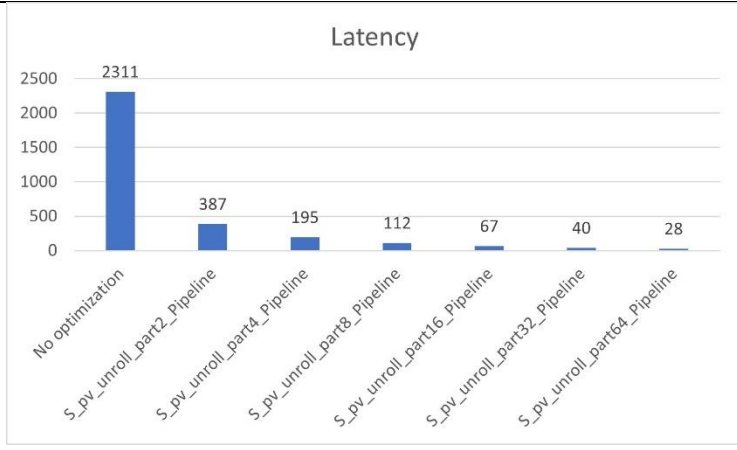


Figure 5.42: Latency graph of the `polyveck_freeze()` function with the implementation of further optimizations

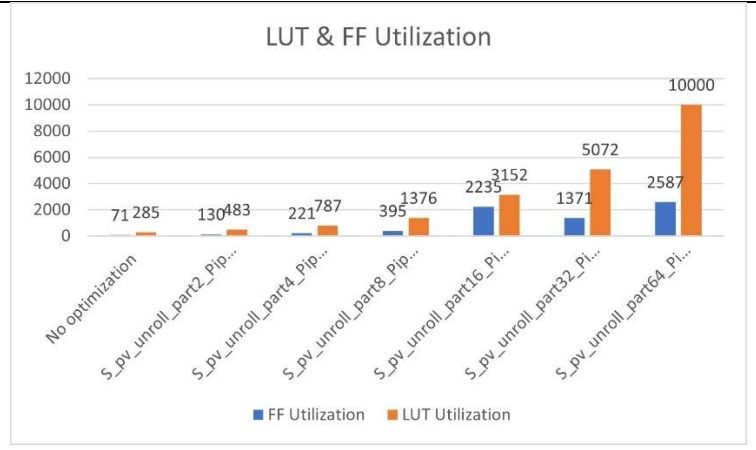


Figure 5.43: LUT and FF Utilization graph of the `polyveck_freeze()` function with the implementation of further optimizations

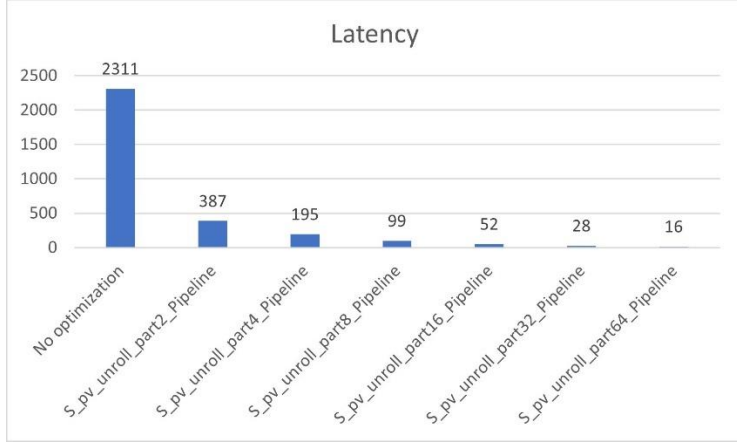


Figure 5.44: Latency graph of the `polyveck_power2round()` function with the implementation of further optimizations

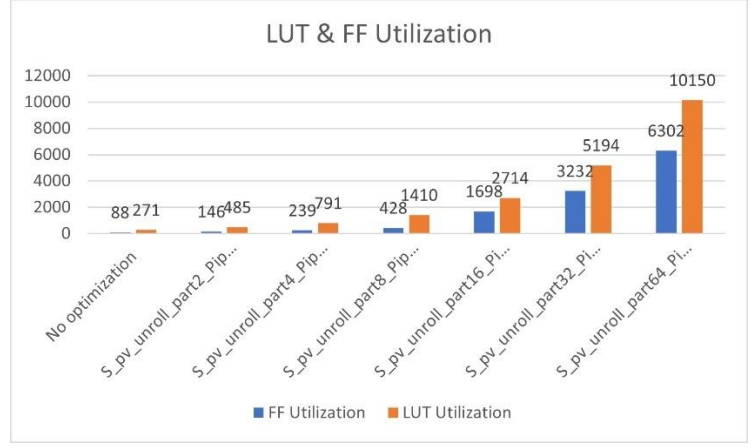


Figure 5.45: LUT and FF Utilization graph of the `polyveck_power2round()` function with the implementation of further optimizations

5.6 Optimization of `polyvecl_pointwise_acc_im()` function

The `polyvecl_pointwise_acc_im()` function pointwise multiplies vectors of polynomials of length L and then accumulates by adding the polynomials in the resulting vector. At first, it computes the pointwise multiplication of the first block of memory of the input vectors u and v of 256 coefficients, resulting in w vector of 256 coefficient. Then it computes the multiplication of the next block of memory and adds it to w and repeats it for all L blocks. After the accumulation a reduction of the coefficients takes place. In other words, this function is important for the multiplication of the polynomials after the Number Theoretic Transformation and before inverting the NTT.

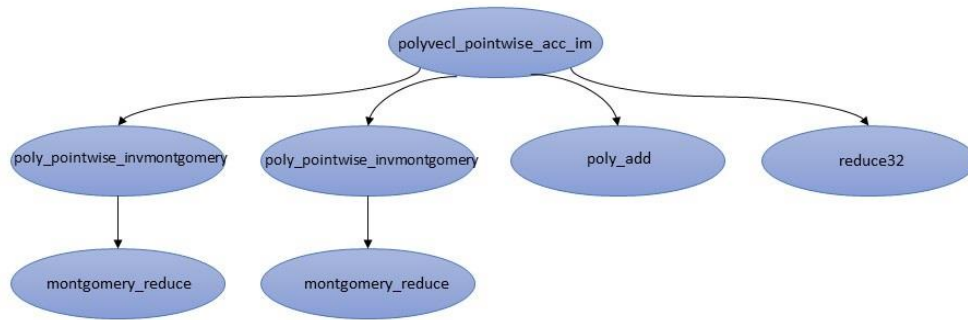


Figure 5.46: Hierarchy of the `polyvecl_pointwise_acc_im()` function

After analyzing the code and the block diagram of the , we figured out that by performing loop fusion we would be able to achieve a slightly better parallelism between the functions `poly_pointwise_invmontgomery()` and `poly_add()`, due to the fact that the `poly_add()` function will not have to wait the previous function to complete its 256 iterations, and less control overhead due to the reduction of a for loop. In order to achieve that, we had to perform function inlining manually and then to fuse the two loops into one.

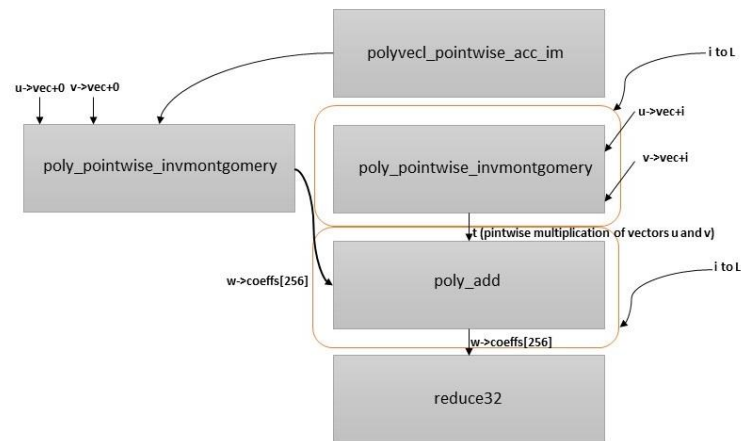


Figure 5.47: Block diagram of `polyvecl_pointwise_acc_im()` function

```

polyvecl_pointwise_acc_invmontgomery_label16:
    for(i = 1; i < L; ++i) {
        poly_pointwise_invmontgomery(&t, u->vec+i, v->vec+i);
        poly_add(w, w, &t);
    }
  
```

↓

```

polyvecl_pointwise_acc_invmontgomery_label16:
    for(i = 1; i < L; ++i) {
        polyvecl_pointwise_acc_invmontgomery_label10:
            for(j=0; j<N; j++) {
                t.coeffs[j] = montgomery_reduce((uint64_t)u->vec[i].coeffs[j]*v->vec[i].coeffs[j]);
                w->coeffs[j] = w->coeffs[j] + t.coeffs[j];
            }
    }
  
```

Figure 5.48: Manually function inlining and Loop Fusion

In the Figure 5. 49 results of the latency and the resource utilization are depicted, after the appliance of the loop fusion technique. This optimization did not affect the resource utilization at all. For a greater reduction of latency, we chose to apply the same optimization as in section 5.5, names as `S_pv_unroll_part4_Pipeline`. This means that we had to perform `ARRAY PARTITION` directive by a factor of 4 in a cyclic manner to the `u`, `w`, `v` and `t` vectors. Then we applied `UNROLL` directive by a factor of 4 to the inner loop of the newly fused loop, to the loop of the function `poly_pointwise_invmontgomery()` and to the loop that iterates 256 the `reduce32()` function, as shown in .

```

polyvecl_pointwise_acc_im
├── w
│   └── coeffs
├── u
│   └── vec
│       └── coeffs
├── v
│   └── vec
│       └── coeffs
└── #pragma HLS ARRAY_PARTITION variable=u->vec->coeffs cyclic factor=4 dim=1
    #pragma HLS ARRAY_PARTITION variable=v->vec->coeffs cyclic factor=4 dim=1
    #pragma HLS ARRAY_PARTITION variable=w->coeffs cyclic factor=4 dim=1
    #pragma HLS ARRAY_PARTITION variable=t.coeffs cyclic factor=4 dim=1

polyvecl_pointwise_acc_invmontgomery_label6
├── polyvecl_pointwise_acc_invmontgomery_label0
│   ├── # HLS UNROLL factor=4
│   └── # HLS PIPELINE
└── polyvecl_pointwise_acc_invmontgomery_label7
    ├── # HLS UNROLL factor=4
    └── # HLS PIPELINE

poly_pointwise_invmontgomery
├── poly_pointwise_invmontgomery_label30
│   ├── # HLS UNROLL factor=4
│   └── # HLS PIPELINE

```

Figure 5. 49:Appliance of `UNROLL`, `PIPELINE` and `ARRAY PARTITION` directive to the `polyvecl_pointwise_acc_im()` function

In Figure 5.50 we can see that the latency of this function has been reduced by a factor of **21**, and in Figure 5.51 we notice that the resource utilization has been increased by a factor of **3.9**, with the combination of the loop fusion and the `S_pv_unroll_part4_Pipeline` solution.

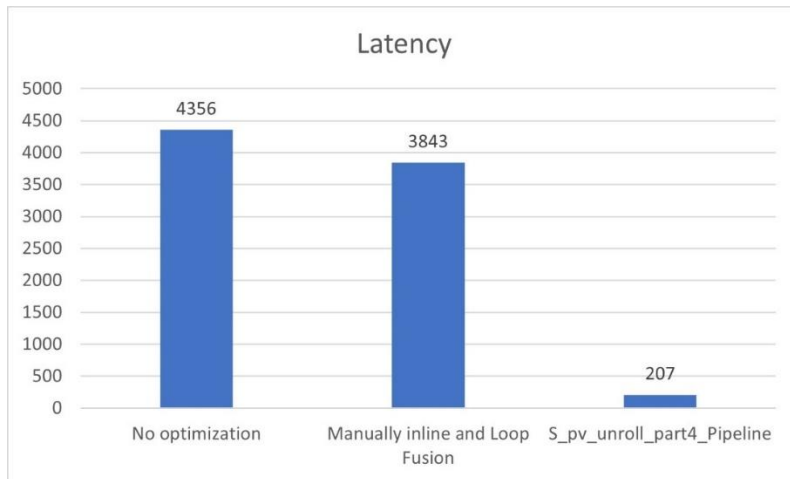


Figure 5.50: Latency graph of the `polyvecl_pointwise_acc_im()` function with the implementation of a) Loop Fusion b) `S_pv_unroll_part4_Pipeline` and Loop Fusion

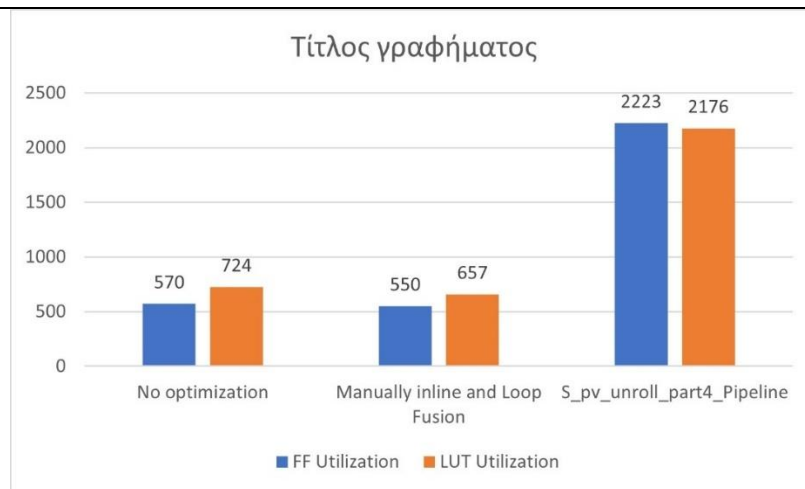


Figure 5.51: FF & LUT Utilization graph of the `polyvecl_pointwise_acc_im()` function with the implementation of a) Loop Fusion b) `S_pv_unroll_part4_Pipeline` and Loop Fusion

Chapter 6

Experimental results and comparisons

6.1 Combination of the optimal solutions to the Dilithium-CRYSTALS

In the context of this work, the implementation we achieved has demonstrated a significant reduction in terms of latency compared to the initial implementation. As shown in Figure 6.1, Figure 6.2 and Figure 6.3, we achieved a remarkable improvement of 3 to 4 times faster implementation. This improvement was achieved while maintaining almost the same resource utilization.

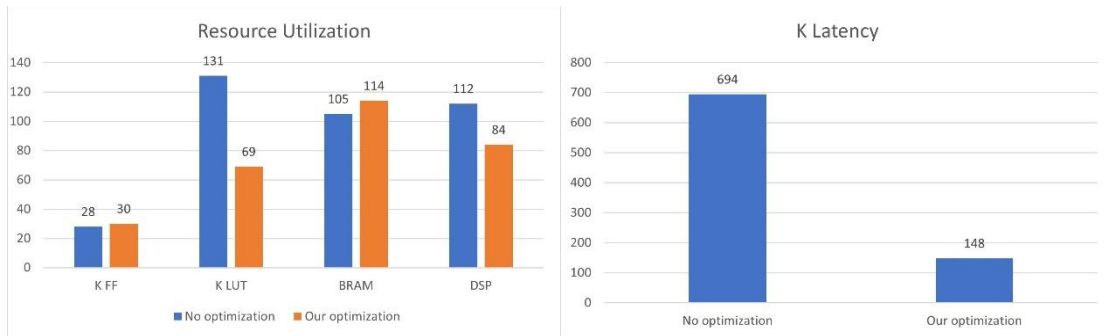


Figure 6.1 Comparison of Latency and Resource Utilization of `crypto_sign()` function of No optimization and Our optimization

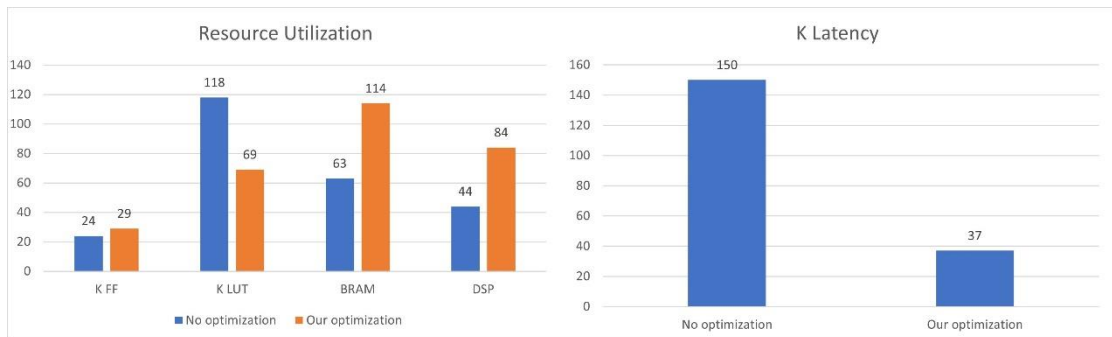


Figure 6.2: Comparison of Latency and Resource Utilization of `crypto_sign_keypair()` function of No optimization and Our optimization

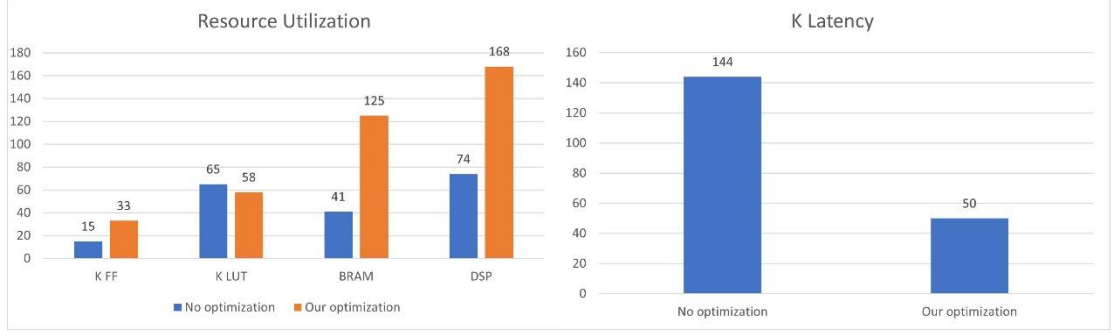


Figure 6.3: Comparison of Latency and Resource Utilization of `crypto_sign_open()` function of No optimization and Our optimization

6.2 Comparison of the results

In this chapter, we initiate a comprehensive comparative analysis between our optimizations, and the optimized implementations of those presented in this research paper [1]. At first, we will provide a brief overview of the research paper's optimization strategies and then we will illustrate the optimization techniques introduced in this thesis. Lastly, we will represent graphically the comparison of the implementations of the Dilithium_weak (Security level = 0).

In the figures below are depicted three implementations of the paper. The Base, the Unroll and the Pipeline. **1)** The Base implementation consists of the inline directives in the `fips202.c` file as we explained before. **2)** The Unroll implementation consists once again of the inline directives in the `fips202.c` file, unroll directives to the outer loop of the `ntt()` and `invntt_frominvmont()` by a factor of 2 and unroll directives to the loop in the `polyvec.c` file, which has the functions that were mentioned in section 5.5. **3)** Lastly, the Pipeline implementation consists of the inline directives in the `fips202.c` file, and pipeline directive to the functions of the `poly.c` file, which are the subfunctions of the `polyvec.c` file.

Our implementation consists of the following optimizations:

- *INLINE directives in the `fips202.c` file.*
- *Reconstruction of the `KeccakF1600_StatePermute()` function to effectuate one permutation per iteration.*
- *Loop Splitting, Loop Normalization and Loop Coalescing of the `invntt_frominvmont()` function.*
- *Supplementary Ram between every stage of the `invntt_frominvmont()` function.*
- *Allocation of one `montgomery_reduce()` function and four mul operations of the `invntt_frominvmont()` function.*
- *Loop Splitting, Loop Normalization and Loop Coalescing of the `ntt()` function.*
- *Supplementary Ram between every stage of the `ntt()` function.*
- *Allocation of one `montgomery_reduce()` function and four mul operations of the `invntt_frominvmont()` function.*
- *ARRAY_PARTITION by a factor of 4 of the inputs and outputs of the functions of the `polyvec.c`, UNROLL by a factor of 4 and pipeline the loops of the function of the `poly.c` file.*
- *Manually inline and Loop Fusion of the `poly_pointwise_invmontgomery()` and `poly_add()` functions in the `polyvec_pointwise_acc_im()` function.*

Experimental results and comparisons

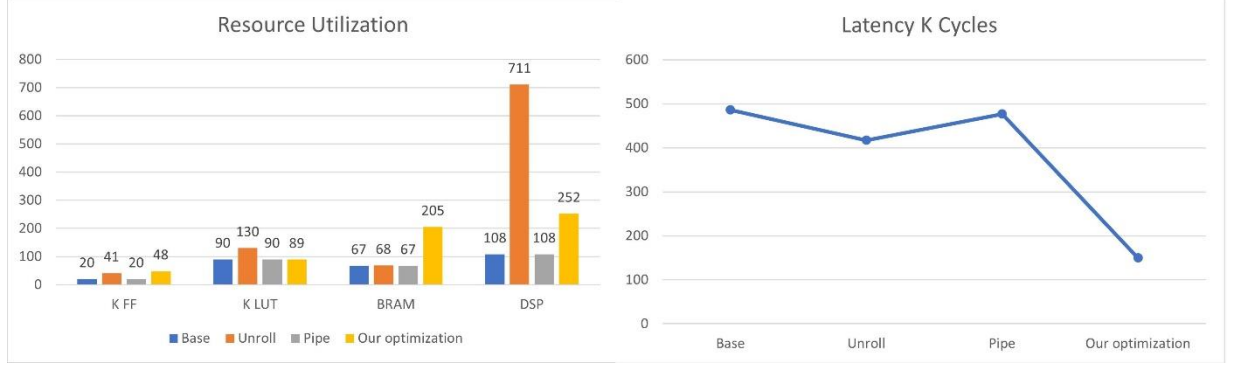


Figure 6.4: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the `crypto_sign()` function

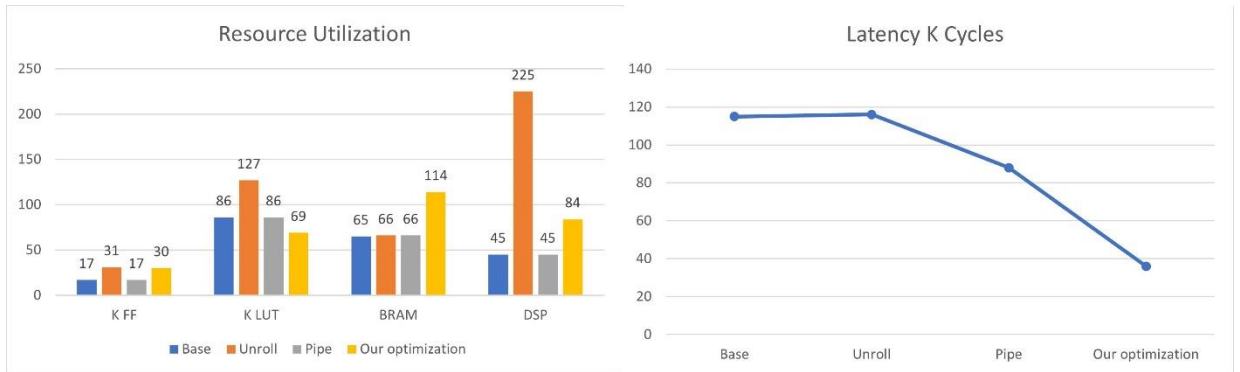


Figure 6.5: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the `crypto_sign_keypair()` function

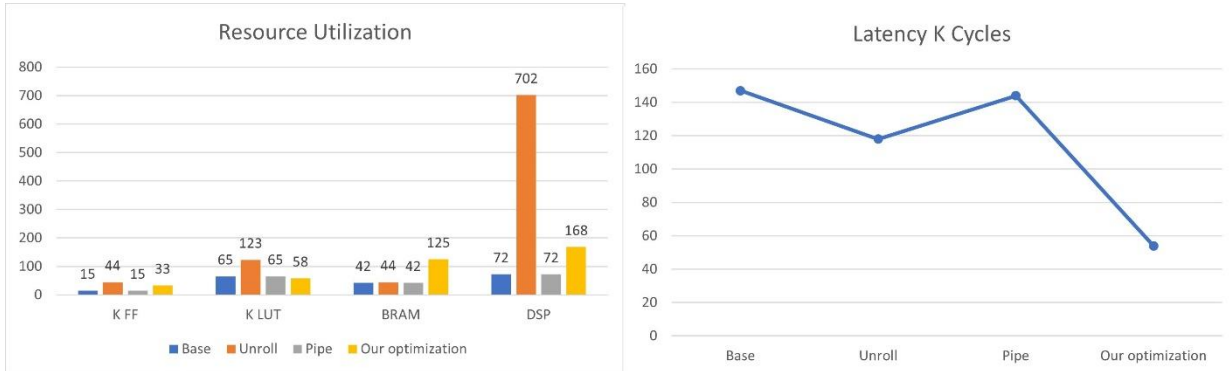


Figure 6.6: Latency and Resource Utilization of the a) Base, b) Unroll, c) Pipeline and d) our optimization of the Dilithium_Weak of the `crypto_sign_open()` function

In conclusion, in this comparative analysis, we have examined the optimizations presented in this thesis alongside those of the referenced research paper. After extensive study and analysis of the CRYSTALS-Dilithium algorithm, we were able to perform optimizations that made the three top functions many times faster than the referenced solutions. Specifically, `crypto_sign()`, `crypto_sign_open()` and `crypto_sign_keypair()`, with our solution, are 3 to 4 times faster.

Chapter 7

Conclusions and future research

7.1 Conclusions

In this thesis the CRYSTALS-Dilithium algorithm, a post quantum cryptography, is presented and implemented on Xilinx Artix-7 platform. The procedure that was followed, from the implementation and verification of the algorithm, until the stage analysis of the results, provides to the designer with an understanding of the capabilities that are offered through the Xilinx tool, as well as the development and improvement of the algorithm.

Xilinx Vivado HLS 2019.1 tool was used for the deployment of this thesis. This tool provides the option to implement an algorithm using a high-level language, which is translated into hardware description languages, such as Verilog and VHDL. This renders that the tool can also be used by non-experts hardware designers.

The abilities offered through this tool are countless, and the most significant is the automation of the process required for the RTL synthesis and the verification of the results through the testbench. This automation not only expedites the development cycle but also enhances the reliability of the results. A further benefit that this tool provides, is the capability to optimize the performance of the hardware, many times faster than designing with a hardware description language.

Of course, there are many difficulties and challenges for the designer. Describing an algorithm in a high-level language is not always easy and feasible to be translated in a hardware description language. It is worth noting that overcoming some of these challenges it often requires a deep understanding of both high-level language and the intricacies of hardware description language. Furthermore, there are many types of codes that are not synthesizable by the Vivado HLS tool. However, all these cons are addressable with the appropriate code redesign and exploitation steps of the tool options.

In conclusion, our exploration of the CRYSTALS-Dilithium algorithm on the Xilinx Artix-7 platform, through Xilinx Vivado HLS tool, has been a valuable effort. It has not only supported our understanding of post quantum cryptography, but most importantly, it has illustrated the potential of the FPGA acceleration through High-Level Synthesis.

7.2 Future research

In the context of this specific thesis. There could exist a future research and extensive exploration to introduce many optimizations. These optimizations could include a wide range of improvements, ranging from algorithmic improvements to hardware architecture improvements.

Specifically, one important optimization that could be implemented would be the reconstruction of the code of the Number Theoretic Transform algorithm. After the loop splitting and the insertion of supplementary RAMs we implemented, we could transform every stage into a subfunction, that would adhere to the one consumer-one producer model. This model is a dataflow pattern where data flows from a computational code that generates data (producer) to a computational code that consumes data (consumer). So after this code transformation, the *DATAFLOW* directive would be applicable the Number Theoretic Transform function.

BIBLIOGRAPHY

- [1] D. Soni. **CRYSTALS-Dilithium** in: **Hardware Architectures for Post-Quantum Digital Signature Schemes**. August 2021.
- [2] Chen Lily, Jordan Stephen, Liu Yi-Kai, Moody Dustin, Peralta Rene, Perner Ray, Smith-Tone Daniel. **Report on Post-Quantum Cryptography**. April 2016.
- [3] C. H. Ugwuishiwu, U. E. Orji, C. I. Ugwu, C. N. Asogwa. **An overview of Quantum Cryptography and Shor's Algorithm**. October 2020.
- [4] Logan O. Mailloux, Charlton D. Lewis II, Casey Riggs, Michael R. Grimaila. **Post-Quantum Cryptography What Advancements in Quantum Computing Mean for IT Professionals**. October 2016.
- [5] Erdem Ozcan, Aydin Aysu. **High-Level-Synthesis of Number-Theoretic Transform: A Case Study for Future Cryptosystems**. DOI 10.1109/LES.2019.2960457.
- [6] Duc Tri Nguyen, VVviet B. Dang and Kris Gaj. **A High-Level Synthesis Approach to the Software/Hardware Codesign of NTT-based Post-Quantum Cryptography Algorithms**. 2019.
- [7] Peter L. Montgomery. **Modular Multiplication Without Trial Division**. 1985.
- [8] C. K. Koc. **Montgomery Reduction with Even Modulus**. 1994.
- [9] **Sponge function – Wikipedia, The Free Encyclopedia**.
https://en.wikipedia.org/wiki/Sponge_function
- [10] Hassan II Mohammedia. **An optimal hardware implementation of the KECCAK hash function on virtex-5 FPGA**. 2019.
- [11] **In-depth Visual Breakdown of the SHA-3 Cryptographic Hashing Algorithm**.
<https://chemejon.io/sha-3-explained/>
- [12] **SHA-3 - Wikipedia** <https://en.wikipedia.org/wiki/SHA-3#Design>
- [13] **Keccak Team**. https://keccak.team/keccak_specs_summary.html
- [14] Federal Information Processing Standards. **Sha-3 Standard: Permutation-Based Hash and Extendable-Output Functions**. August 2015.
- [15] Bertoni Guido, Daemen Joan, Peters Michael, Van Assche Gilles. **The Keccak reference**. January 2011.
- [16] Zhao Cankun, Zhang Neng, Wang Hamming, Yang Bohan, Zhu Wenping, Li Zhengdong, Zhu Min, Yin Shouyi, Wei Shaojun, Liu Leibo. **A Compact and High-Performance Hardware Architecture for CRYSTALS-Dilithium**. 2022.
- [17] **Vivado Design Suite User Guide - High-Level Synthesis**. UG902. Rev. 2020.1. Xilinx. May 2021.
- [18] Mohamed Salah Azzaz, Abdelmadjd Maali, Redouane Kaibou, Ibrahim Kakouche, Mohamed Saad, Hocine Hamil. **FPGA HW/SW Codesign Approach for Real-time Image Processing Using HLS**. 2020
- [19] **UltraFast Vivado HLS Methodology Guide**. UG1197. Rev. 2020.1. Xilinx. June 2020.

- [20] **Vivado HLS Optimization Methodology Guide.** UG1270. Rev. 2017.4. Xilinx. December 2017.
- [21] **Xilinx. pragma HLS dependence.** <https://docs.xilinx.com/r/en-US/ug1399-vitis-hls/pragma-HLS-dependence>
- [22] Jao M.P. Cardoso, Jose Gabriel F. Coutinho, Pedro C. Diniz. **Embedded Computing for High Performance. Efficient Mapping of Computations Using Customization, Code transformations and Compilation.** 2017
- [23] Eric LaForest. **Survey of Loop Transformation Techniques.** March 2010.
- [24] David F. Bacon, Susan L. Graham, Oliver J. Sharp. **Compiler Transformations for High-Performance Computing.**
- [25] Andrea Guerieri, Gabriel Da Silva Marques, Francesco Regazzoni, Andres Upegui. **Design Exploration and Code Optimizations for FPGA-Based Post-Quantum Cryptography using High-Level Synthesis.** March 2022.

